

Physlib Monthly Report

1–31 August 2026

Contributors: aadarsh agarwal, Alex Zugheid, doxtor6, ereinhardt8, Fergus Munro, giuseppe.sorge, Gregory Loges, Jake Uribe, Joseph Tooby-Smith, m.hirotaka, nateabr, Nicolas Rouquette, Owen Parks, RAHUL PAMULA, RaunakChhatwal, Robby Sneiderman, Roman, RoomWithOutRoof, Sankalp Thakur, Tom Diem, Utkarsh Raj, WahajAyub, Will Blair, Zhi Kai Pong

Reviewers: Daniel Morrison, doxtor6, Gregory Loges, Joseph Tooby-Smith, malmortieus537, nateabr, Rodolfo R. Soldati

Generated 18 September 2026 from commit 12e16cf.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 August 2026 on branch `master` of [leanprover-community/physlib](#), between commits 471cf7f and 12e16cf. Over that period 24 contributors landed 81 commits, changing 18,434 lines across 393 files and adding 926 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	4
1.1	Contributors	4
1.2	Reviewers	4
1.3	Modified subfolders	5
1.4	New declarations by kind	6
2	Details by file	6
2.1	Physlib.ClassicalMechanics.Mass.MassUnit	6
2.2	Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Basic	6
2.3	Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Equilibria	14
2.4	Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Geometric.Basic	17
2.5	Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Geometric.PhysicalSpace	23
2.6	Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Geometric.Trajectory	25
2.7	Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Hamiltonian	27
2.8	Physlib.ClassicalMechanics.Pendulum.SimplePendulum.LiftInvariance	29
2.9	Physlib.ClassicalMechanics.Pendulum.SimplePendulum.SmallAngle	30
2.10	Physlib.ClassicalMechanics.RigidBody.Basic	36
2.11	Physlib.Electromagnetism.Charge.ChargeUnit	38
2.12	Physlib.Mathematics.Calculus.Gradient	38
2.13	Physlib.Mathematics.HasTemperateGrowth	39
2.14	Physlib.Mathematics.KroneckerDelta.Basic	39
2.15	Physlib.Mathematics.KroneckerDelta.Contraction	39

2.16	Physlib.Mathematics.LeviCivita.Basic	39
2.17	Physlib.Mathematics.List	41
2.18	Physlib.Mathematics.OneParameterSubgroups.Basic	41
2.19	Physlib.Mathematics.OneParameterSubgroups.Unitary	42
2.20	Physlib.Mathematics.RatComplexNum	44
2.21	Physlib.Mathematics.SpecialFunctions.EllipticIntegral	44
2.22	Physlib.Mathematics.Trigonometry.SinSq	47
2.23	Physlib.Meta.AllFilePaths	47
2.24	Physlib.Meta.Linters.DefsWithUnderscore	47
2.25	Physlib.Particles.NeutrinoPhysics.Basic	48
2.26	Physlib.Particles.StandardModel.HiggsBoson.Basic	49
2.27	Physlib.Particles.SuperSymmetry.SU5.FieldLabels	50
2.28	Physlib.QFT.PerturbationTheory.WickContraction.ExtractEquiv	51
2.29	Physlib.QFT.QED.AnomalyCancellation.Even.BasisLinear	51
2.30	Physlib.QFT.QED.AnomalyCancellation.Even.LineInCubic	56
2.31	Physlib.QFT.QED.AnomalyCancellation.Odd.BasisLinear	56
2.32	Physlib.QuantumMechanics.FiniteTarget	60
2.33	Physlib.QuantumMechanics.HarmonicOscillator.Basic	61
2.34	Physlib.QuantumMechanics.HarmonicOscillator.Eigenstates	61
2.35	Physlib.QuantumMechanics.HilbertSpaces.SpaceD.MomentumStates	62
2.36	Physlib.QuantumMechanics.HilbertSpaces.SpaceD.PositionStates	63
2.37	Physlib.QuantumMechanics.HilbertSpaces.TensorProducts.CompleteTensorProduct	63
2.38	Physlib.QuantumMechanics.Hydrogen.LaplaceRungeLenzVector	67
2.39	Physlib.QuantumMechanics.OperatorAlgebra.Basic	67
2.40	Physlib.QuantumMechanics.OperatorAlgebra.Dynamics.Automorphism	68
2.41	Physlib.QuantumMechanics.OperatorAlgebra.HilbertSpace	69
2.42	Physlib.QuantumMechanics.OperatorAlgebra.Observables.Jordan	69
2.43	Physlib.QuantumMechanics.OperatorAlgebra.Observables.Lie	71
2.44	Physlib.QuantumMechanics.PoschlTeller.Basic	72
2.45	Physlib.QuantumMechanics.QuantumSystem.Basic	73
2.46	Physlib.Relativity.LorentzGroup.Boosts.Axis	74
2.47	Physlib.Relativity.MinkowskiMatrix	75
2.48	Physlib.Relativity.PauliMatrices.Basic	76
2.49	Physlib.Relativity.PauliMatrices.Relations	77
2.50	Physlib.Relativity.PauliMatrices.SelfAdjoint	78
2.51	Physlib.Relativity.PauliMatrices.ToTensor	81
2.52	Physlib.Relativity.SL2C.AxisRotations	82
2.53	Physlib.Relativity.SL2C.Basic	83
2.54	Physlib.Relativity.Tensors.ComponentIdx.Contraction	83
2.55	Physlib.Relativity.Tensors.Contraction.CrossToSlot	83
2.56	Physlib.Relativity.Tensors.Contraction.SuccSuccAbove	84
2.57	Physlib.Relativity.Tensors.Evaluation	84
2.58	Physlib.Relativity.Tensors.LeviCivita.Basic	85
2.59	Physlib.Relativity.Tensors.LeviCivita.Complex	85
2.60	Physlib.Relativity.Tensors.LeviCivita.Contractions	86
2.61	Physlib.Relativity.Tensors.MetricTensor	87
2.62	Physlib.Relativity.Tensors.RealTensor.Contraction.CrossToEnd	87
2.63	Physlib.Relativity.Tensors.RealTensor.Metrics.Basic	88
2.64	Physlib.Relativity.Tensors.RealTensor.Units.Basic	88
2.65	Physlib.Relativity.Tensors.Reindexing	89
2.66	Physlib.Relativity.Tensors.UnitTensor	89

2.67	Physlib.SpaceAndTime.ReferenceFrame	90
2.68	Physlib.SpaceAndTime.Space.Derivatives.Curl	93
2.69	Physlib.SpaceAndTime.Space.LengthUnit	93
2.70	Physlib.SpaceAndTime.Space.Module	93
2.71	Physlib.SpaceAndTime.Time.Derivatives	93
2.72	Physlib.SpaceAndTime.Time.TimeUnit	94
2.73	Physlib.StatisticalMechanics.CanonicalEnsemble.Lemmas	94
2.74	Physlib.StatisticalMechanics.CanonicalEnsemble.TwoState	95
2.75	Physlib.Thermodynamics.Temperature.TemperatureUnits	95
2.76	Physlib.Units.Basic	95
2.77	Physlib.Units.Dimension	98
2.78	Physlib.Units.Exponent	99
2.79	Physlib.Units.ISQDimensionBase	104
2.80	Physlib.Units.LTMCTDimensionBase	104
2.81	Physlib.Units.ParametricDimensionExamples	105
2.82	Physlib.Units.ParametricUnits	105
2.83	Physlib.Units.SIUnitChoices	106
2.84	Physlib.Units.UnitDependent	111
2.85	Physlib.Units.UnitSystem	111
2.86	Physlib.Units.WithDim.Basic	114
2.87	PhyslibAlpha.Mathematics.LadderSystem.Basic	115
2.88	PhyslibAlpha.Mathematics.LadderSystem.Irreducibility	118
2.89	PhyslibAlpha.Mathematics.LadderSystem.OccupationBasis	121
2.90	PhyslibAlpha.Mathematics.LadderSystem.SymmetricPower	124
2.91	PhyslibAlpha.Mathematics.LadderSystem.Vacuum	124
2.92	PhyslibAlpha.QuantumMechanics.HarmonicOscillator.Basic	127
2.93	PhyslibAlpha.QuantumMechanics.HarmonicOscillator.LadderOperators	127
2.94	PhyslibAlpha.QuantumMechanics.HarmonicOscillator.Vacuum	129
2.95	PhyslibAlpha.QuantumMechanics.StinespringDilation	131
2.96	QuantumInfo.Channels.CPTP	132
2.97	QuantumInfo.ForMathlib.IsMaximalSelfAdjoint	133
2.98	QuantumInfo.ForMathlib.LinearEquiv	133
2.99	QuantumInfo.Operators.Unitary	133
2.100	scripts.APIMapIndex	134
2.101	scripts.get_cache	136
2.102	scripts.PhyslibAlpha.alphaFileImports	137
2.103	scripts.PhyslibAlpha.noAlphaImports	138

1 Introduction

During Aug 2026, 24 contributors submitted 81 commits that changed 18,434 lines (+16,441 / -1,993) across 393 files spanning 42 top-level areas of the repository. Of those, 75 files were newly added and 1 was removed outright. This report catalogues 926 newly added Lean declarations: 589 lemmas, 197 defs, 79 instances, 27 theorems, 18 structures, 11 abbrevs, 3 classes, and 2 inductives. We're delighted to recognize the following first-time contributors this month: aadarsh agarwal, ereinhardt8, Fergus Munro, m.hirotaka, Owen Parks, RAHUL PAMULA, RaunakChhatwal, Roman, RoomWithOutRoof, Sankalp Thakur, Tom Diem, Utkarsh Raj, WahajAyub, and Will Blair.

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
aadarsh agarwal	4,007	9
Alex Zughaid	541	3
doxtor6	227	1
ereinhardt8	26	1
Fergus Munro	170	2
giuseppe.sorge	106	1
Gregory Loges	973	5
Jake Uribe	72	1
Joseph Tooby-Smith	40	2
m.hirotaka	1,151	8
nateabr	389	1
Nicolas Rouquette	2,090	2
Owen Parks	2	1
RAHUL PAMULA	23	1
RaunakChhatwal	943	2
Robby Sneiderman	3,954	17
Roman	36	2
RoomWithOutRoof	2	1
Sankalp Thakur	153	6
Tom Diem	3,523	8
Utkarsh Raj	8	2
WahajAyub	142	1
Will Blair	25	2
Zhi Kai Pong	61	2

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Daniel Morrison	3

doxtor6	2
Gregory Loges	6
Joseph Tooby-Smith	72
malmortieus537	1
nateabr	9
Rodolfo R. Soldati	1

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
Physlib/ClassicalMechanics	22	+9	—	+3,522	-144
Physlib/Relativity	66	+8	—	+2,158	-209
Physlib/QuantumMechanics	36	+14	-1	+1,759	-210
Physlib/Units	20	+4	—	+1,595	-286
PhyslibAlpha/Mathematics	6	+5	—	+1,383	-6
Physlib/QFT	44	+2	—	+833	-428
Physlib/Mathematics	28	+7	—	+1,137	-80
PhyslibAlpha/QuantumMechanics	5	+3	—	+880	-119
Physlib/Particles	43	+2	—	+556	-153
Physlib/SpaceAndTime	15	+2	—	+503	-44
scripts	3	+2	—	+429	-0
.github/workflows	5	+2	—	+320	-16
QuantumInfo/ForMathlib	27	—	—	+144	-99
docs	4	+2	—	+220	-2
Physlib/CondensedMatter	8	+6	—	+149	-59
scripts/PhyslibAlpha	3	+2	—	+117	-5
Physlib/Thermodynamics	2	+1	—	+107	-6
Physlib/Electromagnetism	11	—	—	+79	-27
Physlib/Cosmology	1	+1	—	+98	-0
Physlib/Meta	2	+1	—	+84	-0
Physlib/StatisticalMechanics	5	—	—	+47	-17
QuantumInfo/Entropy	3	—	—	+40	-23
lake-cache.toml	1	+1	—	+44	-0
Physlib.lean	1	—	—	+42	-1
lakefile.toml	1	—	—	+33	-2
README.md	1	—	—	+30	-2
lake-manifest.json	1	—	—	+16	-16
QuantumInfo/Channels	6	—	—	+21	-8
Physlib/LatticeQFT	1	+1	—	+22	-0
QuantumInfo/Operators	1	—	—	+10	-10
QuantumInfo/ClassicalInfo	3	—	—	+17	-1
QuantumInfo/States	3	—	—	+14	-1
PhyslibAlpha/Particles	3	—	—	+7	-6
scripts/MetaPrograms	2	—	—	+6	-5
PhyslibAlpha.lean	1	—	—	+8	-0
QuantumInfo/ResourceTheory	3	—	—	+4	-2
AGENTS.md	1	—	—	+2	-2
PhyslibAlpha/SpaceAndTime	1	—	—	+2	-2
lean-toolchain	1	—	—	+1	-1
.codespellignore	1	—	—	+1	-0

Physlib/StringTheory	1	–	–	+0	-1
QuantumInfo/Measurements	1	–	–	+1	-0

1.4 New declarations by kind

Kind	Count
lemma	589
def	197
instance	79
theorem	27
structure	18
abbrev	11
class	3
inductive	2

2 Details by file

2.1 Physlib.ClassicalMechanics.Mass.MassUnit

[Physlib/ClassicalMechanics/Mass/MassUnit.lean](#) on GitHub.

lemma `div_mul_div`

[\[source\]](#)

The unit-ratio cocycle at $\mathbb{R}_{\geq 0}$ (the un-coerced form of `div_mul_div_coe`).

```
lemma div_mul_div (x y z : MassUnit) : (x / y) * (y / z) = x / z
```

2.2 Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Basic

[Physlib/ClassicalMechanics/Pendulum/SimplePendulum/Basic.lean](#) on GitHub.

structure `SimplePendulum`

[\[source\]](#)

The simple gravity pendulum is specified by the mass m of its bob, the length ℓ of its rod, and the gravitational acceleration g . All three are assumed to be positive. The configuration of the pendulum is the angle of the rod from the downward vertical.

```
structure SimplePendulum where
```

```
  m : ℝ
  ℓ : ℝ
  g : ℝ
  m_pos : 0 < m
  ℓ_pos : 0 < ℓ
  g_pos : 0 < g
```

lemma `m_ne_zero`

[\[source\]](#)

The mass of the bob is not equal to zero.

```
@[simp]
```

```
lemma m_ne_zero : S.m ≠ 0
```

lemma ℓ_ne_zero [\[source\]](#)

The length of the rod is not equal to zero.

```
@[simp]
lemma  $\ell\_ne\_zero$  :  $S.\ell \neq 0$ 
```

lemma g_ne_zero [\[source\]](#)

The gravitational acceleration is not equal to zero.

```
@[simp]
lemma  $g\_ne\_zero$  :  $S.g \neq 0$ 
```

def ω [\[source\]](#)

The angular frequency of the simple pendulum, ω , is defined as $\sqrt{g/\ell}$. It is the angular frequency of the small oscillations of the pendulum about the bottom of its swing.

```
noncomputable def  $\omega$  :  $\mathbb{R}$ 
```

lemma ω_pos [\[source\]](#)

The angular frequency of the simple pendulum is positive.

```
@[simp]
lemma  $\omega\_pos$  :  $0 < S.\omega$ 
```

lemma ω_ne_zero [\[source\]](#)

The angular frequency of the simple pendulum is not equal to zero.

```
lemma  $\omega\_ne\_zero$  :  $S.\omega \neq 0$ 
```

lemma ω_sq [\[source\]](#)

The square of the angular frequency of the simple pendulum is equal to g/ℓ .

```
lemma  $\omega\_sq$  :  $S.\omega^2 = S.g / S.\ell$ 
```

lemma $inverse_omega_sq$ [\[source\]](#)

The inverse of the square of the angular frequency of the simple pendulum is ℓ/g .

```
lemma  $inverse\_omega\_sq$  :  $(S.\omega^2)^{-1} = S.\ell / S.g$ 
```

def $inertia$ [\[source\]](#)

The moment of inertia of the simple pendulum about its pivot is $I = m \ell^2$, the moment of inertia of a point mass m at distance ℓ from the axis.

```
def  $inertia$  :  $\mathbb{R}$ 
```

lemma inertia_pos[\[source\]](#)

The moment of inertia of the simple pendulum is positive.

```
lemma inertia_pos : 0 < S.inertia
```

lemma inertia_ne_zero[\[source\]](#)

The moment of inertia of the simple pendulum is not equal to zero.

```
@[simp]
```

```
lemma inertia_ne_zero : S.inertia ≠ 0
```

lemma ω_sq_mul_inertia[\[source\]](#)

The square of the angular frequency times the moment of inertia is $m g \ell$, the coefficient appearing in the potential energy and in the torque. This is the identity by which the mass cancels from the equation of motion.

```
lemma ω_sq_mul_inertia : S.ω ^ 2 * S.inertia = S.m * S.g * S.ℓ
```

def kineticEnergy[\[source\]](#)

The kinetic energy of the simple pendulum along a lift θ of the angle is $\frac{1}{2} I \|\dot{\theta}\|^2$, where I is the moment of inertia about the pivot.

```
noncomputable def kineticEnergy (θ : Time → EuclideanSpace ℝ (Fin 1)) : Time → ℝ
```

def potentialEnergy[\[source\]](#)

The potential energy of the simple pendulum at the angle x is $m g \ell (1 - \cos(x))$, the work done against gravity in raising the bob from the bottom of the swing. It is normalized to vanish at the bottom.

```
noncomputable def potentialEnergy (x : EuclideanSpace ℝ (Fin 1)) : ℝ
```

def energy[\[source\]](#)

The energy of the simple pendulum is the kinetic energy plus the potential energy.

```
noncomputable def energy (θ : Time → EuclideanSpace ℝ (Fin 1)) : Time → ℝ
```

lemma kineticEnergy_eq[\[source\]](#)

The kinetic energy of the simple pendulum, written out.

```
lemma kineticEnergy_eq (θ : Time → EuclideanSpace ℝ (Fin 1)) :  
  S.kineticEnergy θ = fun t => (1 / (2 : ℝ)) * S.inertia * ⟨⟨∂ₜ θ t, ∂ₜ θ t⟩⟩_ℝ
```

lemma potentialEnergy_eq[\[source\]](#)

The potential energy of the simple pendulum, written out.

```
lemma potentialEnergy_eq (x : EuclideanSpace ℝ (Fin 1)) :  
  S.potentialEnergy x = S.m * S.g * S.ℓ * (1 - Real.cos (x 0))
```

lemma SimplePendulum.energy_eq[\[source\]](#)

The energy of the simple pendulum, written out.

```
lemma energy_eq (θ : Time → EuclideanSpace ℝ (Fin 1)) :
  S.energy θ = fun t => S.kineticEnergy θ t + S.potentialEnergy (θ t)
```

lemma potentialEnergy_nonneg[\[source\]](#)

The potential energy of the simple pendulum is non-negative, the bottom of the swing being the lowest point of the circle on which the bob moves.

```
lemma potentialEnergy_nonneg (x : EuclideanSpace ℝ (Fin 1)) : 0 ≤ S.potentialEnergy x
```

lemma potentialEnergy_le[\[source\]](#)

The potential energy of the simple pendulum is at most $2 m g \ell$, its value at the top of the swing.

```
lemma potentialEnergy_le (x : EuclideanSpace ℝ (Fin 1)) :
  S.potentialEnergy x ≤ 2 * (S.m * S.g * S.ℓ)
```

lemma potentialEnergy_eq_zero_iff[\[source\]](#)

The potential energy of the simple pendulum vanishes exactly when the cosine of the angle is equal to 1, that is exactly at the bottom of the swing.

```
lemma potentialEnergy_eq_zero_iff (x : EuclideanSpace ℝ (Fin 1)) :
  S.potentialEnergy x = 0 ↔ Real.cos (x 0) = 1
```

lemma potentialEnergy_contDiff[\[source\]](#)

The potential energy of the simple pendulum is a smooth function of the angle.

```
@[fun_prop]
lemma potentialEnergy_contDiff (n : WithTop ℕ) : ContDiff ℝ n S.potentialEnergy
```

lemma differentiable_potentialEnergy[\[source\]](#)

The potential energy of the simple pendulum is a differentiable function of the angle. This is differentiability in the angle; for differentiability in time along a smooth lift of the angle see `potentialEnergy_differentiable`.

```
@[fun_prop]
lemma differentiable_potentialEnergy : Differentiable ℝ S.potentialEnergy
```

lemma gradient_potentialEnergy[\[source\]](#)

The gradient of the potential energy of the simple pendulum is $m g \ell \sin \theta$ times the unit vector of the angular coordinate.

```
lemma gradient_potentialEnergy (x : EuclideanSpace ℝ (Fin 1)) :
  gradient S.potentialEnergy x =
    (S.m * S.g * S.ℓ * Real.sin (x 0)) • EuclideanSpace.single 0 1
```

lemma kineticEnergy_differentiable[\[source\]](#)

Along a smooth lift of the angle the kinetic energy is differentiable in time.

```
@[fun_prop]
lemma kineticEnergy_differentiable (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ)
:
  Differentiable ℝ (S.kineticEnergy θ)
```

lemma potentialEnergy_differentiable[\[source\]](#)

Along a smooth lift of the angle the potential energy is a differentiable function of the time. This is differentiability in time along the lift; for differentiability in the angle see differentiable_potentialEnergy.

```
@[fun_prop]
lemma potentialEnergy_differentiable (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ) :
  Differentiable ℝ (fun t => S.potentialEnergy (θ t))
```

lemma energy_differentiable[\[source\]](#)

Along a smooth lift of the angle the energy is differentiable in time.

```
@[fun_prop]
lemma energy_differentiable (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ) :
  Differentiable ℝ (S.energy θ)
```

lemma kineticEnergy_deriv[\[source\]](#)

The rate of change of the kinetic energy is the angular velocity paired with the angular momentum's rate of change, $\mathbf{I} \dot{\theta}$.

```
lemma kineticEnergy_deriv (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ) :
  ∂_t (S.kineticEnergy θ) = fun t => ⟨⟨∂_t θ t, S.inertia • ∂_t (∂_t θ) t⟩⟩_ℝ
```

lemma potentialEnergy_deriv[\[source\]](#)

The rate of change of the potential energy is the angular velocity paired with the gradient of the potential.

```
lemma potentialEnergy_deriv (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ) :
  ∂_t (fun t => S.potentialEnergy (θ t)) =
  fun t => ⟨⟨∂_t θ t, gradient S.potentialEnergy (θ t)⟩⟩_ℝ
```

lemma energy_deriv[\[source\]](#)

The rate of change of the energy is the angular velocity paired with the sum of $\mathbf{I} \dot{\theta}$ and the gradient of the potential; the equation of motion is exactly the vanishing of that sum.

```
lemma energy_deriv (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ) :
  ∂_t (S.energy θ) =
  fun t => ⟨⟨∂_t θ t, S.inertia • ∂_t (∂_t θ) t + gradient S.potentialEnergy (θ t)⟩⟩_ℝ
```

def lagrangian[\[source\]](#)

The Lagrangian of the simple pendulum, $L(t, \theta, \dot{\theta}) = \frac{1}{2} I \|\dot{\theta}\|^2 - V(\theta)$, the kinetic energy minus the potential energy as a function on phase space. It does not depend on the time.

```
@[nolint unusedArguments]
```

```
noncomputable def lagrangian (t : Time) (x v : EuclideanSpace ℝ (Fin 1)) : ℝ
```

lemma lagrangian_eq[\[source\]](#)

The Lagrangian of the simple pendulum, written out in the input data.

```
lemma lagrangian_eq :
```

```
S.lagrangian = fun _ x v =>
```

```
(1 / (2 : ℝ)) * S.inertia * ⟨⟨v, v⟩⟩_ℝ - S.m * S.g * S.ℓ * (1 - Real.cos (x 0))
```

lemma lagrangian_eq_kineticEnergy_sub_potentialEnergy[\[source\]](#)

Along a lift of the angle the Lagrangian of the simple pendulum is the kinetic energy minus the potential energy.

```
lemma lagrangian_eq_kineticEnergy_sub_potentialEnergy (t : Time)
```

```
(θ : Time → EuclideanSpace ℝ (Fin 1)) :
```

```
S.lagrangian t (θ t) (∂t θ t) = S.kineticEnergy θ t - S.potentialEnergy (θ t)
```

lemma contDiff_lagrangian[\[source\]](#)

The Lagrangian of the simple pendulum is a smooth function of the time, the angle and the angular velocity jointly.

```
@[fun_prop]
```

```
lemma contDiff_lagrangian (n : WithTop ℕ) : ContDiff ℝ n |S.lagrangian
```

lemma gradient_lagrangian_position_eq[\[source\]](#)

The gradient of the Lagrangian of the simple pendulum in the angle is minus the gradient of the potential energy, $-m g \ell \sin \theta$ times the unit vector of the angular coordinate.

```
lemma gradient_lagrangian_position_eq (t : Time) (x v : EuclideanSpace ℝ (Fin 1)) :
```

```
gradient (fun x => S.lagrangian t x v) x =
```

```
-((S.m * S.g * S.ℓ * Real.sin (x 0)) • EuclideanSpace.single 0 1)
```

lemma gradient_lagrangian_velocity_eq[\[source\]](#)

The gradient of the Lagrangian of the simple pendulum in the angular velocity is the angular momentum $I \dot{\theta}$ about the pivot.

```
lemma gradient_lagrangian_velocity_eq (t : Time) (x v : EuclideanSpace ℝ (Fin 1)) :
```

```
gradient (S.lagrangian t x) v = S.inertia • v
```

def torque[\[source\]](#)

The generalized force of the simple pendulum conjugate to the angle, that is the torque about the pivot, is minus the gradient of the potential energy, $\tau = -\partial V / \partial \theta$.

```
noncomputable def torque (x : EuclideanSpace ℝ (Fin 1)) : EuclideanSpace ℝ (Fin 1)
```

lemma torque_eq[\[source\]](#)

The torque of the simple pendulum is $-m g \ell \sin \theta$ times the unit vector of the angular coordinate. It is restoring near the bottom of the swing: for $|\theta| < \pi$ it opposes the displacement, and it vanishes both at the bottom and at the inverted position.

```
lemma torque_eq (x : EuclideanSpace ℝ (Fin 1)) :  
  S.torque x = -((S.m * S.g * S.ℓ * Real.sin (x 0)) • EuclideanSpace.single 0 1)
```

lemma torque_apply[\[source\]](#)

The single component of the torque of the simple pendulum is $-m g \ell \sin \theta$.

```
lemma torque_apply (x : EuclideanSpace ℝ (Fin 1)) :  
  S.torque x 0 = -(S.m * S.g * S.ℓ * Real.sin (x 0))
```

def EquationOfMotion[\[source\]](#)

The equation of motion of the simple pendulum: at every instant the rate of change $\dot{L}$ of the angular momentum about the pivot equals the torque $\tau(\theta)$ of gravity.

This pointwise relation, and not the vanishing of the variational derivative of the action, is the definition of the equation of motion here; see the discussion in section E. For a smooth lift of the angle the two agree, by `equationOfMotion_iff_gradLagrangian_zero` of section G.

```
def EquationOfMotion (θ : Time → EuclideanSpace ℝ (Fin 1)) : Prop
```

lemma equationOfMotion_iff_newtons_2nd_law[\[source\]](#)

The equation of motion of the simple pendulum with all of its terms on one side: at every instant the rate of change $\dot{L}$ of the angular momentum plus the gradient of the potential energy vanishes. This is the rotational form of Newton's second law, in the shape in which `DampedHarmonicOscillator` states its own; the sum on the left is exactly the combination that `energy_deriv` pairs with the velocity $\partial_t \theta$.

```
lemma equationOfMotion_iff_newtons_2nd_law (θ : Time → EuclideanSpace ℝ (Fin 1)) :  
  S.EquationOfMotion θ ↔  
    ∀ t, S.inertia • ∂t (∂t θ) t + gradient S.potentialEnergy (θ t) = 0
```

def IsSolution[\[source\]](#)

A solution of the simple pendulum is a smooth lift of the angle satisfying the equation of motion.

```
def IsSolution (θ : Time → EuclideanSpace ℝ (Fin 1)) : Prop
```

lemma contDiff[\[source\]](#)

A solution of the simple pendulum is smooth.

```
lemma IsSolution.contDiff {S : SimplePendulum} {θ : Time → EuclideanSpace ℝ (Fin 1)}
  (h : S.IsSolution θ) : ContDiff ℝ ∞ θ
```

lemma equationOfMotion[\[source\]](#)

A solution of the simple pendulum satisfies the equation of motion.

```
lemma IsSolution.equationOfMotion {S : SimplePendulum} {θ : Time → EuclideanSpace ℝ (Fin 1)}
  (h : S.IsSolution θ) : S.EquationOfMotion θ
```

lemma equationOfMotion_iff_scalar[\[source\]](#)

*The equation of motion of the simple pendulum in scalar form, $\theta'' + \omega^2 \sin \theta = 0$.
The mass has cancelled: only the angular frequency $\omega = \sqrt{g/\ell}$ survives.*

```
lemma equationOfMotion_iff_scalar (θ : Time → EuclideanSpace ℝ (Fin 1)) :
  S.EquationOfMotion θ ↔ ∀ t, ∂ₜ (∂ₜ θ) t θ + S.ω ^ 2 * Real.sin (θ t θ) = 0
```

lemma equationOfMotion_iff_of_eq_ω[\[source\]](#)

Two simple pendulums with the same angular frequency have the same angular equation of motion, and hence the same angular motions; in particular, changing only the mass does not affect the angular motion. An equal ω still permits different lengths, and so different trajectories of the bob in space.

```
lemma equationOfMotion_iff_of_eq_ω (S' : SimplePendulum) (h : S'.ω = S.ω)
  (θ : Time → EuclideanSpace ℝ (Fin 1)) :
  S'.EquationOfMotion θ ↔ S.EquationOfMotion θ
```

def gradLagrangian[\[source\]](#)

The variational derivative of the action of the simple pendulum, the action being the time integral of the Lagrangian along a lift of the angle.

```
noncomputable def gradLagrangian (θ : Time → EuclideanSpace ℝ (Fin 1)) :
  Time → EuclideanSpace ℝ (Fin 1)
```

lemma gradLagrangian_eq_eulerLagrangeOp[\[source\]](#)

For a smooth lift of the angle the variational derivative of the action of the simple pendulum is the Euler–Lagrange operator of its Lagrangian.

```
lemma gradLagrangian_eq_eulerLagrangeOp (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : ContDiff ℝ ∞ θ) :
  S.gradLagrangian θ = eulerLagrangeOp S.lagrangian θ
```

lemma gradLagrangian_eq_torque[\[source\]](#)

For a smooth lift of the angle the variational derivative of the action of the simple pendulum is the torque minus the rate of change $I \theta''$ of the angular momentum.

```
lemma gradLagrangian_eq_torque (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ) :
  S.gradLagrangian θ = fun t => S.torque (θ t) - S.inertia • ∂t (∂t θ) t
```

lemma equationOfMotion_iff_gradLagrangian_zero

[\[source\]](#)

For a smooth lift of the angle the equation of motion of the simple pendulum holds if and only if the variational derivative of the action vanishes: the smooth motions of the pendulum are the critical points of its action.

```
lemma equationOfMotion_iff_gradLagrangian_zero (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : ContDiff ℝ ∞ θ) :
  S.EquationOfMotion θ ↔ S.gradLagrangian θ = 0
```

lemma isSolution_iff

[\[source\]](#)

A lift of the angle is a solution of the simple pendulum if and only if it is smooth and the variational derivative of the action vanishes along it.

```
lemma isSolution_iff (θ : Time → EuclideanSpace ℝ (Fin 1)) :
  S.IsSolution θ ↔ ContDiff ℝ ∞ θ ∧ S.gradLagrangian θ = 0
```

lemma energy_conservation_of_equationOfMotion

[\[source\]](#)

Along a smooth lift of the angle satisfying the equation of motion the time derivative of the energy of the simple pendulum vanishes.

```
lemma energy_conservation_of_equationOfMotion (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : ContDiff ℝ ∞ θ) (h : S.EquationOfMotion θ) : ∂t (S.energy θ) = 0
```

lemma energy_conservation_of_equationOfMotion'

[\[source\]](#)

Along a smooth lift of the angle satisfying the equation of motion the energy of the simple pendulum at any time is equal to its initial value.

```
lemma energy_conservation_of_equationOfMotion' (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : ContDiff ℝ ∞ θ) (h : S.EquationOfMotion θ) (t : Time) :
  S.energy θ t = S.energy θ 0
```

lemma IsSolution.energy_eq

[\[source\]](#)

The energy of the simple pendulum along a solution at any time is equal to its initial value.

```
lemma IsSolution.energy_eq {S : SimplePendulum} {θ : Time → EuclideanSpace ℝ (Fin 1)}
  (h : S.IsSolution θ) (t : Time) : S.energy θ t = S.energy θ 0
```

2.3 Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Equilibria

[Physlib/ClassicalMechanics/Pendulum/SimplePendulum/Equilibria.lean](#) on GitHub.

lemma equationOfMotion_const_zero

[\[source\]](#)

The constant lift at the angle θ — the bob hanging at rest at the bottom of its swing — satisfies the equation of motion of the simple pendulum.

```
lemma equationOfMotion_const_zero :
  S.EquationOfMotion (fun _ => (θ : EuclideanSpace ℝ (Fin 1)))
```

lemma isSolution_const_zero

[\[source\]](#)

The hanging equilibrium is a solution of the simple pendulum: the constant lift at the angle θ is smooth and satisfies the equation of motion. It is the simplest explicit solution of the pendulum.

```
lemma isSolution_const_zero : S.IsSolution (fun _ => 0)
```

lemma equationOfMotion_const_pi

[\[source\]](#)

The constant lift at the angle π — the bob balanced directly above the pivot — satisfies the equation of motion of the simple pendulum.

```
lemma equationOfMotion_const_pi :
  S.EquationOfMotion (fun _ => EuclideanSpace.single 0 Real.pi)
```

lemma isSolution_const_pi

[\[source\]](#)

The inverted equilibrium is a solution of the simple pendulum: the constant lift at the angle π is smooth and satisfies the equation of motion.

```
lemma isSolution_const_pi : S.IsSolution (fun _ => EuclideanSpace.single 0 Real.pi)
```

lemma equationOfMotion_const_iff

[\[source\]](#)

A constant lift satisfies the equation of motion of the simple pendulum if and only if the sine of its angle vanishes — classically, the angles straight down and straight up: the constant solutions are exactly the equilibria.

```
lemma equationOfMotion_const_iff (x : EuclideanSpace ℝ (Fin 1)) :
  S.EquationOfMotion (fun _ => x) ↔ Real.sin (x 0) = 0
```

def separatrixEnergy

[\[source\]](#)

The separatrix energy of the simple pendulum is $2 m g \ell$, the energy of the inverted equilibrium. It is the threshold separating the two regimes of the motion, libration below it and rotation above it.

```
def separatrixEnergy : ℝ
```

lemma separatrixEnergy_eq

[\[source\]](#)

The separatrix energy of the simple pendulum, written out.

```
lemma separatrixEnergy_eq : S.separatrixEnergy = 2 * (S.m * S.g * S.ℓ)
```

lemma separatrixEnergy_pos[\[source\]](#)

The separatrix energy of the simple pendulum is positive.

```
lemma separatrixEnergy_pos : 0 < S.separatrixEnergy
```

lemma energy_const_pi[\[source\]](#)

The energy of the simple pendulum along the inverted equilibrium is the separatrix energy: the bob balanced at the top has no kinetic energy and the full potential energy $2 m g \ell$.

```
lemma energy_const_pi :  
  S.energy (fun _ => EuclideanSpace.single 0 Real.pi) = fun _ => S.separatrixEnergy
```

lemma kineticEnergy_nonneg[\[source\]](#)

The kinetic energy of the simple pendulum is non-negative along every lift of the angle.

```
lemma kineticEnergy_nonneg (θ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :  
  0 ≤ S.kineticEnergy θ t
```

lemma inertia_mul_inner_deriv_le[\[source\]](#)

The moment of inertia times the square of the angular speed, $I \dot{\theta}^2$, is at most twice the total energy, along every lift of the angle.

```
lemma inertia_mul_inner_deriv_le (θ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :  
  S.inertia * ⟨⟨∂ₜ θ t, ∂ₜ θ t⟩⟩_ℝ ≤ 2 * S.energy θ t
```

lemma potentialEnergy_le_energy[\[source\]](#)

The potential energy of the simple pendulum is at most the total energy along every lift of the angle.

```
lemma potentialEnergy_le_energy (θ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :  
  S.potentialEnergy (θ t) ≤ S.energy θ t
```

lemma neg_one_lt_cos_of_energy_lt[\[source\]](#)

Libration: along a smooth lift of the angle satisfying the equation of motion, with energy below the separatrix energy, the cosine of the angle stays strictly above -1 — the bob never reaches the top of the swing.

```
lemma neg_one_lt_cos_of_energy_lt (θ : Time → EuclideanSpace ℝ (Fin 1))  
  (hθ : ContDiff ℝ ∞ θ) (h : S.EquationOfMotion θ)  
  (hE : S.energy θ 0 < S.separatrixEnergy) (t : Time) : -1 < Real.cos (θ t 0)
```

lemma deriv_ne_zero_of_energy_gt[\[source\]](#)

Rotation: along a smooth lift of the angle satisfying the equation of motion, with energy above the separatrix energy, the angular velocity never vanishes — the bob never halts.

```

lemma deriv_ne_zero_of_energy_gt (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : ContDiff ℝ ∞ θ) (h : S.EquationOfMotion θ)
  (hE : S.separatrixEnergy < S.energy θ 0) (t : Time) : ∂t θ t ≠ 0

```

lemma potentialEnergy_eq_energy_of_deriv_eq_zero

[\[source\]](#)

Turning points: along a smooth lift of the angle satisfying the equation of motion, at an instant where the angular velocity vanishes, the potential energy equals the conserved total energy.

```

lemma potentialEnergy_eq_energy_of_deriv_eq_zero (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : ContDiff ℝ ∞ θ) (h : S.EquationOfMotion θ) (t : Time) (h0 : ∂t θ t = 0) :
  S.potentialEnergy (θ t) = S.energy θ 0

```

2.4 Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Geometric.Basic

[Physlib/ClassicalMechanics/Pendulum/SimplePendulum/Geometric/Basic.lean](#) on GitHub.

structure ConfigurationSpace

[\[source\]](#)

The configuration space of the planar simple pendulum: the angle of the rod from the downward vertical, modulo 2π .

```

structure ConfigurationSpace where
  angle : Real.Angle

```

lemma ext

[\[source\]](#)

Two configurations are equal precisely when their angles are equal.

```

@[ext]
lemma ext {p q : ConfigurationSpace} (h : p.angle = q.angle) : p = q

```

def angleEquiv

[\[source\]](#)

The identification of the configuration space with `Real.Angle`.

```

def angleEquiv : ConfigurationSpace ≈ Real.Angle where

```

instance instTopologicalSpace

[\[source\]](#)

The topology of the configuration space, induced from `Real.Angle`.

```

instance instTopologicalSpace : TopologicalSpace ConfigurationSpace

```

def angleHomeomorph

[\[source\]](#)

The identification with `Real.Angle` as a homeomorphism.

```

def angleHomeomorph : ConfigurationSpace ≈t Real.Angle where

```

def toCircle[\[source\]](#)

The point of the unit circle $e^{i\theta}$ corresponding to a configuration at angle θ .

```
def toCircle (q : ConfigurationSpace) : Circle
```

def circleHomeomorph[\[source\]](#)

The identification of the configuration space with the unit circle.

```
def circleHomeomorph : ConfigurationSpace ≈τ Circle
```

lemma circleHomeomorph_apply[\[source\]](#)

The identification with the unit circle is given by `ConfigurationSpace.toCircle`.

```
lemma circleHomeomorph_apply (q : ConfigurationSpace) : circleHomeomorph q = q.toCircle
```

def ofCircle[\[source\]](#)

The configuration corresponding to a point of the unit circle.

```
def ofCircle : Circle → ConfigurationSpace
```

lemma toCircle_ofCircle[\[source\]](#)

The point of the unit circle of the configuration attached to a point of the unit circle is that point.

```
@[simp]
```

```
lemma toCircle_ofCircle (z : Circle) : (ofCircle z).toCircle = z
```

lemma ofCircle_toCircle[\[source\]](#)

The configuration attached to the point of the unit circle of a configuration is that configuration.

```
@[simp]
```

```
lemma ofCircle_toCircle (q : ConfigurationSpace) : ofCircle q.toCircle = q
```

instance instT2Space[\[source\]](#)

The configuration space is Hausdorff, being homeomorphic to the unit circle.

```
instance instT2Space : T2Space ConfigurationSpace
```

instance instCompactSpace[\[source\]](#)

The configuration space is compact, being homeomorphic to the unit circle.

```
instance instCompactSpace : CompactSpace ConfigurationSpace
```

instance instSecondCountableTopology[\[source\]](#)

The configuration space is second countable, being homeomorphic to the unit circle.

```
instance instSecondCountableTopology : SecondCountableTopology ConfigurationSpace
```

instance instChartedSpace[\[source\]](#)

The charts of the configuration space: the identification with the unit circle followed by a chart of the circle.

```
instance instChartedSpace : ChartedSpace (EuclideanSpace  $\mathbb{R}$  (Fin 1)) ConfigurationSpace where
```

lemma chartAt_eq[\[source\]](#)

The chart at a configuration is the identification with the unit circle followed by the chart of the circle at the corresponding point.

```
lemma chartAt_eq (q : ConfigurationSpace) :  
  chartAt (EuclideanSpace  $\mathbb{R}$  (Fin 1)) q =  
    circleHomeomorph.toOpenPartialHomeomorph.trans  
      (chartAt (EuclideanSpace  $\mathbb{R}$  (Fin 1)) q.toCircle)
```

lemma chartAt_source[\[source\]](#)

The domain of the chart at a configuration is the preimage under the identification with the unit circle of the domain of the chart of the circle at the corresponding point.

```
lemma chartAt_source (q : ConfigurationSpace) :  
  (chartAt (EuclideanSpace  $\mathbb{R}$  (Fin 1)) q).source =  
    circleHomeomorph  $^{-1}$ ' (chartAt (EuclideanSpace  $\mathbb{R}$  (Fin 1)) q.toCircle).source
```

lemma chartAt_target[\[source\]](#)

The codomain of the chart at a configuration is the codomain of the chart of the circle at the corresponding point.

```
lemma chartAt_target (q : ConfigurationSpace) :  
  (chartAt (EuclideanSpace  $\mathbb{R}$  (Fin 1)) q).target =  
    (chartAt (EuclideanSpace  $\mathbb{R}$  (Fin 1)) q.toCircle).target
```

instance instIsManifold[\[source\]](#)

The configuration space is an analytic manifold: every change of charts is a change of charts of the unit circle.

```
instance instIsManifold : IsManifold ( $\mathcal{R}$  1)  $\omega$  ConfigurationSpace where
```

def ofAngle[\[source\]](#)

The configuration at angle θ (measured from the downward vertical).

```
def ofAngle ( $\theta$  :  $\mathbb{R}$ ) : ConfigurationSpace
```

lemma ofAngle_angle[\[source\]](#)

The angle of the configuration at angle θ is θ modulo 2π .

```
@[simp]
lemma ofAngle_angle (θ : ℝ) : (ofAngle θ).angle = θ
```

lemma ofAngle_add_two_pi[\[source\]](#)

Adding a full turn to the angle leaves the configuration unchanged.

```
lemma ofAngle_add_two_pi (θ : ℝ) : ofAngle (θ + 2 * Real.pi) = ofAngle θ
```

lemma ofAngle_periodic[\[source\]](#)

The angular lift is periodic with period 2π .

```
lemma ofAngle_periodic : Function.Periodic ofAngle (2 * Real.pi)
```

lemma ofAngle_eq_iff[\[source\]](#)

Two angles describe the same configuration exactly when they differ by a whole number of turns.

```
lemma ofAngle_eq_iff (θ₁ θ₂ : ℝ) :
  ofAngle θ₁ = ofAngle θ₂ ↔ ∃ n : ℤ, θ₂ = θ₁ + n * (2 * Real.pi)
```

lemma ofAngle_surjective[\[source\]](#)

Every configuration is the configuration at some real angle: the lift is surjective.

```
lemma ofAngle_surjective : Function.Surjective ofAngle
```

lemma continuous_ofAngle[\[source\]](#)

The angular lift is continuous.

```
@[fun_prop]
lemma continuous_ofAngle : Continuous ofAngle
```

lemma isCoveringMap_ofAngle[\[source\]](#)

The angular lift is a covering map.

```
lemma isCoveringMap_ofAngle : IsCoveringMap ofAngle
```

lemma toCircle_ofAngle[\[source\]](#)

The configuration at angle θ corresponds to the point $e^{i\theta}$ of the unit circle.

```
@[simp]
lemma toCircle_ofAngle (θ : ℝ) : (ofAngle θ).toCircle = Circle.exp θ
```

lemma ofCircle_circleExp[\[source\]](#)

The configuration of a point $e^{i\theta}$ of the unit circle is ofAngle θ .

```
@[simp]
```

```
lemma ofCircle_circleExp (θ : ℝ) : ofCircle (Circle.exp θ) = ofAngle θ
```

lemma contMDiff_ofAngle[\[source\]](#)

The angular lift is analytic: read in the charts pulled back from the circle it is Circle.exp.

```
lemma contMDiff_ofAngle : ContMDiff ℐ(ℝ, ℝ) (ℝ 1) ω ofAngle
```

def cos[\[source\]](#)

The cosine of the angle of a configuration.

```
def cos (q : ConfigurationSpace) : ℝ
```

def sin[\[source\]](#)

The sine of the angle of a configuration.

```
def sin (q : ConfigurationSpace) : ℝ
```

lemma cos_angle[\[source\]](#)

The cosine of a configuration is the cosine of its angle.

```
lemma cos_angle (q : ConfigurationSpace) : q.cos = Real.Angle.cos q.angle
```

lemma sin_angle[\[source\]](#)

The sine of a configuration is the sine of its angle.

```
lemma sin_angle (q : ConfigurationSpace) : q.sin = Real.Angle.sin q.angle
```

lemma cos_ofAngle[\[source\]](#)

The cosine of the configuration at angle θ is cos θ .

```
@[simp]
```

```
lemma cos_ofAngle (θ : ℝ) : (ofAngle θ).cos = Real.cos θ
```

lemma sin_ofAngle[\[source\]](#)

The sine of the configuration at angle θ is sin θ .

```
@[simp]
```

```
lemma sin_ofAngle (θ : ℝ) : (ofAngle θ).sin = Real.sin θ
```

lemma cos_sq_add_sin_sq[\[source\]](#)

The Pythagorean identity for the angle of a configuration.

```
lemma cos_sq_add_sin_sq (q : ConfigurationSpace) : q.cos ^ 2 + q.sin ^ 2 = 1
```

lemma continuous_cos[\[source\]](#)

The cosine of the angle depends continuously on the configuration.

```
@[fun_prop]
```

```
lemma continuous_cos : Continuous (cos : ConfigurationSpace → ℝ)
```

lemma continuous_sin[\[source\]](#)

The sine of the angle depends continuously on the configuration.

```
@[fun_prop]
```

```
lemma continuous_sin : Continuous (sin : ConfigurationSpace → ℝ)
```

def toSpace[\[source\]](#)

The position of the bob in the plane, for a rod of length ℓ . ℓ is not assumed positive; the bob is at distance $|\ell|$ from the pivot.

```
def toSpace (ℓ : ℝ) (q : ConfigurationSpace) : Space 2
```

lemma toSpace_apply_zero[\[source\]](#)

*The horizontal coordinate of the bob, $\ell * q.sin$.*

```
@[simp]
```

```
lemma toSpace_apply_zero (ℓ : ℝ) (q : ConfigurationSpace) :  
  toSpace ℓ q 0 = ℓ * q.sin
```

lemma toSpace_apply_one[\[source\]](#)

*The vertical coordinate of the bob, $-(\ell * q.cos)$.*

```
@[simp]
```

```
lemma toSpace_apply_one (ℓ : ℝ) (q : ConfigurationSpace) :  
  toSpace ℓ q 1 = -(\ell * q.cos)
```

lemma toSpace_ofAngle[\[source\]](#)

The position of the bob for the configuration at angle θ .

```
@[simp]
```

```
lemma toSpace_ofAngle (ℓ θ : ℝ) :  
  toSpace ℓ (ofAngle θ) = (![ℓ * Real.sin θ, -ℓ * Real.cos θ])
```

lemma toSpace_norm[\[source\]](#)

The rod-length constraint: the bob is at distance $|\ell|$ from the pivot.

```
@[simp]
lemma toSpace_norm (ℓ : ℝ) (q : ConfigurationSpace) : ||toSpace ℓ q|| = |ℓ|
```

lemma continuous_toSpace

[\[source\]](#)

The position of the bob depends continuously on the configuration.

```
@[fun_prop]
lemma continuous_toSpace (ℓ : ℝ) : Continuous (toSpace ℓ)
```

lemma toSpace_injective

[\[source\]](#)

For a rod of nonzero length the configuration is determined by the position of the bob.

```
lemma toSpace_injective {ℓ : ℝ} (hℓ : ℓ ≠ 0) : Function.Injective (toSpace ℓ)
```

lemma toSpace_isClosedEmbedding

[\[source\]](#)

For $\ell \neq 0$ the position map is a closed embedding of the configuration circle.

```
lemma toSpace_isClosedEmbedding {ℓ : ℝ} (hℓ : ℓ ≠ 0) : Topology.IsClosedEmbedding (toSpace ℓ)
```

2.5 Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Geometric.PhysicalSpace

[Physlib/ClassicalMechanics/Pendulum/SimplePendulum/Geometric/PhysicalSpace.lean](#) on GitHub.

def spaceTrajectory

[\[source\]](#)

The position of the bob in the plane along the trajectory described by a lift θ of the angle, for the pendulum S .

```
noncomputable def spaceTrajectory (θ : Time → EuclideanSpace ℝ (Fin 1)) : Time → Space 2
```

lemma spaceTrajectory_eq

[\[source\]](#)

Along the trajectory described by a lift θ of the angle the bob of the pendulum S is at $(\ell \sin (\theta \ t \ \theta), -\ell \cos (\theta \ t \ \theta))$.

```
lemma spaceTrajectory_eq (θ : Time → EuclideanSpace ℝ (Fin 1)) :
  S.spaceTrajectory θ = fun t => {![S.ℓ * Real.sin (θ t θ), -S.ℓ * Real.cos (θ t θ)]}
```

lemma spaceTrajectory_apply_zero

[\[source\]](#)

The horizontal position of the bob along a lifted trajectory is $\ell \sin (\theta \ t \ \theta)$.

```
lemma spaceTrajectory_apply_zero (θ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :
  S.spaceTrajectory θ t θ = S.ℓ * Real.sin (θ t θ)
```

lemma spaceTrajectory_apply_one

[\[source\]](#)

The vertical position of the bob along a lifted trajectory is $-\ell \cos (\theta \ t \ \theta)$, the pivot being the origin and the second axis pointing upwards.

```

lemma spaceTrajectory_apply_one (θ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :
  S.spaceTrajectory θ t 1 = -S.ℓ * Real.cos (θ t 0)

```

lemma norm_spaceTrajectory

[\[source\]](#)

The rod-length constraint along a lifted trajectory: the bob of the pendulum S stays at distance ℓ from the pivot, the length of the rod being positive.

```

lemma norm_spaceTrajectory (θ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :
  ||S.spaceTrajectory θ t|| = S.ℓ

```

lemma deriv_sin_coord

[\[source\]](#)

The time derivative of the sine of the angle read from a lift is the cosine of the angle times the angular velocity.

```

lemma deriv_sin_coord (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : Differentiable ℝ θ)
  (t : Time) : ∂t (fun s => Real.sin (θ s 0)) t = Real.cos (θ t 0) * (∂t θ t) 0

```

lemma deriv_cos_coord

[\[source\]](#)

The time derivative of the cosine of the angle read from a lift is minus the sine of the angle times the angular velocity.

```

lemma deriv_cos_coord (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : Differentiable ℝ θ)
  (t : Time) : ∂t (fun s => Real.cos (θ s 0)) t = -Real.sin (θ t 0) * (∂t θ t) 0

```

lemma differentiable_spaceTrajectory

[\[source\]](#)

Along a differentiable lift of the angle the position of the bob is differentiable in time.

```

@[fun_prop]
lemma differentiable_spaceTrajectory (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : Differentiable ℝ θ) : Differentiable ℝ (S.spaceTrajectory θ)

```

lemma deriv_spaceTrajectory

[\[source\]](#)

The velocity of the bob along a differentiable lift θ of the angle is $(\ell \cos (\theta t 0), \ell \sin (\theta t 0))$ times the angular velocity: the vector tangent to the circle of radius ℓ at the position of the bob.

```

lemma deriv_spaceTrajectory (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : Differentiable ℝ θ)
  (t : Time) :
  ∂t (S.spaceTrajectory θ) t =
    (![S.ℓ * Real.cos (θ t 0) * (∂t θ t) 0, S.ℓ * Real.sin (θ t 0) * (∂t θ t) 0])

```

lemma norm_sq_deriv_spaceTrajectory

[\[source\]](#)

The square of the speed of the bob along a differentiable lift of the angle is $\ell^2 \dot{\theta}^2$.

```

lemma norm_sq_deriv_spaceTrajectory (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : Differentiable ℝ θ) (t : Time) :
  ||∂t (S.spaceTrajectory θ) t|| ^ 2 = S.ℓ ^ 2 * ((∂t θ t) 0) ^ 2

```

lemma kineticEnergy_eq_space[\[source\]](#)

The chart kinetic energy of the pendulum along a differentiable lift of the angle is the kinetic energy of the bob in physical space.

```
lemma kineticEnergy_eq_space (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : Differentiable ℝ θ)
  (t : Time) :
  S.kineticEnergy θ t = (1 / (2 : ℝ)) * S.m * ||∂t (S.spaceTrajectory θ) t|| ^ 2
```

lemma potentialEnergy_eq_height[\[source\]](#)

The chart potential energy of the pendulum is the gravitational potential of the bob in physical space, normalized to vanish at the bottom of the swing: the height of the bob above the bottom is its vertical position plus ℓ .

```
lemma potentialEnergy_eq_height (θ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :
  S.potentialEnergy (θ t) = S.m * S.g * (S.spaceTrajectory θ t 1 + S.ℓ)
```

lemma lagrangian_eq_space[\[source\]](#)

The chart Lagrangian of the pendulum along a differentiable lift of the angle is the constrained Lagrangian of the bob in physical space: the kinetic energy of the point mass minus its gravitational potential.

```
lemma lagrangian_eq_space (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : Differentiable ℝ θ)
  (t : Time) :
  S.lagrangian t (θ t) (∂t θ t) =
    (1 / (2 : ℝ)) * S.m * ||∂t (S.spaceTrajectory θ) t|| ^ 2
    - S.m * S.g * (S.spaceTrajectory θ t 1 + S.ℓ)
```

lemma energy_eq_space[\[source\]](#)

The chart energy of the pendulum along a differentiable lift of the angle is the total energy of the bob in physical space: the kinetic energy of the point mass plus its gravitational potential.

```
lemma energy_eq_space (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : Differentiable ℝ θ)
  (t : Time) :
  S.energy θ t =
    (1 / (2 : ℝ)) * S.m * ||∂t (S.spaceTrajectory θ) t|| ^ 2
    + S.m * S.g * (S.spaceTrajectory θ t 1 + S.ℓ)
```

2.6 Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Geometric.Trajectory

[Physlib/ClassicalMechanics/Pendulum/SimplePendulum/Geometric/Trajectory.lean](#) on GitHub.

abbrev Trajectory[\[source\]](#)

A trajectory of the pendulum: a curve in the configuration space.

```
abbrev Trajectory
```

def ofLift[\[source\]](#)

The trajectory on the circle described by a lift θ of the angle.

```
def ofLift (θ : Time → EuclideanSpace ℝ (Fin 1)) : Trajectory
```

lemma ofLift_apply

[\[source\]](#)

At time t the trajectory described by a lift θ of the angle is the configuration at angle $\theta \ t \ \theta$.

```
lemma ofLift_apply (θ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :
  ofLift θ t = ConfigurationSpace.ofAngle (θ t θ)
```

lemma ofLift_eq_iff

[\[source\]](#)

Two lifts of the angle describe the same trajectory exactly when at each time they differ by a whole number of turns.

```
lemma ofLift_eq_iff (θ₁ θ₂ : Time → EuclideanSpace ℝ (Fin 1)) :
  ofLift θ₁ = ofLift θ₂ ↔ ∀ t, ∃ n : ℤ, θ₂ t θ = θ₁ t θ + n * (2 * Real.pi)
```

lemma ofLift_add_int_mul_two_pi

[\[source\]](#)

Shifting a lift of the angle by a whole number of turns leaves the described trajectory unchanged.

```
lemma ofLift_add_int_mul_two_pi (θ : Time → EuclideanSpace ℝ (Fin 1)) (n : ℤ) :
  ofLift (fun t => θ t + (n * (2 * Real.pi)) • EuclideanSpace.single 0 1) = ofLift θ
```

lemma continuous_ofLift

[\[source\]](#)

The trajectory described by a continuous lift of the angle is continuous.

```
lemma continuous_ofLift {θ : Time → EuclideanSpace ℝ (Fin 1)} (hθ : Continuous θ) :
  Continuous (ofLift θ)
```

lemma contMDiff_ofLift

[\[source\]](#)

The trajectory described by a C^n lift of the angle is a C^n curve in the configuration circle.

```
lemma contMDiff_ofLift {n : WithTop ℕ} {θ : Time → EuclideanSpace ℝ (Fin 1)}
  (hθ : ContDiff ℝ n θ) : ContMDiff ℐ(ℝ, Time) (ℝ 1) n (ofLift θ)
```

def toSpace

[\[source\]](#)

The physical position of the bob along a trajectory, over time, for a rod of length ℓ .

```
def toSpace (ℓ : ℝ) (γ : Trajectory) (t : Time) : Space 2
```

lemma toSpace_ofLift

[\[source\]](#)

Along the trajectory described by a lift θ of the angle the bob is at $(\ell \sin (\theta \ t \ \theta), -\ell \cos (\theta \ t \ \theta))$.

```
lemma toSpace_ofLift (ℓ : ℝ) (θ : Time → EuclideanSpace ℝ (Fin 1)) (t : Time) :
  toSpace ℓ (ofLift θ) t = (![ℓ * Real.sin (θ t θ), -ℓ * Real.cos (θ t θ)])
```

lemma norm_toSpace

[\[source\]](#)

The rod-length constraint along a trajectory: the bob stays at distance $|\ell|$ from the pivot.

```
lemma norm_toSpace (ℓ : ℝ) (γ : Trajectory) (t : Time) : ||toSpace ℓ γ t|| = |ℓ|
```

lemma continuous_toSpace

[\[source\]](#)

The physical position of the bob along a continuous trajectory depends continuously on the time.

```
lemma continuous_toSpace (ℓ : ℝ) {γ : Trajectory} (hγ : Continuous γ) :
  Continuous (toSpace ℓ γ)
```

2.7 Physlib.ClassicalMechanics.Pendulum.SimplePendulum.Hamiltonian

[Physlib/ClassicalMechanics/Pendulum/SimplePendulum/Hamiltonian.lean](#) on GitHub.

def canonicalMomentum

[\[source\]](#)

The canonical momentum of the simple pendulum, $p = \partial L / \partial \dot{\theta} = I \dot{\theta}$, as a linear equivalence between velocities and momenta in the angular chart.

```
noncomputable def canonicalMomentum (t : Time) (x : EuclideanSpace ℝ (Fin 1)) :
  EuclideanSpace ℝ (Fin 1) ≈[ℝ] EuclideanSpace ℝ (Fin 1) where
```

lemma canonicalMomentum_eq

[\[source\]](#)

The canonical momentum of the simple pendulum is the angular momentum $I \dot{\theta}$ about the pivot.

```
lemma canonicalMomentum_eq (t : Time) (x v : EuclideanSpace ℝ (Fin 1)) :
  S.canonicalMomentum t x v = S.inertia • v
```

def hamiltonian

[\[source\]](#)

The Hamiltonian of the simple pendulum as a function of time, momentum and angle: the Legendre transform $H(t, p, \theta) = \langle\langle p, \dot{\theta} \rangle\rangle - L(t, \theta, \dot{\theta})$ of the Lagrangian, the angular velocity $\dot{\theta}$ being recovered from the momentum by inverting the canonical momentum.

```
noncomputable def hamiltonian (t : Time) (p x : EuclideanSpace ℝ (Fin 1)) : ℝ
```

lemma hamiltonian_eq

[\[source\]](#)

The Hamiltonian of the simple pendulum is the kinetic energy written in terms of the momentum, $\frac{1}{2} (1/I) \|p\|^2$, plus the potential energy.

```

lemma hamiltonian_eq :
  S.hamiltonian = fun _ p x =>
    (1 / (2 : ℝ)) * (1 / S.inertia) * ⟨⟨p, p⟩⟩_ℝ + S.potentialEnergy x

```

lemma hamiltonian_contDiff

[\[source\]](#)

The Hamiltonian of the simple pendulum is a smooth function of the time, the momentum and the angle jointly.

@[fun_prop]

```

lemma hamiltonian_contDiff (n : WithTop ℕ∞) : ContDiff ℝ n |S.hamiltonian

```

lemma gradient_hamiltonian_position_eq

[\[source\]](#)

The gradient of the Hamiltonian of the simple pendulum in the angle is the gradient of the potential energy, that is minus the torque.

```

lemma gradient_hamiltonian_position_eq (t : Time) (x p : EuclideanSpace ℝ (Fin 1)) :
  gradient (S.hamiltonian t p) x = gradient S.potentialEnergy x

```

lemma gradient_hamiltonian_momentum_eq

[\[source\]](#)

The gradient of the Hamiltonian of the simple pendulum in the momentum is the angular velocity (1/I) p recovered from the momentum.

```

lemma gradient_hamiltonian_momentum_eq (t : Time) (x p : EuclideanSpace ℝ (Fin 1)) :
  gradient (S.hamiltonian t · x) p = (1 / S.inertia) • p

```

lemma hamiltonian_eq_energy

[\[source\]](#)

Along any lift of the angle, the Hamiltonian of the simple pendulum evaluated on the canonical momentum of the lift is the energy of the lift. This holds whether or not the lift satisfies the equation of motion.

```

lemma hamiltonian_eq_energy (θ : Time → EuclideanSpace ℝ (Fin 1)) :
  (fun t => S.hamiltonian t (S.canonicalMomentum t (θ t) (∂t θ t)) (θ t)) = S.energy θ

```

def hamiltonEqOp

[\[source\]](#)

The Hamilton-equations operator of the Hamiltonian of the simple pendulum, on momentum-angle phase space; its vanishing is equivalent to Hamilton's equations for the momentum and the angle.

```

noncomputable def hamiltonEqOp (p θ : Time → EuclideanSpace ℝ (Fin 1))

```

lemma equationOfMotion_iff_hamiltonEqOp_eq_zero

[\[source\]](#)

For a smooth lift of the angle the equation of motion of the simple pendulum holds if and only if Hamilton's equations hold for the lift and its canonical momentum, that is if and only if the Hamilton-equations operator vanishes on the pair.

```

lemma equationOfMotion_iff_hamiltonEqOp_eq_zero (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : ContDiff ℝ ∞ θ) :

```

```
S.EquationOfMotion θ ↔
S.hamiltonEqOp (fun t => S.canonicalMomentum t (θ t) (∂ₜ θ t)) θ = 0
```

lemma equationOfMotion_tfae[\[source\]](#)

For a smooth lift of the angle the following formulations of the dynamics of the simple pendulum are equivalent: 1. the equation of motion, the pointwise rotational Newton law balancing the rate of change of the angular momentum against the torque; 2. the scalar mass-independent form $\ddot{\theta} + \omega^2 \sin \theta = 0$ of the equation of motion; 3. Hamilton's equations for the lift and its canonical momentum, as the vanishing of the Hamilton-equations operator; 4. the Lagrangian variational principle, the vanishing of the variational gradient of the action integral of the Lagrangian, written through the variational calculus directly; the same principle, through the variational derivative of the action, is recorded standalone as `equationOfMotion_iff_gradLagrangian_zero` in the `Basic` module; 5. the Hamiltonian variational principle, the vanishing of the variational gradient of the phase-space action on the pair of the canonical momentum and the lift.

```
lemma equationOfMotion_tfae (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ) :
List.TFAE [S.EquationOfMotion θ,
  ∀ t, ∂ₜ (∂ₜ θ) t θ + S.ω ^ 2 * Real.sin (θ t θ) = 0,
  S.hamiltonEqOp (fun t => S.canonicalMomentum t (θ t) (∂ₜ θ t)) θ = 0,
  (δ (q' := θ), ∫ t, S.lagrangian t (q' t) (fderiv ℝ q' t 1)) = 0,
  (δ (pq' := fun t => (S.canonicalMomentum t (θ t) (∂ₜ θ t), θ t)),
    ∫ t, ⟨⟨(pq' t).1, ∂ₜ (Prod.snd ∘ pq') t⟩⟩_ℝ -
      S.hamiltonian t (pq' t).1 (pq' t).2) = 0]
```

2.8 Physlib.ClassicalMechanics.Pendulum.SimplePendulum.LiftInvariance

[Physlib/ClassicalMechanics/Pendulum/SimplePendulum/LiftInvariance.lean](#) on GitHub.

lemma potentialEnergy_add_int_mul_two_pi[\[source\]](#)

The potential energy of the simple pendulum is invariant under shifting the angle by a whole number of turns.

```
lemma potentialEnergy_add_int_mul_two_pi (x : EuclideanSpace ℝ (Fin 1)) (n : ℤ) :
S.potentialEnergy (x + (n * (2 * Real.pi))) • EuclideanSpace.single 0 1) =
S.potentialEnergy x
```

lemma torque_add_int_mul_two_pi[\[source\]](#)

The torque of the simple pendulum is invariant under shifting the angle by a whole number of turns.

```
lemma torque_add_int_mul_two_pi (x : EuclideanSpace ℝ (Fin 1)) (n : ℤ) :
S.torque (x + (n * (2 * Real.pi))) • EuclideanSpace.single 0 1) = S.torque x
```

lemma kineticEnergy_add_const[\[source\]](#)

The kinetic energy of the simple pendulum along a lift of the angle is invariant under shifting the lift by any constant: the shift drops out of the velocity.

```

lemma kineticEnergy_add_const (θ : Time → EuclideanSpace ℝ (Fin 1))
  (c : EuclideanSpace ℝ (Fin 1)) :
  S.kineticEnergy (fun t => θ t + c) = S.kineticEnergy θ

```

lemma energy_add_int_mul_two_pi

[\[source\]](#)

The energy of the simple pendulum along a lift of the angle is invariant under shifting the lift by a whole number of turns: A.1 supplies the invariance of the potential energy, and the velocity is unchanged by a constant shift.

```

lemma energy_add_int_mul_two_pi (θ : Time → EuclideanSpace ℝ (Fin 1)) (n : ℤ) :
  S.energy (fun t => θ t + (n * (2 * Real.pi)) • EuclideanSpace.single 0 1) =
  S.energy θ

```

lemma equationOfMotion_add_int_mul_two_pi

[\[source\]](#)

A lift of the angle shifted by a whole number of turns satisfies the equation of motion of the simple pendulum if and only if the lift itself does.

```

lemma equationOfMotion_add_int_mul_two_pi (θ : Time → EuclideanSpace ℝ (Fin 1)) (n : ℤ) :
  S.EquationOfMotion (fun t => θ t + (n * (2 * Real.pi)) • EuclideanSpace.single 0 1) ↔
  S.EquationOfMotion θ

```

lemma isSolution_add_int_mul_two_pi

[\[source\]](#)

A lift of the angle shifted by a whole number of turns is a solution of the simple pendulum if and only if the lift itself is.

```

lemma isSolution_add_int_mul_two_pi (θ : Time → EuclideanSpace ℝ (Fin 1)) (n : ℤ) :
  S.IsSolution (fun t => θ t + (n * (2 * Real.pi)) • EuclideanSpace.single 0 1) ↔
  S.IsSolution θ

```

lemma ofAngle_add_int_mul_two_pi_coord

[\[source\]](#)

A lift of the angle and its shift by a whole number of turns describe the same configuration of the simple pendulum.

```

lemma ofAngle_add_int_mul_two_pi_coord (x : EuclideanSpace ℝ (Fin 1)) (n : ℤ) :
  ConfigurationSpace.ofAngle
    ((x + (n * (2 * Real.pi)) • EuclideanSpace.single (0 : Fin 1) (1 : ℝ)) 0) =
  ConfigurationSpace.ofAngle (x 0)

```

2.9 Physlib.ClassicalMechanics.Pendulum.SimplePendulum.SmallAngle

[Physlib/ClassicalMechanics/Pendulum/SimplePendulum/SmallAngle.lean](#) on GitHub.

def toHarmonicOscillator

[\[source\]](#)

The harmonic oscillator to which the pendulum linearizes: mass $I = m \ell^2$, spring constant $m g \ell$.

```

noncomputable def toHarmonicOscillator : HarmonicOscillator where

```

lemma toHarmonicOscillator_m[\[source\]](#)

The mass of the harmonic oscillator associated to the simple pendulum is the moment of inertia $I = m \ell^2$ of the pendulum about its pivot.

@[simp]

```
lemma toHarmonicOscillator_m : S.toHarmonicOscillator.m = S.inertia
```

lemma toHarmonicOscillator_k[\[source\]](#)

The spring constant of the harmonic oscillator associated to the simple pendulum is $m g \ell$, the coefficient of the linearized torque.

@[simp]

```
lemma toHarmonicOscillator_k : S.toHarmonicOscillator.k = S.m * S.g * S.l
```

lemma toHarmonicOscillator_ω[\[source\]](#)

The angular frequency of the harmonic oscillator associated to the simple pendulum is the angular frequency $\omega = \sqrt{g/\ell}$ of the small oscillations of the pendulum.

```
lemma toHarmonicOscillator_ω : S.toHarmonicOscillator.ω = S.ω
```

def LinearizedEquationOfMotion[\[source\]](#)

The linearized equation of motion $\theta'' + \omega^2 \theta = 0$ of the simple pendulum, the small-angle form of the equation of motion, in which the torque is replaced by its linearization at the bottom of the swing.

```
def LinearizedEquationOfMotion (θ : Time → EuclideanSpace ℝ (Fin 1)) : Prop
```

lemma linearizedEquationOfMotion_iff_newton[\[source\]](#)

The linearized equation of motion in the rotational form of Newton's second law: at every instant the rate of change $I \theta''$ of the angular momentum equals the force $-m g \ell \theta$ of the associated harmonic oscillator. No smoothness is required: the two pointwise equations differ by the nonzero factor I .

```
lemma linearizedEquationOfMotion_iff_newton (θ : Time → EuclideanSpace ℝ (Fin 1)) :
```

```
S.LinearizedEquationOfMotion θ ↔
```

```
∀ t, S.inertia • ∂ₜ (∂ₜ θ) t = S.toHarmonicOscillator.force (θ t)
```

lemma linearizedEquationOfMotion_iff[\[source\]](#)

For a smooth lift of the angle, the linearized equation of motion of the simple pendulum is the equation of motion of the associated harmonic oscillator. The oscillator's equation of motion is the vanishing of the variational gradient of its action, which is totalized; smoothness is the regularity under which that variational description agrees with the pointwise one. The smoothness-free pointwise content is `linearizedEquationOfMotion_iff_newton`.

```
lemma linearizedEquationOfMotion_iff (θ : Time → EuclideanSpace ℝ (Fin 1))
```

```
(hθ : ContDiff ℝ ∞ θ) :
```

```
S.LinearizedEquationOfMotion θ ↔ S.toHarmonicOscillator.EquationOfMotion θ
```

lemma fderiv_torque_zero_apply[\[source\]](#)

The derivative of the torque at the hanging equilibrium is the force of the associated harmonic oscillator: $(D\tau)(\theta) \ v = -m \ g \ \ell \ v$. Linearizing the pendulum is differentiating its torque at the equilibrium.

```
lemma fderiv_torque_zero_apply (v : EuclideanSpace ℝ (Fin 1)) :
  fderiv ℝ S.torque 0 v = S.toHarmonicOscillator.force v
```

lemma linearizedEquationOfMotion_iff_fderiv_torque[\[source\]](#)

The linearized equation of motion is the equation of motion with the torque replaced by its derivative at the hanging equilibrium.

```
lemma linearizedEquationOfMotion_iff_fderiv_torque (θ : Time → EuclideanSpace ℝ (Fin 1)) :
  S.LinearizedEquationOfMotion θ ↔
  ∀ t, S.inertia • ∂t (∂t θ) t = fderiv ℝ S.torque 0 (θ t)
```

def smallAngleTrajectory[\[source\]](#)

The small-angle motion of the simple pendulum with initial angle $IC.x_0$ and initial angular velocity $IC.v_0$: the trajectory of the associated harmonic oscillator with the same initial conditions.

```
noncomputable def smallAngleTrajectory (IC : HarmonicOscillator.InitialConditions) :
  Time → EuclideanSpace ℝ (Fin 1)
```

lemma smallAngleTrajectory_contDiff[\[source\]](#)

The small-angle trajectories of the simple pendulum are smooth in time.

```
@[fun_prop]
lemma smallAngleTrajectory_contDiff (IC : HarmonicOscillator.InitialConditions)
  {n : WithTop ℕ∞} : ContDiff ℝ n (S.smallAngleTrajectory IC)
```

lemma smallAngleTrajectory_at_zero[\[source\]](#)

At time 0 the small-angle trajectory passes through its initial angle.

```
@[simp]
lemma smallAngleTrajectory_at_zero (IC : HarmonicOscillator.InitialConditions) :
  S.smallAngleTrajectory IC 0 = IC.x0
```

lemma smallAngleTrajectory_velocity_at_zero[\[source\]](#)

At time 0 the small-angle trajectory moves with its initial angular velocity.

```
@[simp]
lemma smallAngleTrajectory_velocity_at_zero (IC : HarmonicOscillator.InitialConditions) :
  ∂t (S.smallAngleTrajectory IC) 0 = IC.v0
```

lemma smallAngleTrajectory_eq[\[source\]](#)

The closed form of the small-angle motion: $\cos(\omega t) x_0 + (\sin(\omega t)/\omega) v_0$, the trajectory of the associated harmonic oscillator with its frequency read as the ω of the pendulum.

```
lemma smallAngleTrajectory_eq (IC : HarmonicOscillator.InitialConditions) :
  S.smallAngleTrajectory IC = fun t : Time =>
    Real.cos (S.ω * t.val) • IC.x₀ + (Real.sin (S.ω * t.val) / S.ω) • IC.v₀
```

lemma smallAngleTrajectory_linearizedEquationOfMotion

[\[source\]](#)

The small-angle trajectories satisfy the linearized equation of motion: for every choice of initial conditions the linearized equation has a smooth solution assuming them.

```
lemma smallAngleTrajectory_linearizedEquationOfMotion
  (IC : HarmonicOscillator.InitialConditions) :
  S.LinearizedEquationOfMotion (S.smallAngleTrajectory IC)
```

lemma linearized_unique

[\[source\]](#)

Uniqueness of the small-angle motions: a smooth solution of the linearized equation of motion is determined by its initial angle and initial angular velocity, being the small-angle trajectory of those initial conditions. This is the uniqueness theorem for the associated harmonic oscillator, transferred through linearizedEquationOfMotion_iff.

```
lemma linearized_unique (IC : HarmonicOscillator.InitialConditions)
  (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ)
  (h : S.LinearizedEquationOfMotion θ) (hθ : θ 0 = IC.x₀) (hv : ∂ₜ θ 0 = IC.v₀) :
  θ = S.smallAngleTrajectory IC
```

def releasedFromRest

[\[source\]](#)

The small-angle motion of the pendulum released from rest at initial angle θ_0 : the cosine $\theta_0 \cos(\omega t)$ of angular frequency ω .

```
noncomputable def releasedFromRest (θ₀ : ℝ) : Time → EuclideanSpace ℝ (Fin 1)
```

lemma releasedFromRest_eq

[\[source\]](#)

The motion released from rest at angle θ_0 is the small-angle trajectory of the initial conditions with initial angle θ_0 and zero initial angular velocity.

```
lemma releasedFromRest_eq (θ₀ : ℝ) :
  S.releasedFromRest θ₀ = S.smallAngleTrajectory (EuclideanSpace.single 0 θ₀, 0)
```

lemma releasedFromRest_at_zero

[\[source\]](#)

At time 0 the motion released from rest at angle θ_0 is at the angle θ_0 .

```
@[simp]
lemma releasedFromRest_at_zero (θ₀ : ℝ) :
  S.releasedFromRest θ₀ 0 = EuclideanSpace.single 0 θ₀
```

lemma releasedFromRest_velocity_at_zero[\[source\]](#)

The motion released from rest at angle θ_0 is genuinely released from rest: its angular velocity at time 0 vanishes.

@[simp]

```
lemma releasedFromRest_velocity_at_zero ( $\theta_0 : \mathbb{R}$ ) :  $\partial_t$  (S.releasedFromRest  $\theta_0$ ) 0 = 0
```

lemma releasedFromRest_linearizedEquationOfMotion[\[source\]](#)

The motion released from rest at angle θ_0 satisfies the linearized equation of motion.

```
lemma releasedFromRest_linearizedEquationOfMotion ( $\theta_0 : \mathbb{R}$ ) :  
  S.LinearizedEquationOfMotion (S.releasedFromRest  $\theta_0$ )
```

def smallAnglePeriod[\[source\]](#)

The period $2\pi \sqrt{\ell/g}$ of the small oscillations of the simple pendulum: the period of the associated harmonic oscillator. Within the linearization it does not depend on the amplitude — the small oscillations are isochronous, as derived by Huygens (1673).

```
noncomputable def smallAnglePeriod :  $\mathbb{R}$ 
```

lemma smallAnglePeriod_eq_two_pi_div_omega[\[source\]](#)

The period of the small oscillations is $2\pi/\omega$, one full circle of phase at the angular frequency ω of the small oscillations.

```
lemma smallAnglePeriod_eq_two_pi_div_omega : S.smallAnglePeriod = 2 * Real.pi / S.omega
```

lemma smallAnglePeriod_eq[\[source\]](#)

The closed form of the small-angle period: $2\pi \sqrt{\ell/g}$. Neither the mass of the bob nor the amplitude of the swing appears.

```
lemma smallAnglePeriod_eq : S.smallAnglePeriod = 2 * Real.pi *  $\sqrt{S.\ell / S.g}$ 
```

lemma smallAnglePeriod_pos[\[source\]](#)

The period of the small oscillations is positive.

```
lemma smallAnglePeriod_pos : 0 < S.smallAnglePeriod
```

lemma smallAngleTrajectory_periodic[\[source\]](#)

The small-angle trajectories of the simple pendulum are periodic with the small-angle period $2\pi \sqrt{\ell/g}$.

```
lemma smallAngleTrajectory_periodic (IC : HarmonicOscillator.InitialConditions) :  
  Function.Periodic (S.smallAngleTrajectory IC) (S.smallAnglePeriod : Time)
```

lemma releasedFromRest_periodic[\[source\]](#)

The motion released from rest at angle θ_0 is periodic with the small-angle period: after each time $2\pi \sqrt{\ell/g}$ the motion returns to the angle θ_0 with zero angular velocity.

```
lemma releasedFromRest_periodic ( $\theta_0 : \mathbb{R}$ ) :  
  Function.Periodic (S.releasedFromRest  $\theta_0$ ) (S.smallAnglePeriod : Time)
```

lemma smallAngleTrajectory_energy[\[source\]](#)

Along a small-angle trajectory the energy of the associated harmonic oscillator is the constant $\frac{1}{2} (I \|v_0\|^2 + m g \ell \|IC.x_0\|^2)$ fixed by the initial data: the rotational kinetic term of the initial angular velocity plus the potential term of the initial angle.

```
lemma smallAngleTrajectory_energy (IC : HarmonicOscillator.InitialConditions) :  
  S.toHarmonicOscillator.energy (S.smallAngleTrajectory IC) =  
    fun _ => 1 / 2 * (S.inertia * \|IC.v_0\|^2 + S.m * S.g * S.l * \|IC.x_0\|^2)
```

lemma torque_sub_toHarmonicOscillator_force[\[source\]](#)

The difference between the torque of the simple pendulum and the force of its associated harmonic oscillator is $m g \ell (\theta - \sin \theta)$ times the unit vector of the angular direction: exactly the term the linearization discards.

```
lemma torque_sub_toHarmonicOscillator_force ( $x : \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}$ ) :  
  S.torque x - S.toHarmonicOscillator.force x =  
    (S.m * S.g * S.l * (x 0 - Real.sin (x 0))) • EuclideanSpace.single 0 1
```

lemma norm_torque_sub_toHarmonicOscillator_force_le[\[source\]](#)

The normed form of the cubic bound: the torque of the simple pendulum differs from the force of its associated harmonic oscillator by at most $m g \ell \|\theta\|^3 / 6$ in norm.

```
lemma norm_torque_sub_toHarmonicOscillator_force_le ( $x : \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}$ ) :  
  \|S.torque x - S.toHarmonicOscillator.force x\| ≤ S.m * S.g * S.l * \|x\|^3 / 6
```

lemma abs_torque_add_linear_le[\[source\]](#)

The torque of the simple pendulum differs from its linearization $-m g \ell \theta$ by at most $m g \ell |\theta|^3 / 6$: the error of the small-angle approximation is cubic in the angle.

```
lemma abs_torque_add_linear_le ( $x : \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}$ ) :  
  |S.torque x 0 + S.m * S.g * S.l * x 0| ≤ S.m * S.g * S.l * |x 0|^3 / 6
```

lemma gradLagrangian_sub_toHarmonicOscillator[\[source\]](#)

Along a smooth lift of the angle, the variational gradients of the actions of the simple pendulum and of its associated harmonic oscillator differ exactly by the difference between the torque and the linearized force: the inertial terms cancel.

```
lemma gradLagrangian_sub_toHarmonicOscillator ( $\theta : \text{Time} \rightarrow \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}$ )  
  (h $\theta : \text{ContDiff } \mathbb{R} \infty \theta$ ) :  
  S.gradLagrangian  $\theta$  - S.toHarmonicOscillator.gradLagrangian  $\theta$  =  
    fun t => S.torque ( $\theta$  t) - S.toHarmonicOscillator.force ( $\theta$  t)
```

lemma norm_gradLagrangian_sub_toHarmonicOscillator_le[\[source\]](#)

The normed form: along a smooth lift of the angle the variational gradients of the two actions differ at every instant by at most $m g \ell \|\theta\|^3 / 6$ — the two actions have the same critical-point equation to cubic accuracy in the angle.

```
lemma norm_gradLagrangian_sub_toHarmonicOscillator_le (θ : Time → EuclideanSpace ℝ (Fin 1))
  (hθ : ContDiff ℝ ∞ θ) (t : Time) :
  ||S.gradLagrangian θ t - S.toHarmonicOscillator.gradLagrangian θ t|| ≤
  S.m * S.g * S.ℓ * ||θ t|| ^ 3 / 6
```

lemma norm_equationOfMotion_residual_le[\[source\]](#)

A motion satisfying the linearized equation of motion nearly solves the equation of motion of the pendulum itself: at every instant the residual $I \theta'' - \tau(\theta)$ has norm at most $m g \ell \|\theta\|^3 / 6$, cubically small for small angles.

```
lemma norm_equationOfMotion_residual_le (θ : Time → EuclideanSpace ℝ (Fin 1))
  (h : S.LinearizedEquationOfMotion θ) (t : Time) :
  ||S.inertia • ∂ₜ (∂ₜ θ) t - S.torque (θ t)|| ≤ S.m * S.g * S.ℓ * ||θ t|| ^ 3 / 6
```

lemma norm_gradLagrangian_le_of_linearizedEquationOfMotion[\[source\]](#)

The variational form of the residual: a smooth motion of the linearized dynamics is a near-critical point of the pendulum's own action — along it the variational gradient of the pendulum's action is cubically small in the angle.

```
lemma norm_gradLagrangian_le_of_linearizedEquationOfMotion
  (θ : Time → EuclideanSpace ℝ (Fin 1)) (hθ : ContDiff ℝ ∞ θ)
  (h : S.LinearizedEquationOfMotion θ) (t : Time) :
  ||S.gradLagrangian θ t|| ≤ S.m * S.g * S.ℓ * ||θ t|| ^ 3 / 6
```

2.10 Physlib.ClassicalMechanics.RigidBody.Basic

[Physlib/ClassicalMechanics/RigidBody/Basic.lean](#) on GitHub.

def inertiaTensorAbout[\[source\]](#)

The inertia tensor of the rigid body about the point p , that is with the moments taken about p rather than about the origin of the reference frame. Taking p to be the origin recovers `RigidBody.inertiaTensor`, see `RigidBody.inertiaTensorAbout_zero`.

```
noncomputable def inertiaTensorAbout {d : ℕ} (R : RigidBody d) (p : Space d) :
  Matrix (Fin d) (Fin d) ℝ
```

lemma inertiaTensorAbout_apply[\[source\]](#)

The (i, j) entry of the inertia tensor about p as a moment of the mass distribution.

```
lemma inertiaTensorAbout_apply {d : ℕ} (R : RigidBody d) (p : Space d) (i j : Fin d) :
  R.inertiaTensorAbout p i j = R.p (cmap (fun y => (if i = j then 1 else 0) *
    ∑ k : Fin d, (y k - p k) ^ 2 - (y i - p i) * (y j - p j)) (by fun_prop))
```

lemma inertiaTensorAbout_zero[\[source\]](#)

The inertia tensor about the origin is the inertia tensor.

```
@[simp]
lemma inertiaTensorAbout_zero {d : ℕ} (R : RigidBody d) :
  R.inertiaTensorAbout 0 = R.inertiaTensor
```

lemma inertiaTensorAbout_symmetric[\[source\]](#)

The inertia tensor about any point is symmetric.

```
lemma inertiaTensorAbout_symmetric {d : ℕ} (R : RigidBody d) (p : Space d) (i j : Fin d) :
  R.inertiaTensorAbout p i j = R.inertiaTensorAbout p j i
```

lemma inertiaTensorAbout_integrand_split[\[source\]](#)

The integrand of the inertia tensor about p splits into the integrand about the centre of mass, terms linear in the centred coordinates $y - c$, and a constant built from the displacement $c - p$.

```
lemma inertiaTensorAbout_integrand_split {d : ℕ} (R : RigidBody d) (p : Space d) (i j : Fin d) :
  :
  cmap (fun y => (if i = j then 1 else 0) * ∑ k : Fin d, (y k - p k) ^ 2
    - (y i - p i) * (y j - p j)) (by fun_prop)
  = cmap (fun y => (if i = j then 1 else 0) * ∑ k : Fin d, (y k - R.centerOfMass k) ^ 2
    - (y i - R.centerOfMass i) * (y j - R.centerOfMass j)) (by fun_prop)
    + ∑ k : Fin d, (2 * (if i = j then 1 else 0) * (R.centerOfMass k - p k)) •
      cmap (fun y => y k - R.centerOfMass k) (by fun_prop)
    - (R.centerOfMass j - p j) • cmap (fun y => y i - R.centerOfMass i) (by fun_prop)
    - (R.centerOfMass i - p i) • cmap (fun y => y j - R.centerOfMass j) (by fun_prop)
    + ((if i = j then 1 else 0) * ∑ k : Fin d, (R.centerOfMass k - p k) ^ 2
      - (R.centerOfMass i - p i) * (R.centerOfMass j - p j)) •
      (1 : C^T (ℐ(ℝ, Space d), Space d; ℐ(ℝ, ℝ), ℝ))
```

theorem inertiaTensorAbout_eq_centerOfMass_add_pointMass[\[source\]](#)

***The parallel-axis theorem**:* the inertia tensor about a point p is the inertia tensor about the centre of mass c plus the inertia tensor about p of a point particle carrying the total mass of the body and sitting at c , that is $M (|c - p|^2 1 - (c - p) \otimes (c - p))$.

```
theorem inertiaTensorAbout_eq_centerOfMass_add_pointMass {d : ℕ} (R : RigidBody d) (h : R.mass
  ≠ 0)
  (p : Space d) :
  R.inertiaTensorAbout p = R.inertiaTensorAbout R.centerOfMass
    + R.mass • (⟨⟨R.centerOfMass - p, R.centerOfMass - p⟩⟩_ℝ • (1 : Matrix (Fin d) (Fin d) ℝ)
      - Matrix.vecMulVec ↑(R.centerOfMass - p) ↑(R.centerOfMass - p))
```

lemma inertiaTensor_eq_centerOfMass_add_pointMass[\[source\]](#)

The inertia tensor about the origin is the inertia tensor about the centre of mass c plus $M (|c|^2 1 - c \otimes c)$.

```
lemma inertiaTensor_eq_centerOfMass_add_pointMass {d : ℕ} (R : RigidBody d) (h : R.mass ≠ 0) :
  R.inertiaTensor = R.inertiaTensorAbout R.centerOfMass
```

```
+ R.mass • (⟨⟨R.centerOfMass, R.centerOfMass⟩⟩_ℝ • (1 : Matrix (Fin d) (Fin d) ℝ)
- Matrix.vecMulVec ↑R.centerOfMass ↑R.centerOfMass)
```

2.11 Physlib.Electromagnetism.Charge.ChargeUnit

[Physlib/Electromagnetism/Charge/ChargeUnit.lean](#) on GitHub.

lemma div_mul_div

[\[source\]](#)

The unit-ratio cocycle at $\mathbb{R}_{\geq 0}$ (the un-coerced form of `div_mul_div_coe`).

```
lemma div_mul_div (x y z : ChargeUnit) : (x / y) * (y / z) = x / z
```

2.12 Physlib.Mathematics.Calculus.Gradient

[Physlib/Mathematics/Calculus/Gradient.lean](#) on GitHub.

lemma gradient_add_const

[\[source\]](#)

Adding a constant to a function does not change its gradient.

```
lemma gradient_add_const {f : F → ℝ} (c : ℝ) (x : F) :
gradient (fun y => f y + c) x = gradient f x
```

lemma gradient_const_mul

[\[source\]](#)

The gradient of a constant multiple of a differentiable function is the constant multiple of the gradient.

```
lemma gradient_const_mul {f : F → ℝ} {x : F} (c : ℝ) (hf : DifferentiableAt ℝ f x) :
gradient (fun y => c * f y) x = c • gradient f x
```

lemma gradient_inner_self

[\[source\]](#)

The gradient of $y \mapsto \langle y, y \rangle$ at x is $2 \cdot x$.

```
lemma gradient_inner_self (x : F) : gradient (fun y : F => ⟨y, y⟩_ℝ) x = (2 : ℝ) • x
```

lemma gradient_const_mul_inner_self

[\[source\]](#)

*The gradient of $y \mapsto c * \langle y, y \rangle$ at x is $(2 * c) \cdot x$.*

```
lemma gradient_const_mul_inner_self (c : ℝ) (x : F) :
gradient (fun y : F => c * ⟨y, y⟩_ℝ) x = (2 * c) • x
```

lemma gradient_coord

[\[source\]](#)

The gradient of the i -th coordinate functional on Euclidean space is the i -th basis vector.

```
lemma gradient_coord {ι : Type*} [Fintype ι] [DecidableEq ι] (i : ι) (x : EuclideanSpace ℝ ι) :
gradient (fun y : EuclideanSpace ℝ ι => y i) x = EuclideanSpace.single i 1
```

lemma gradient_comp_coord[\[source\]](#)

Chain rule for a function of one coordinate: the gradient of $y \mapsto f(y\ i)$ at x is $f' \cdot \text{EuclideanSpace.single } i\ 1$, where f' is the derivative of f at $x\ i$.

```
lemma gradient_comp_coord {ι : Type*} [Fintype ι] [DecidableEq ι] {f : ℝ → ℝ} {f' : ℝ}
  (i : ι) (x : EuclideanSpace ℝ ι) (hf : HasDerivAt f f' (x i)) :
  gradient (fun y : EuclideanSpace ℝ ι => f (y i)) x = f' • EuclideanSpace.single i 1
```

2.13 Physlib.Mathematics.HasTemperateGrowth

[Physlib/Mathematics/HasTemperateGrowth.lean](#) on GitHub.

lemma prod[\[source\]](#)

The finite product of functions of temperate growth is again of temperate growth.

```
@[to_fun (attr := fun_prop)]
lemma prod {ι : Type*} {s : Finset ι} {E F : Type*} [NormedAddCommGroup E]
  [NormedSpace ℝ E] [NormedCommRing F] [NormedAlgebra ℝ F] {f : ι → E → F}
  (hf : ∀ i ∈ s, HasTemperateGrowth (f i)) : HasTemperateGrowth (∏ i ∈ s, f i)
```

2.14 Physlib.Mathematics.KroneckerDelta.Basic

[Physlib/Mathematics/KroneckerDelta/Basic.lean](#) on GitHub.

lemma generalizedKroneckerDelta_comp_perm[\[source\]](#)

Simultaneously reindexing the upper and lower slots of a generalized Kronecker delta by the same permutation leaves it unchanged.

```
@[simp]
lemma generalizedKroneckerDelta_comp_perm {α ι : Type} [DecidableEq α] [DecidableEq ι]
  [Fintype ι] (μ ν : ι → α) (e : Equiv.Perm ι) :
  generalizedKroneckerDelta (μ ∘ e) (ν ∘ e) = generalizedKroneckerDelta μ ν
```

2.15 Physlib.Mathematics.KroneckerDelta.Contraction

[Physlib/Mathematics/KroneckerDelta/Contraction.lean](#) on GitHub.

lemma sum_generalizedKroneckerDelta_mul_snoc[\[source\]](#)

Symbol-level triple contraction with the free index in the last slot.

```
lemma sum_generalizedKroneckerDelta_mul_snoc (σ τ : Fin 4) :
  ∑ h : Fin 3 → Fin 4,
    generalizedKroneckerDelta (Fin.snoc h σ) id * generalizedKroneckerDelta (Fin.snoc h τ)
  id =
  6 * ((kroneckerDelta σ τ : ℕ) : ℤ)
```

2.16 Physlib.Mathematics.LeviCivita.Basic

[Physlib/Mathematics/LeviCivita/Basic.lean](#) on GitHub.

def leviCivitaSymbol[\[source\]](#)

The Levi-Civita symbol on a finite index type ι : `leviCivitaSymbol g` is the sign of g when $g : \iota \rightarrow \iota$ is a permutation, and 0 otherwise. It is the generalized Kronecker delta of g against the identity, i.e. the determinant of the matrix of Kronecker deltas $\delta[g\ i, j]$.

For $\iota = \text{Fin } d$ this is the Levi-Civita symbol $\varepsilon_{\{i_1 \dots i_d\}}$ in dimension d , normalized by $\varepsilon_{\{0\ 1 \dots (d-1)\}} = 1$.

```
def leviCivitaSymbol (g :  $\iota \rightarrow \iota$ ) :  $\mathbb{Z}$ 
```

lemma leviCivitaSymbol_eq_det[\[source\]](#)

The Levi-Civita symbol as the determinant of the matrix of Kronecker deltas $\delta[g\ i, j]$.

```
lemma leviCivitaSymbol_eq_det (g :  $\iota \rightarrow \iota$ ) :  
  leviCivitaSymbol g = Matrix.det (fun i j => ((kroneckerDelta (g i) j :  $\mathbb{N}$ ) :  $\mathbb{Z}$ ))
```

lemma leviCivitaSymbol_id[\[source\]](#)

The Levi-Civita symbol of the identity, i.e. $\varepsilon_{\{0\ 1 \dots (d-1)\}}$, is 1.

```
@[simp]  
lemma leviCivitaSymbol_id : leviCivitaSymbol (id :  $\iota \rightarrow \iota$ ) = 1
```

lemma leviCivitaSymbol_perm[\[source\]](#)

The Levi-Civita symbol of a permutation σ is the sign of σ .

```
@[simp]  
lemma leviCivitaSymbol_perm ( $\sigma$  : Equiv.Perm  $\iota$ ) :  
  leviCivitaSymbol  $\uparrow\sigma$  = (Equiv.Perm.sign  $\sigma$  :  $\mathbb{Z}$ )
```

lemma leviCivitaSymbol_comp_swap[\[source\]](#)

The Levi-Civita symbol is antisymmetric under transposition of two of its indices: precomposing with the swap of two distinct index positions negates it.

```
lemma leviCivitaSymbol_comp_swap (g :  $\iota \rightarrow \iota$ ) {i j :  $\iota$ } (hij : i  $\neq$  j) :  
  leviCivitaSymbol (g  $\circ$  Equiv.swap i j) = - leviCivitaSymbol g
```

lemma leviCivitaSymbol_swap_comp[\[source\]](#)

The Levi-Civita symbol is antisymmetric under transposition of two index values: postcomposing with the swap of two distinct values exchanges those two values whenever they occur and negates it.

```
lemma leviCivitaSymbol_swap_comp (g :  $\iota \rightarrow \iota$ ) {i j :  $\iota$ } (hij : i  $\neq$  j) :  
  leviCivitaSymbol (Equiv.swap i j  $\circ$  g) = - leviCivitaSymbol g
```

lemma leviCivitaSymbol_eq_zero_of_eq[\[source\]](#)

The Levi-Civita symbol vanishes on a repeated index: if two distinct index positions $i \neq j$ carry the same value, the symbol is zero.

```
lemma leviCivitaSymbol_eq_zero_of_eq {g :  $\iota \rightarrow \iota$ } {i j :  $\iota$ } (hi j :  $i \neq j$ ) (h :  $g\ i = g\ j$ ) :
  leviCivitaSymbol g = 0
```

lemma leviCivitaSymbol_eq_zero_of_not_injective[\[source\]](#)

The Levi-Civita symbol vanishes on maps which are not injective.

```
lemma leviCivitaSymbol_eq_zero_of_not_injective {g :  $\iota \rightarrow \iota$ } (h :  $\neg$  Function.Injective g) :
  leviCivitaSymbol g = 0
```

lemma leviCivitaSymbol_eq_zero_iff[\[source\]](#)

The Levi-Civita symbol vanishes exactly on maps with a repeated index, i.e. on maps which are not injective.

```
lemma leviCivitaSymbol_eq_zero_iff {g :  $\iota \rightarrow \iota$ } :
  leviCivitaSymbol g = 0  $\leftrightarrow$   $\neg$  Function.Injective g
```

2.17 Physlib.Mathematics.List

[Physlib/Mathematics/List.lean](#) on GitHub.

lemma takeWhile_eraseIdx[\[source\]](#)

```
lemma takeWhile_eraseIdx {I : Type} (P : I  $\rightarrow$  Prop) [DecidablePred P] :
  (l : List I)  $\rightarrow$  (i :  $\mathbb{N}$ )  $\rightarrow$  (hi :  $\forall$  (i j : Fin l.length),  $i < j \rightarrow P\ (l.get\ j) \rightarrow P\ (l.get\ i)$ )  $\rightarrow$ 
  List.takeWhile P (List.eraseIdx l i) = (List.takeWhile P l).eraseIdx i
| [], _, h => by
  rfl
| a :: [], 0, h => by
  simp only [List.takeWhile, List.eraseIdx_zero]
  split <;> rfl
| a :: [], Nat.succ n, h => by
  simp only [Nat.succ_eq_add_one, List.eraseIdx_cons_succ, List.eraseIdx_nil]
  rw [List.eraseIdx_of_length_le ((List.takeWhile_sublist _).length_le.trans (by simp))]
| a :: b :: l, 0, h => by
  simp only [List.takeWhile, List.eraseIdx_zero]
  by_cases hPb : P b
  · have hPa : P a
```

2.18 Physlib.Mathematics.OneParameterSubgroups.Basic

[Physlib/Mathematics/OneParameterSubgroups/Basic.lean](#) on GitHub.

instance NormedAlgebra $\mathbb{Q}$ E[\[source\]](#)

NormedSpace.exp's API wants a $\mathbb{Q}$ -algebra structure (for the $1/n!$ coefficients), which isn't automatic from NormedAlgebra $\mathbb{R}$ E alone; derive it once here rather than at each use site.

```
local instance : NormedAlgebra  $\mathbb{Q}$  E
```

lemma exists_isUnit_intervalIntegral[\[source\]](#)

```
lemma exists_isUnit_intervalIntegral [Nontrivial E] (U : AddChar ℝ E) (hU : Continuous U) :
  ∃ d : ℝ, 0 < d ∧ IsUnit (∫ x in (0 : ℝ)..d, U x)
```

lemma mul_intervalIntegral_eq_sub[\[source\]](#)

Translating a one-parameter subgroup translates its interval integral.

```
lemma mul_intervalIntegral_eq_sub (U : AddChar ℝ E) (hU : Continuous U) (s t : ℝ) :
  U s * ∫ x in (0 : ℝ)..t, U x =
  (∫ x in (0 : ℝ)..(s + t), U x) - ∫ x in (0 : ℝ)..s, U x
```

lemma differentiable[\[source\]](#)

```
lemma differentiable [Nontrivial E] (U : AddChar ℝ E) (hU : Continuous U) :
  Differentiable ℝ U
```

lemma apply_eq_exp_smul_deriv[\[source\]](#)

```
lemma apply_eq_exp_smul_deriv (U : AddChar ℝ E) (hU : Continuous U) (t : ℝ) :
  U t = NormedSpace.exp (t • deriv U 0)
```

lemma generator_unique[\[source\]](#)

Any exponential generator of a one-parameter subgroup is its derivative at zero.

```
lemma generator_unique (U : AddChar ℝ E) (A : E)
  (h : ∀ t : ℝ, U t = NormedSpace.exp (t • A)) : A = deriv U 0
```

2.19 Physlib.Mathematics.OneParameterSubgroups.Unitary

[Physlib/Mathematics/OneParameterSubgroups/Unitary.lean](#) on GitHub.

structure UnitaryOneParameterGroup[\[source\]](#)

A norm-continuous unitary one-parameter group on a complex Hilbert space.

```
structure UnitaryOneParameterGroup (H : Type*) [NormedAddCommGroup H] [InnerProductSpace ℂ H]
  [CompleteSpace H] where
  toAddChar : AddChar ℝ (H →L[ℂ] H)
  mem_unitary : ∀ t, toAddChar t ∈ unitary (H →L[ℂ] H)
  continuous : Continuous toAddChar
```

instance CoeFun (UnitaryOneParameterGroup H) fun _ => ℝ → (H →L[ℂ] H)[\[source\]](#)

```
instance : CoeFun (UnitaryOneParameterGroup H) fun _ => ℝ → (H →L[ℂ] H)
```

lemma ext[\[source\]](#)

@[ext]

```
lemma ext {U V : UnitaryOneParameterGroup H} (h : ∀ t, U t = V t) : U = V
```

lemma apply_eq_exp_deriv[\[source\]](#)

```
lemma apply_eq_exp_deriv (U : UnitaryOneParameterGroup H) (t : ℝ) :
  U t = NormedSpace.exp ((t : ℂ) • deriv U 0)
```

lemma adjoint_eq[\[source\]](#)

```
@[simp]
lemma adjoint_eq (U : UnitaryOneParameterGroup H) (t : ℝ) : (U t).adjoint = U (-t)
```

def generator[\[source\]](#)

The bounded self-adjoint generator of U , in the convention $U(t) = \exp(-itA)$.

```
noncomputable def generator (U : UnitaryOneParameterGroup H) : H →L[ℂ] H
```

lemma apply_eq_exp_generator[\[source\]](#)

```
lemma apply_eq_exp_generator (U : UnitaryOneParameterGroup H) (t : ℝ) :
  U t = NormedSpace.exp ((-t : ℂ) * Complex.I) • U.generator
```

lemma generator_unique[\[source\]](#)

Any exponential generator of a unitary one-parameter group equals $U.generator$; the uniqueness of the derivative at zero ($OneParameterSubgroup.generator_unique$) is the underlying fact, restated here in terms of the physics convention.

```
lemma generator_unique (U : UnitaryOneParameterGroup H) (A : H →L[ℂ] H)
  (h : ∀ t : ℝ, U t = NormedSpace.exp ((-t : ℂ) * Complex.I) • A) :
  A = U.generator
```

lemma adjoint_deriv_eq_neg[\[source\]](#)

```
lemma adjoint_deriv_eq_neg (U : UnitaryOneParameterGroup H) :
  (deriv U 0).adjoint = -deriv U 0
```

lemma generator_isSelfAdjoint[\[source\]](#)

The generator of a unitary one-parameter group is self-adjoint.

```
lemma generator_isSelfAdjoint (U : UnitaryOneParameterGroup H) : IsSelfAdjoint U.generator
```

def ofSelfAdjoint[\[source\]](#)

The unitary one-parameter group generated by a bounded self-adjoint operator.

```
def ofSelfAdjoint {A : H →L[ℂ] H} (hA : IsSelfAdjoint A) :
  UnitaryOneParameterGroup H
```

lemma ofSelfAdjoint_apply[\[source\]](#)

```
@[simp]
lemma ofSelfAdjoint_apply {A : H →L[ℂ] H} (hA : IsSelfAdjoint A) (t : ℝ) :
```

```
ofSelfAdjoint hA t = NormedSpace.exp ((-t : ℂ) * Complex.I) • A)
```

lemma ofSelfAdjoint_generator

[\[source\]](#)

ofSelfAdjoint recovers U from its own generator: this is the defining property used to show `stoneEquiv` is a left inverse.

```
@[simp]
```

```
lemma ofSelfAdjoint_generator (U : UnitaryOneParameterGroup H) :  
  ofSelfAdjoint U.generator_isSelfAdjoint = U
```

lemma generator_ofSelfAdjoint

[\[source\]](#)

The generator of `ofSelfAdjoint hA` is A itself: this is the defining property used to show `stoneEquiv` is a right inverse.

```
@[simp]
```

```
lemma generator_ofSelfAdjoint {A : H →L[ℂ] H} (hA : IsSelfAdjoint A) :  
  (ofSelfAdjoint hA).generator = A
```

def stoneEquiv

[\[source\]](#)

Stone's correspondence between norm-continuous unitary one-parameter groups and bounded self-adjoint generators.

```
noncomputable def stoneEquiv :
```

```
  UnitaryOneParameterGroup H ≈ {A : H →L[ℂ] H // IsSelfAdjoint A} where
```

2.20 Physlib.Mathematics.RatComplexNum

[Physlib/Mathematics/RatComplexNum.lean](#) on GitHub.

lemma neg_I_mul_toComplexNum

[\[source\]](#)

Multiplication by $-I$ under the inclusion of `RatComplexNum` into the complex numbers.

```
lemma neg_I_mul_toComplexNum (a : RatComplexNum) :  
  (-I) * toComplexNum a = toComplexNum (- ((0, 1) * a))
```

lemma toComplexNum_eq_add_neg_add_iff

[\[source\]](#)

Equality with a four-term sum of rational complex numbers can be checked before their inclusion into the complex numbers.

```
lemma toComplexNum_eq_add_neg_add_iff {a b c d e : RatComplexNum} :  
  (toComplexNum a = toComplexNum b + -toComplexNum c + toComplexNum d + toComplexNum e) ↔  
    a = b + -c + d + e
```

2.21 Physlib.Mathematics.SpecialFunctions.EllipticIntegral

[Physlib/Mathematics/SpecialFunctions/EllipticIntegral.lean](#) on GitHub.

def completeEllipticK[\[source\]](#)

The complete elliptic integral of the first kind in the parameter convention, $K(m) = \int_0^{\pi/2} \frac{1}{\sqrt{1 - m \sin^2 \phi}} d\phi$. The physics literature often writes $K(k)$ with $m = k^2$. The integrand is written as a real power rather than $1 / \sqrt{(\dots)}$ so that continuity, positivity and monotonicity in m come from the `rpow` API (`Continuous.rpow_const`, `Real.rpow_pos_of_pos`, `Real.rpow_le_rpow_of_nonpos`); the two forms agree by `Real.sqrt_eq_rpow` and `Real.rpow_neg`. For $m \geq 1$ see the module docstring.

@[pp_nodot]

noncomputable def completeEllipticK (m : ℝ) : ℝ**lemma completeEllipticK_def**[\[source\]](#)

Unfolding lemma: `completeEllipticK m` is the interval integral of its integrand over $[0, \pi/2]$.

lemma completeEllipticK_def (m : ℝ) :completeEllipticK m = $\int_0^{\pi/2} \frac{1}{\sqrt{1 - m \sin^2 \phi}} d\phi$ **lemma completeEllipticK_radican_pos**[\[source\]](#)

For $m < 1$ the radicand $1 - m \sin^2 \phi$ of the integrand of `completeEllipticK m` is positive at every angle ϕ .

lemma completeEllipticK_radican_pos {m : ℝ} (hm : m < 1) (ϕ : ℝ) : $0 < 1 - m \sin^2 \phi$ **lemma continuous_completeEllipticK_integrand**[\[source\]](#)

For $m < 1$ the integrand of `completeEllipticK m` is continuous, the real power being taken at a positive base.

lemma continuous_completeEllipticK_integrand {m : ℝ} (hm : m < 1) :Continuous fun ϕ : ℝ => $\frac{1}{\sqrt{1 - m \sin^2 \phi}}$ **lemma intervalIntegrable_completeEllipticK_integrand**[\[source\]](#)

For $m < 1$ the integrand of `completeEllipticK m` is interval integrable on $[0, \pi/2]$.

lemma intervalIntegrable_completeEllipticK_integrand {m : ℝ} (hm : m < 1) :IntervalIntegrable (fun ϕ : ℝ => $\frac{1}{\sqrt{1 - m \sin^2 \phi}}$) volume 0 (π / 2)**lemma completeEllipticK_zero**[\[source\]](#)

$K 0 = \pi / 2$: at parameter zero the integrand of `completeEllipticK` is the constant 1.

@[simp]

lemma completeEllipticK_zero : completeEllipticK 0 = π / 2**lemma completeEllipticK_pos**[\[source\]](#)

For $m < 1$ the complete elliptic integral `completeEllipticK m` is positive.

```
lemma completeEllipticK_pos {m : ℝ} (hm : m < 1) : 0 < completeEllipticK m
```

lemma completeEllipticK_mono

[\[source\]](#)

completeEllipticK is monotone on its domain: for $m_1 \leq m_2 < 1$, $K m_1 \leq K m_2$. The integrand is pointwise monotone in the parameter, the radicand being positive for both parameters.

```
@[gcongr]
```

```
lemma completeEllipticK_mono {m1 m2 : ℝ} (h12 : m1 ≤ m2) (h2 : m2 < 1) :
  completeEllipticK m1 ≤ completeEllipticK m2
```

lemma monotoneOn_completeEllipticK

[\[source\]](#)

completeEllipticK is monotone on its domain $(-\infty, 1)$, as a MonotoneOn statement.

```
lemma monotoneOn_completeEllipticK : MonotoneOn completeEllipticK (Set.Iio 1)
```

lemma completeEllipticK_strictMono

[\[source\]](#)

completeEllipticK is strictly monotone on its domain: for $m_1 < m_2 < 1$, $K m_1 < K m_2$. The pointwise inequality between the integrands is strict at $\varphi = \pi / 2$, where $\sin^2 \varphi = 1$.

```
@[gcongr]
```

```
lemma completeEllipticK_strictMono {m1 m2 : ℝ} (h12 : m1 < m2) (h2 : m2 < 1) :
  completeEllipticK m1 < completeEllipticK m2
```

lemma strictMonoOn_completeEllipticK

[\[source\]](#)

completeEllipticK is strictly increasing on $(-\infty, 1)$, as a bundled StrictMonoOn.

```
lemma strictMonoOn_completeEllipticK : StrictMonoOn completeEllipticK (Set.Iio 1)
```

lemma pi_div_two_le_completeEllipticK

[\[source\]](#)

For $0 \leq m < 1$ the complete elliptic integral $\text{completeEllipticK } m$ is at least its value $\pi / 2$ at $m = 0$.

```
lemma pi_div_two_le_completeEllipticK {m : ℝ} (hm0 : 0 ≤ m) (hm1 : m < 1) :
  π / 2 ≤ completeEllipticK m
```

lemma completeEllipticK_le

[\[source\]](#)

*For $0 \leq m < 1$ the complete elliptic integral $\text{completeEllipticK } m$ is at most $\pi / 2 * (1 - m) ^ (-1/2)$: the radicand is at least $1 - m$, $\sin^2 \varphi$ being at most 1, so the integrand is at most the constant $(1 - m) ^ (-1/2)$. With $\text{pi_div_two_le_completeEllipticK}$ this sandwiches K on $[0, 1)$; for the pendulum, where $m = \sin^2 (\theta_0 / 2)$, it bounds the period by $T \leq 2\pi \sqrt{\ell / g} / \cos (\theta_0 / 2)$.*

```
lemma completeEllipticK_le {m : ℝ} (hm0 : 0 ≤ m) (hm1 : m < 1) :
  completeEllipticK m ≤ π / 2 * (1 - m) ^ (-1 / 2 : ℝ)
```

lemma continuousOn_completeEllipticK[\[source\]](#)

completeEllipticK is continuous on its domain $(-\infty, 1)$.

```
lemma continuousOn_completeEllipticK : ContinuousOn completeEllipticK (Set.Iio 1)
```

lemma continuousAt_completeEllipticK[\[source\]](#)

completeEllipticK is continuous at every point $m < 1$ of its domain, $(-\infty, 1)$ being a neighbourhood of m .

```
lemma continuousAt_completeEllipticK {m : ℝ} (hm : m < 1) : ContinuousAt completeEllipticK m
```

lemma continuousAt_completeEllipticK_zero[\[source\]](#)

completeEllipticK is continuous at $m = 0$, an interior point of its domain $(-\infty, 1)$.

```
lemma continuousAt_completeEllipticK_zero : ContinuousAt completeEllipticK 0
```

2.22 Physlib.Mathematics.Trigonometry.SinSq

[Physlib/Mathematics/Trigonometry/SinSq.lean](#) on GitHub.

lemma sin_sq_lt_one[\[source\]](#)

$\sin x^2 < 1$ for $|x| < \pi / 2$: the cosine is positive there, and $1 - \sin^2 x = \cos^2 x$.

```
lemma sin_sq_lt_one {x : ℝ} (h : |x| < π / 2) : sin x ^ 2 < 1
```

lemma sin_half_sq_lt_one[\[source\]](#)

The half-angle form of `sin_sq_lt_one`: $\sin (\theta / 2)^2 < 1$ for $|\theta| < \pi$. For the pendulum this says that the parameter $\sin^2 (\theta_0 / 2)$ of the period formula lies in the domain of `completeEllipticK` for every libration amplitude $|\theta_0| < \pi$.

```
lemma sin_half_sq_lt_one {θ : ℝ} (h : |θ| < π) : sin (θ / 2) ^ 2 < 1
```

2.23 Physlib.Meta.AllFilePaths

[Physlib/Meta/AllFilePaths.lean](#) on GitHub.

def getFilePaths[\[source\]](#)

Gets an array of all file paths in the supplied directory.

```
partial def getFilePaths (moduleName : String) : IO (Array FilePath)
```

2.24 Physlib.Meta.Linters.DefsWithUnderscore

[Physlib/Meta/Linters/DefsWithUnderscore.lean](#) on GitHub.

def parserDeclTypes[\[source\]](#)

The types of the declarations that syntax, notation and macro commands produce, along with the pretty-printer declarations generated beside them. These hold parser data rather than a mathematical definition, so their names are exempt from the naming convention.

```
def parserDeclTypes : List Name
```

def projectSuffix[\[source\]](#)

The suffix Lean appends to the generated name of an anonymous instance to keep it unique across projects, for example `_physlib` for a declaration of the Physlib project. This mirrors `Lean.Elab.Command.NameGen.mkBaseNameWithSuffix`.

```
def projectSuffix (env : Environment) (declName : Name) : Option String
```

def hasNumericSuffix[\[source\]](#)

Whether the final component of `declName` ends in an underscore followed by a number, the suffix Lean adds when the name it generated for an anonymous instance is already taken.

```
def hasNumericSuffix (declName : Name) : Bool
```

def defsWithUnderscoreExempt[\[source\]](#)

Whether the `defsWithUnderscore` linter should pass over `declName`.

```
def defsWithUnderscoreExempt (declName : Name) : CoreM Bool
```

def withDefsWithUnderscoreExemptions[\[source\]](#)

`withDefsWithUnderscoreExemptions` linter behaves like `linter` except that it passes over the declarations selected by `defsWithUnderscoreExempt`.

```
def withDefsWithUnderscoreExemptions (linter : NamedLinter) : NamedLinter
```

2.25 Physlib.Particles.NeutrinoPhysics.Basic

[Physlib/Particles/NeutrinoPhysics/Basic.lean](#) on GitHub.

def diagPhaseUnitary[\[source\]](#)

diagonal phase matrix $\text{diag}(i\theta_i)$ is part of the unitary group

```
def diagPhaseUnitary (θ : Fin 3 → ℝ) : unitaryGroup (Fin 3) ℂ
```

def PMNSDiracEquivalence[\[source\]](#)

The Dirac PMNS matrix equivalence relations

```
def PMNSDiracEquivalence (U V : unitaryGroup (Fin 3) ℂ) : Prop
```

lemma PMNSDiracEquivalence_refl[\[source\]](#)

The relation PMNSDiracEquivalence is reflexive.

lemma PMNSDiracEquivalence_refl :

$\forall U : \text{unitaryGroup (Fin 3) } \mathbb{C}, \text{PMNSDiracEquivalence } U \ U$

lemma PMNSDiracEquivalence_symm[\[source\]](#)

The relation PMNSDiracEquivalence is symmetric.

lemma PMNSDiracEquivalence_symm :

$\forall U \ V : \text{unitaryGroup (Fin 3) } \mathbb{C}, \text{PMNSDiracEquivalence } U \ V \rightarrow \text{PMNSDiracEquivalence } V \ U$

lemma PMNSDiracEquivalence_trans[\[source\]](#)

The relation PMNSDiracEquivalence is transitive.

lemma PMNSDiracEquivalence_trans {U V W : unitaryGroup (Fin 3) } $\mathbb{C}$:

$\text{PMNSDiracEquivalence } U \ V \rightarrow \text{PMNSDiracEquivalence } V \ W \rightarrow \text{PMNSDiracEquivalence } U \ W$

2.26 Physlib.Particles.StandardModel.HiggsBoson.Basic

[Physlib/Particles/StandardModel/HiggsBoson/Basic.lean](#) on GitHub.

def repGaugeGroupI[\[source\]](#)

The representation of the gauge group GaugeGroupI on HiggsVec: the SU(2) factor acts by matrix multiplication, and the U(1) factor by scalar multiplication with its third power.

def repGaugeGroupI : Representation $\mathbb{C}$ GaugeGroupI HiggsVec **where**

lemma repGaugeGroupI_apply[\[source\]](#)

lemma repGaugeGroupI_apply (g : StandardModel.GaugeGroupI) (ϕ : HiggsVec) :

$\text{repGaugeGroupI } g \ \phi = (\text{WithLp.toLp } 2 <| \ g.\text{toU1}^3 \bullet (g.\text{toSU2.1} *_v \ \phi.\text{ofLp}))$

lemma repGaugeGroupI_apply_eq_U1_mul_SU2[\[source\]](#)

lemma repGaugeGroupI_apply_eq_U1_mul_SU2 (g : StandardModel.GaugeGroupI) (ϕ : HiggsVec) :

$\text{repGaugeGroupI } g \ \phi = (\text{WithLp.toLp } 2 <| \ g.\text{toSU2.1} *_v \ (g.\text{toU1}^3 \bullet \phi.\text{ofLp}))$

lemma repGaugeGroupI_apply_eq_U1_smul_SU2[\[source\]](#)

lemma repGaugeGroupI_apply_eq_U1_smul_SU2 (g : StandardModel.GaugeGroupI) (ϕ : HiggsVec) :

$\text{repGaugeGroupI } g \ \phi = (\text{WithLp.toLp } 2 <| \ (g.\text{toU1}^3 \bullet g.\text{toSU2.1}) *_v \ \phi.\text{ofLp})$

lemma repGaugeGroupI_inner[\[source\]](#)

@[simp]

lemma repGaugeGroupI_inner (g : StandardModel.GaugeGroupI) ($\phi \ \psi$: HiggsVec) :

$\langle\langle \text{repGaugeGroupI } g \ \phi, \text{repGaugeGroupI } g \ \psi \rangle\rangle_{\mathbb{C}} = \langle\langle \phi, \psi \rangle\rangle_{\mathbb{C}}$

lemma repGaugeGroupI_norm[\[source\]](#)`@[simp]`

```
lemma repGaugeGroupI_norm (g : StandardModel.GaugeGroupI) (φ : HiggsVec) :
  ||repGaugeGroupI g φ|| = ||φ||
```

lemma repGaugeGroupI_mul_apply[\[source\]](#)

```
lemma repGaugeGroupI_mul_apply (g1 g2 : StandardModel.GaugeGroupI) (φ : HiggsVec) :
  repGaugeGroupI (g1 * g2) φ = repGaugeGroupI g1 (repGaugeGroupI g2 φ)
```

lemma repGaugeGroupI_inv_apply_eq_iff[\[source\]](#)

```
lemma repGaugeGroupI_inv_apply_eq_iff (g : StandardModel.GaugeGroupI) (φ ψ : HiggsVec) :
  repGaugeGroupI g-1 φ = ψ ↔ φ = repGaugeGroupI g ψ
```

lemma toRealGroupElem_apply_self[\[source\]](#)

```
lemma toRealGroupElem_apply_self (φ : HiggsVec) :
  repGaugeGroupI (toRealGroupElem φ) φ = ofReal (||φ|| ^ 2)
```

lemma exists_repGaugeGroupI_eq_iff_norm_eq[\[source\]](#)

Two Higgs vectors are in the same gauge orbit (i.e. related by repGaugeGroupI) if and only if they have the same norm.

```
lemma exists_repGaugeGroupI_eq_iff_norm_eq (φ : HiggsVec) (ψ : HiggsVec) :
  (∃ g : GaugeGroupI, repGaugeGroupI g φ = ψ) ↔ ||ψ|| = ||φ||
```

lemma ofU1Subgroup_repGaugeGroupI_apply[\[source\]](#)

```
lemma ofU1Subgroup_repGaugeGroupI_apply (g : unitary ℂ) (φ : HiggsVec) :
  repGaugeGroupI (StandardModel.GaugeGroupI.ofU1Subgroup g) φ =
  (WithLp.toLp 2 <| !![1, 0; 0, g.1 ^ 6] *v φ.ofLp)
```

lemma repGaugeGroupI_phase_snd[\[source\]](#)

```
lemma repGaugeGroupI_phase_snd (φ : HiggsVec) :
  ∃ g : StandardModel.GaugeGroupI,
    (repGaugeGroupI g φ).ofLp 1 = ||(φ.ofLp 1)|| ∧
    (∀ φ1 : HiggsVec, (repGaugeGroupI g φ1).ofLp 0 = φ1.ofLp 0) ∧
    (∀ a : ℝ, repGaugeGroupI g (!₂[a, 0] : HiggsVec) = (!₂[a, 0] : HiggsVec))
```

2.27 Physlib.Particles.SuperSymmetry.SU5.FieldLabels

[Physlib/Particles/SuperSymmetry/SU5/FieldLabels.lean](#) on GitHub.

instance Fintype FieldLabel[\[source\]](#)

```
instance : Fintype FieldLabel where
```

2.28 Physlib.QFT.PerturbationTheory.WickContraction.ExtractEquiv

[Physlib/QFT/PerturbationTheory/WickContraction/ExtractEquiv.lean](#) on GitHub.

instance fintypeZero

[\[source\]](#)

The fintype instance of WickContraction 0 defined through its single element empty.

```
instance fintypeZero : Fintype (WickContraction 0) where
```

instance fintypeSucc

[\[source\]](#)

The fintype instance of WickContraction n, for n.succ this is defined through the equivalence extractEquiv.

```
instance fintypeSucc : (n : ℕ) → Fintype (WickContraction n)
| 0 => fintypeZero
| Nat.succ n => by
  letI
```

2.29 Physlib.QFT.QED.AnomalyCancellation.Even.BasisLinear

[Physlib/QFT/QED/AnomalyCancellation/Even/BasisLinear.lean](#) on GitHub.

def Unshifted.basisAsCharges

[\[source\]](#)

The unshifted part of the basis as charges.

```
def basisAsCharges (j : Fin n.succ) : (PureU1 (2 * n.succ)).Charges
```

lemma basis_on_evenFst_self

[\[source\]](#)

```
lemma basis_on_evenFst_self (j : Fin n.succ) : basisAsCharges j (evenFst j) = 1
```

lemma basis_on_evenFst_other

[\[source\]](#)

```
lemma basis_on_evenFst_other {k j : Fin n.succ} (h : k ≠ j) :
  basisAsCharges k (evenFst j) = 0
```

lemma Unshifted.basis_on_other

[\[source\]](#)

```
lemma basis_on_other {k : Fin n.succ} {j : Fin (2 * n.succ)} (h1 : j ≠ evenFst k)
  (h2 : j ≠ evenSnd k) : basisAsCharges k j = 0
```

lemma basis_evenSnd_eq_neg_evenFst

[\[source\]](#)

```
lemma basis_evenSnd_eq_neg_evenFst (j i : Fin n.succ) :
  basisAsCharges j (evenSnd i) = - basisAsCharges j (evenFst i)
```

lemma basis_on_evenSnd_self

[\[source\]](#)

```
lemma basis_on_evenSnd_self (j : Fin n.succ) : basisAsCharges j (evenSnd j) = - 1
```

lemma basis_on_evenSnd_other[\[source\]](#)

```
lemma basis_on_evenSnd_other {k j : Fin n.succ} (h : k ≠ j) : basisAsCharges k (evenSnd j) = 0
```

lemma Unshifted.basis_linearACC[\[source\]](#)

```
lemma basis_linearACC (j : Fin n.succ) : (accGrav (2 * n.succ)) (basisAsCharges j) = 0
```

lemma Unshifted.basis_accCube[\[source\]](#)

```
lemma basis_accCube (j : Fin n.succ) :  
  accCube (2 * n.succ) (basisAsCharges j) = 0
```

def Unshifted.basis[\[source\]](#)

The unshifted part of the basis as LinSols.

```
@[simps!]  
def basis (j : Fin n.succ) : (PureU1 (2 * n.succ)).LinSols
```

def Unshifted.planeCharges[\[source\]](#)

A point in the span of the unshifted part of the basis as a charge.

```
def planeCharges (f : Fin n.succ → ℚ) : (PureU1 (2 * n.succ)).Charges
```

lemma planeCharges_evenFst[\[source\]](#)

```
lemma planeCharges_evenFst (f : Fin n.succ → ℚ) (j : Fin n.succ) :  
  planeCharges f (evenFst j) = f j
```

lemma planeCharges_evenSnd[\[source\]](#)

```
lemma planeCharges_evenSnd (f : Fin n.succ → ℚ) (j : Fin n.succ) :  
  planeCharges f (evenSnd j) = - f j
```

lemma planeCharges_evenSnd_evenFst[\[source\]](#)

```
lemma planeCharges_evenSnd_evenFst (f : Fin n.succ → ℚ) :  
  planeCharges f ∘ evenSnd = - planeCharges f ∘ evenFst
```

lemma planeCharges_linearACC[\[source\]](#)

```
lemma planeCharges_linearACC (f : Fin n.succ → ℚ) :  
  (accGrav (2 * n.succ)) (planeCharges f) = 0
```

lemma Unshifted.planeCharges_accCube[\[source\]](#)

```
lemma planeCharges_accCube (f : Fin n.succ → ℚ) : accCube (2 * n.succ) (planeCharges f) = 0
```

lemma Unshifted.planeCharges_zero[\[source\]](#)

```
lemma planeCharges_zero (f : Fin n.succ → ℚ) (h : planeCharges f = 0) : ∀ i, f i = 0
```

def Unshifted.planeLinSols[\[source\]](#)

A point in the span of the unshifted part of the basis.

```
def planeLinSols (f : Fin n.succ → ℚ) : (PureU1 (2 * n.succ)).LinSols
```

lemma Unshifted.planeLinSols_val[\[source\]](#)

```
lemma planeLinSols_val (f : Fin n.succ → ℚ) : (planeLinSols f).val = planeCharges f
```

theorem Unshifted.basis_linear_independent[\[source\]](#)

```
theorem basis_linear_independent : LinearIndependent ℚ (@basis n)
```

lemma vectorLikeEven_in_span[\[source\]](#)

```
lemma vectorLikeEven_in_span (S : (PureU1 (2 * n.succ)).LinSols)
  (hS : VectorLikeEven S.val) : ∃ (M : (FamilyPermutations (2 * n.succ)).group),
  (FamilyPermutations (2 * n.succ)).linSolRep M S ∈ Submodule.span ℚ (Set.range basis)
```

def Shifted.basisAsCharges[\[source\]](#)

The shifted part of the basis as charges.

```
def basisAsCharges (j : Fin n) : (PureU1 (2 * n.succ)).Charges
```

lemma basis_on_evenShiftFst_self[\[source\]](#)

```
lemma basis_on_evenShiftFst_self (j : Fin n) : basisAsCharges j (evenShiftFst j) = 1
```

lemma Shifted.basis_on_other[\[source\]](#)

```
lemma basis_on_other {k : Fin n} {j : Fin (2 * n.succ)} (h1 : j ≠ evenShiftFst k)
  (h2 : j ≠ evenShiftSnd k) : basisAsCharges k j = 0
```

lemma basis_on_evenShiftFst_other[\[source\]](#)

```
lemma basis_on_evenShiftFst_other {k j : Fin n} (h : k ≠ j) :
  basisAsCharges k (evenShiftFst j) = 0
```

lemma basis_evenShiftSnd_eq_neg_evenShiftFst[\[source\]](#)

```
lemma basis_evenShiftSnd_eq_neg_evenShiftFst (j i : Fin n) :
  basisAsCharges j (evenShiftSnd i) = - basisAsCharges j (evenShiftFst i)
```

lemma basis_on_evenShiftSnd_self[\[source\]](#)

```
lemma basis_on_evenShiftSnd_self (j : Fin n) : basisAsCharges j (evenShiftSnd j) = - 1
```

lemma basis_on_evenShiftSnd_other[\[source\]](#)

```
lemma basis_on_evenShiftSnd_other {k j : Fin n} (h : k ≠ j) :
  basisAsCharges k (evenShiftSnd j) = 0
```

lemma basis_on_evenShiftZero[\[source\]](#)

```
lemma basis_on_evenShiftZero (j : Fin n) : basisAsCharges j evenShiftZero = 0
```

lemma basis_on_evenShiftLast[\[source\]](#)

```
lemma basis_on_evenShiftLast (j : Fin n) : basisAsCharges j evenShiftLast = 0
```

lemma Shifted.basis_linearACC[\[source\]](#)

```
lemma basis_linearACC (j : Fin n) : (accGrav (2 * n.succ)) (basisAsCharges j) = 0
```

lemma Shifted.basis_accCube[\[source\]](#)

```
lemma basis_accCube (j : Fin n) :
  accCube (2 * n.succ) (basisAsCharges j) = 0
```

def Shifted.basis[\[source\]](#)

The shifted part of the basis as LinSols.

```
@[simps!]
def basis (j : Fin n) : (PureU1 (2 * n.succ)).LinSols
```

def Shifted.planeCharges[\[source\]](#)

A point in the span of the shifted part of the basis as a charge.

```
def planeCharges (f : Fin n → ℚ) : (PureU1 (2 * n.succ)).Charges
```

lemma planeCharges_evenShiftFst[\[source\]](#)

```
lemma planeCharges_evenShiftFst (f : Fin n → ℚ) (j : Fin n) :
  planeCharges f (evenShiftFst j) = f j
```

lemma planeCharges_evenShiftSnd[\[source\]](#)

```
lemma planeCharges_evenShiftSnd (f : Fin n → ℚ) (j : Fin n) :
  planeCharges f (evenShiftSnd j) = - f j
```

lemma planeCharges_evenShiftZero[\[source\]](#)

```
lemma planeCharges_evenShiftZero (f : Fin n → ℚ) : planeCharges f (evenShiftZero) = 0
```

lemma planeCharges_evenShiftLast[\[source\]](#)

```
lemma planeCharges_evenShiftLast (f : Fin n → ℚ) : planeCharges f evenShiftLast = 0
```

lemma Shifted.planeCharges_accCube[\[source\]](#)

```
lemma planeCharges_accCube (f : Fin n → ℚ) : accCube (2 * n.succ) (planeCharges f) = 0
```

lemma Shifted.planeCharges_zero[\[source\]](#)

```
lemma planeCharges_zero (f : Fin n → ℚ) (h : planeCharges f = 0) : ∀ i, f i = 0
```

lemma planeCharges_in_span[\[source\]](#)

```
lemma planeCharges_in_span (f : Fin n → ℚ) :  
  planeCharges f ∈ Submodule.span ℚ (Set.range basisAsCharges)
```

def Shifted.planeLinSols[\[source\]](#)

A point in the span of the shifted part of the basis.

```
def planeLinSols (f : Fin n → ℚ) : (PureU1 (2 * n.succ)).LinSols
```

lemma Shifted.planeLinSols_val[\[source\]](#)

```
lemma planeLinSols_val (f : Fin n → ℚ) : (planeLinSols f).val = planeCharges f
```

theorem Shifted.basis_linear_independent[\[source\]](#)

```
theorem basis_linear_independent : LinearIndependent ℚ (@basis n)
```

lemma smul_basisAsCharges_in_span[\[source\]](#)

```
lemma smul_basisAsCharges_in_span (S : (PureU1 (2 * n.succ)).LinSols) (j : Fin n) :  
  (S.val (evenShiftSnd j) - S.val (evenShiftFst j)) • basisAsCharges j ∈  
  Submodule.span ℚ (Set.range basisAsCharges)
```

lemma swap_as_add[\[source\]](#)

Swapping the elements `evenShiftFst j` and `evenShiftSnd j` is equivalent to adding a vector `basisAsCharges j`.

```
lemma swap_as_add {S S' : (PureU1 (2 * n.succ)).LinSols} (j : Fin n)  
  (hS : ((FamilyPermutations (2 * n.succ)).linSolRep  
    (Equiv.swap (evenShiftFst j) (evenShiftSnd j))) S = S') :  
  S'.val = S.val + (S.val (evenShiftSnd j) - S.val (evenShiftFst j)) • basisAsCharges j
```

lemma unshifted_unshifted_shifted_accCube[\[source\]](#)

```
lemma unshifted_unshifted_shifted_accCube (g : Fin n.succ → ℚ) (j : Fin n) :
  accCubeTriLinSymm (Unshifted.planeCharges g) (Unshifted.planeCharges g)
    (Shifted.basisAsCharges j)
  = g (j.succ) ^ 2 - g (j.castSucc) ^ 2
```

lemma shifted_shifted_unshifted_accCube[\[source\]](#)

```
lemma shifted_shifted_unshifted_accCube (g : Fin n → ℚ) (j : Fin n.succ) :
  accCubeTriLinSymm (Shifted.planeCharges g) (Shifted.planeCharges g) (Unshifted.
    basisAsCharges j)
  = (Shifted.planeCharges g (evenFst j))^2 - (Shifted.planeCharges g (evenSnd j))^2
```

2.30 Physlib.QFT.QED.AnomalyCancellation.Even.LineInCubic

[Physlib/QFT/QED/AnomalyCancellation/Even/LineInCubic.lean](#) on GitHub.

lemma line_in_cubic_unshifted_unshifted_shifted[\[source\]](#)

*This lemma states that for a given S of type $(\text{PureU1 } (2 * n.succ)).\text{AnomalyFreeLinear}$ and a proof h that the line through S lies on a cubic curve, for any functions $g : \text{Fin } n.succ \rightarrow \mathbb{Q}$ and $f : \text{Fin } n \rightarrow \mathbb{Q}$, if $S.\text{val} = \text{Unshifted.planeCharges } g + \text{Shifted.planeCharges } f$, then $\text{accCubeTriLinSymm.toFun } (\text{Unshifted.planeCharges } g, \text{Unshifted.planeCharges } g, \text{Shifted.planeCharges } f) = 0$.*

```
lemma line_in_cubic_unshifted_unshifted_shifted
  {S : (PureU1 (2 * n.succ)).LinSols} (h : LineInCubic S) :
  ∀ (g : Fin n.succ → ℚ) (f : Fin n → ℚ)
    (_ : S.val = Unshifted.planeCharges g + Shifted.planeCharges f),
  accCubeTriLinSymm (Unshifted.planeCharges g) (Unshifted.planeCharges g)
    (Shifted.planeCharges f) = 0
```

lemma unshifted_unshifted_shifted_accCube'[\[source\]](#)

```
lemma unshifted_unshifted_shifted_accCube' {S : (PureU1 (2 * n.succ.succ)).LinSols}
  (f : Fin n.succ.succ → ℚ) (g : Fin n.succ → ℚ) (hS : S.val = Pa f g) :
  accCubeTriLinSymm (Unshifted.planeCharges f) (Unshifted.planeCharges f)
    (Shifted.basisAsCharges (Fin.last n)) =
  - (S.val (evenShiftSnd (Fin.last n)) + S.val (evenShiftFst (Fin.last n))) *
  (2 * S.val evenShiftLast +
  S.val (evenShiftSnd (Fin.last n)) + S.val (evenShiftFst (Fin.last n)))
```

2.31 Physlib.QFT.QED.AnomalyCancellation.Odd.BasisLinear

[Physlib/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean](#) on GitHub.

def Unshifted.basisAsCharges[\[source\]](#)

The unshifted part of the basis as charge assignments.

```
def basisAsCharges (j : Fin n) : (PureU1 (2 * n + 1)).Charges
```

lemma basis_on_oddFst_self[\[source\]](#)

```
lemma basis_on_oddFst_self (j : Fin n) : basisAsCharges j (oddFst j) = 1
```

lemma basis_on_oddFst_other[\[source\]](#)

```
lemma basis_on_oddFst_other {k j : Fin n} (h : k ≠ j) :
  basisAsCharges k (oddFst j) = 0
```

lemma Unshifted.basis_on_other[\[source\]](#)

```
lemma basis_on_other {k : Fin n} {j : Fin (2 * n + 1)} (h1 : j ≠ oddFst k) (h2 : j ≠ oddSnd k)
  :
  basisAsCharges k j = 0
```

lemma basis_oddSnd_eq_minus_oddFst[\[source\]](#)

```
lemma basis_oddSnd_eq_minus_oddFst (j i : Fin n) :
  basisAsCharges j (oddSnd i) = - basisAsCharges j (oddFst i)
```

lemma basis_on_oddSnd_self[\[source\]](#)

```
lemma basis_on_oddSnd_self (j : Fin n) : basisAsCharges j (oddSnd j) = - 1
```

lemma basis_on_oddSnd_other[\[source\]](#)

```
lemma basis_on_oddSnd_other {k j : Fin n} (h : k ≠ j) : basisAsCharges k (oddSnd j) = 0
```

lemma basis_on_oddMid[\[source\]](#)

```
lemma basis_on_oddMid (j : Fin n) : basisAsCharges j oddMid = 0
```

lemma Unshifted.basis_linearACC[\[source\]](#)

```
lemma basis_linearACC (j : Fin n) : (accGrav (2 * n + 1)) (basisAsCharges j) = 0
```

def Unshifted.basis[\[source\]](#)

The unshifted part of the basis as LinSols.

```
@[simps!]
```

```
def basis (j : Fin n) : (PureU1 (2 * n + 1)).LinSols
```

def Unshifted.planeCharges[\[source\]](#)

A point in the span of the unshifted part of the basis as a charge.

```
def planeCharges (f : Fin n → ℚ) : (PureU1 (2 * n + 1)).Charges
```

lemma planeCharges_oddFst[\[source\]](#)

```
lemma planeCharges_oddFst (f : Fin n → ℚ) (j : Fin n) : planeCharges f (oddFst j) = f j
```

lemma planeCharges_oddSnd[\[source\]](#)

```
lemma planeCharges_oddSnd (f : Fin n → ℚ) (j : Fin n) : planeCharges f (oddSnd j) = - f j
```

lemma planeCharges_oddMid[\[source\]](#)

```
lemma planeCharges_oddMid (f : Fin n → ℚ) : planeCharges f oddMid = 0
```

lemma Unshifted.planeCharges_linearACC[\[source\]](#)

```
lemma planeCharges_linearACC (f : Fin n → ℚ) : (accGrav (2 * n + 1)) (planeCharges f) = 0
```

lemma Unshifted.planeCharges_accCube[\[source\]](#)

```
lemma planeCharges_accCube (f : Fin n → ℚ) : accCube (2 * n + 1) (planeCharges f) = 0
```

lemma Unshifted.planeCharges_zero[\[source\]](#)

```
lemma planeCharges_zero (f : Fin n → ℚ) (h : planeCharges f = 0) : ∀ i, f i = 0
```

def Unshifted.planeLinSols[\[source\]](#)

A point in the span of the unshifted part of the basis.

```
def planeLinSols (f : Fin n → ℚ) : (PureU1 (2 * n + 1)).LinSols
```

lemma Unshifted.planeLinSols_val[\[source\]](#)

```
lemma planeLinSols_val (f : Fin n → ℚ) : (planeLinSols f).val = planeCharges f
```

theorem Unshifted.basis_linear_independent[\[source\]](#)

```
theorem basis_linear_independent : LinearIndependent ℚ (@basis n)
```

def Shifted.basisAsCharges[\[source\]](#)

The shifted part of the basis as charge assignments.

```
def basisAsCharges (j : Fin n) : (PureU1 (2 * n + 1)).Charges
```

lemma basis_on_oddShiftFst_self[\[source\]](#)

```
lemma basis_on_oddShiftFst_self (j : Fin n) : basisAsCharges j (oddShiftFst j) = 1
```

lemma basis_on_oddShiftFst_other[\[source\]](#)

```
lemma basis_on_oddShiftFst_other {k j : Fin n} (h : k ≠ j) :
  basisAsCharges k (oddShiftFst j) = 0
```

lemma Shifted.basis_on_other[\[source\]](#)

```
lemma basis_on_other {k : Fin n} {j : Fin (2 * n + 1)}
  (h1 : j ≠ oddShiftFst k) (h2 : j ≠ oddShiftSnd k) :
  basisAsCharges k j = 0
```

lemma basis_oddShiftSnd_eq_minus_oddShiftFst[\[source\]](#)

```
lemma basis_oddShiftSnd_eq_minus_oddShiftFst (j i : Fin n) :
  basisAsCharges j (oddShiftSnd i) = - basisAsCharges j (oddShiftFst i)
```

lemma basis_on_oddShiftSnd_self[\[source\]](#)

```
lemma basis_on_oddShiftSnd_self (j : Fin n) : basisAsCharges j (oddShiftSnd j) = - 1
```

lemma basis_on_oddShiftSnd_other[\[source\]](#)

```
lemma basis_on_oddShiftSnd_other {k j : Fin n} (h : k ≠ j) :
  basisAsCharges k (oddShiftSnd j) = 0
```

lemma basis_on_oddShiftZero[\[source\]](#)

```
lemma basis_on_oddShiftZero (j : Fin n) : basisAsCharges j oddShiftZero = 0
```

lemma Shifted.basis_linearACC[\[source\]](#)

```
lemma basis_linearACC (j : Fin n) : (accGrav (2 * n + 1)) (basisAsCharges j) = 0
```

def Shifted.basis[\[source\]](#)

The shifted part of the basis as LinSols.

```
@[simps!]
def basis (j : Fin n) : (PureU1 (2 * n + 1)).LinSols
```

lemma swap_as_add[\[source\]](#)

Swapping the elements oddShiftFst j and oddShiftSnd j is equivalent to adding a vector basisAsCharges j.

```
lemma swap_as_add {S S' : (PureU1 (2 * n + 1)).LinSols} (j : Fin n)
  (hS : ((FamilyPermutations (2 * n + 1)).linSolRep
    (Equiv.swap (oddShiftFst j) (oddShiftSnd j))) S = S') :
  S'.val = S.val + (S.val (oddShiftSnd j) - S.val (oddShiftFst j)) • basisAsCharges j
```

def Shifted.planeCharges[\[source\]](#)

A point in the span of the shifted part of the basis as a charge.

```
def planeCharges (f : Fin n → ℚ) : (PureU1 (2 * n + 1)).Charges
```

lemma planeCharges_oddShiftFst[\[source\]](#)

```
lemma planeCharges_oddShiftFst (f : Fin n → ℚ) (j : Fin n) :  
  planeCharges f (oddShiftFst j) = f j
```

lemma planeCharges_oddShiftSnd[\[source\]](#)

```
lemma planeCharges_oddShiftSnd (f : Fin n → ℚ) (j : Fin n) :  
  planeCharges f (oddShiftSnd j) = - f j
```

lemma planeCharges_oddShiftZero[\[source\]](#)

```
lemma planeCharges_oddShiftZero (f : Fin n → ℚ) : planeCharges f oddShiftZero = 0
```

lemma Shifted.planeCharges_linearACC[\[source\]](#)

```
lemma planeCharges_linearACC (f : Fin n → ℚ) : (accGrav (2 * n + 1)) (planeCharges f) = 0
```

lemma Shifted.planeCharges_accCube[\[source\]](#)

```
lemma planeCharges_accCube (f : Fin n → ℚ) : accCube (2 * n + 1) (planeCharges f) = 0
```

lemma Shifted.planeCharges_zero[\[source\]](#)

```
lemma planeCharges_zero (f : Fin n → ℚ) (h : planeCharges f = 0) : ∀ i, f i = 0
```

def Shifted.planeLinSols[\[source\]](#)

A point in the span of the shifted part of the basis.

```
def planeLinSols (f : Fin n → ℚ) : (PureU1 (2 * n + 1)).LinSols
```

lemma Shifted.planeLinSols_val[\[source\]](#)

```
lemma planeLinSols_val (f : Fin n → ℚ) : (planeLinSols f).val = planeCharges f
```

theorem Shifted.basis_linear_independent[\[source\]](#)

```
theorem basis_linear_independent : LinearIndependent ℚ (@basis n)
```

2.32 Physlib.QuantumMechanics.FiniteTarget

[Physlib/QuantumMechanics/FiniteTarget.lean](#) on GitHub.

def unitaryTimeEvolution[\[source\]](#)*The unitary time evolution generated by the Hamiltonian of A.*

```
noncomputable def unitaryTimeEvolution : UnitaryOneParameterGroup H
```

lemma unitaryTimeEvolution_generator[\[source\]](#)*The generator of time evolution is the Hamiltonian divided by $\hbar$.*

```
@[simp]
lemma unitaryTimeEvolution_generator :
  A.unitaryTimeEvolution.generator = (( $\hbar$  :  $\mathbb{C}$ )-1) • A.Ham
```

2.33 Physlib.QuantumMechanics.HarmonicOscillator.Basic[Physlib/QuantumMechanics/HarmonicOscillator/Basic.lean](#) on GitHub.**def ξ Equiv**[\[source\]](#)*The continuous linear equivalence which rescales x_i to $\xi_i x_i$.*

```
def  $\xi$ Equiv : Space d  $\approx$  L[ $\mathbb{R}$ ] Space d where
```

lemma ξ Equiv_apply[\[source\]](#)

```
@[simp]
lemma  $\xi$ Equiv_apply (x : Space d) (i : Fin d) : Q. $\xi$ Equiv x i = Q. $\xi$  i * x i
```

lemma ξ Equiv_symm_apply[\[source\]](#)

```
@[simp]
lemma  $\xi$ Equiv_symm_apply (x : Space d) (i : Fin d) : Q. $\xi$ Equiv.symm x i = (Q. $\xi$  i)-1 * x i
```

2.34 Physlib.QuantumMechanics.HarmonicOscillator.Eigenstates[Physlib/QuantumMechanics/HarmonicOscillator/Eigenstates.lean](#) on GitHub.**def eigenEnergy**[\[source\]](#)*The energy eigenvalues, $\sum_i \hbar \omega_i (n_i + \frac{1}{2})$.*

```
def eigenEnergy :  $\mathbb{R}$ 
```

lemma eigenEnergy_eq[\[source\]](#)

```
lemma eigenEnergy_eq : Q.eigenEnergy n =  $\sum_i \hbar * Q.\omega_i * (n_i + 1 / 2)$ 
```

lemma eigenEnergy_strictMono[\[source\]](#)

```
lemma eigenEnergy_strictMono : StrictMono Q.eigenEnergy
```

def eigenCoeff[\[source\]](#)

The i th normalization constant for $Q.eigenfunction\ n$, $1 / \sqrt{2^{n_i} * n_i! * \sqrt{\pi} * \xi_i}$.

```
def eigenCoeff (i : Fin d) : ℝ
```

lemma eigenCoeff_eq[\[source\]](#)

```
lemma eigenCoeff_eq (i : Fin d) : Q.eigenCoeff n i = 1 / √(2 ^ n i * (n i)! * √π * Q.ξ i)
```

def eigenfunction[\[source\]](#)

The eigenfunction labelled by the integer quantum numbers $n : \text{Fin } d \rightarrow \mathbb{N}$, defined as a product of (physicist's) Hermite polynomials multiplying a Gaussian with covariance controlled by the characteristic lengths, $Q.\xi$.

```
def eigenfunction : S(Space d, ℂ)
```

lemma eigenfunction_eq[\[source\]](#)

```
lemma eigenfunction_eq :
  Q.eigenfunction n = compCLMOfContinuousLinearEquiv C Q.ξEquiv.symm (smulLeftCLM C
    (fun x ↦ ∏ i, Q.eigenCoeff n i * physHermite (n i) (x i)) (stdGaussian (Space d) C))
```

lemma eigenfunction_apply[\[source\]](#)

```
lemma eigenfunction_apply :
  Q.eigenfunction n x =
    ∏ i, Q.eigenCoeff n i *
      physHermite (n i) (x i / Q.ξ i) * cexp (-2-1 * (x i / Q.ξ i) ^ 2)
```

def eigenstate[\[source\]](#)

$Q.eigenfunction\ n$ as an element of the Schwartz submodule of the Hilbert space.

```
def eigenstate : SchwartzSubmodule d
```

lemma eigenstate_eq[\[source\]](#)

```
lemma eigenstate_eq : Q.eigenstate n = schartzEquiv _ (Q.eigenfunction n)
```

lemma eigenstates_orthonormal[\[source\]](#)

The energy eigenstates are orthonormal.

```
@[simp, sorryful]
```

```
lemma eigenstates_orthonormal : ⟨⟨Q.eigenstate n : Q.HS), Q.eigenstate n'⟩⟩_C = δ[n,n']
```

2.35 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.MomentumStates

[Physlib/QuantumMechanics/HilbertSpaces/SpaceD/MomentumStates.lean](#) on GitHub.

def momentumState[\[source\]](#)

Momentum state as a member of the strong dual of the Schwartz space.

For a given p this corresponds to the non-normalizable plane wave $\exp (I \, p \cdot_v x)$.

```
def momentumState (p : Space d) : StrongDual ℂ S(Space d, ℂ)
```

lemma momentumState_apply[\[source\]](#)

The defining property of momentum states.

```
@[simp]
lemma momentumState_apply (p : Space d) (ψ : S(Space d, ℂ)) :
  momentumState p ψ = ℱ ψ ((2 * π)-1 • p)
```

lemma eq_of_eq_momentumState[\[source\]](#)

Two Schwartz maps are equal if they are equal on all momentum states.

```
lemma eq_of_eq_momentumState {ψ φ : S(Space d, ℂ)}
  (h : ∀ p, momentumState p ψ = momentumState p φ) : ψ = φ
```

2.36 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.PositionStates

[Physlib/QuantumMechanics/HilbertSpaces/SpaceD/PositionStates.lean](#) on GitHub.

def positionState[\[source\]](#)

Position state as a member of the strong dual of the Schwartz space.

For a given x this corresponds to the non-normalizable wavefunction $\psi(y) = \delta^d(y - x)$

```
def positionState (x : Space d) : StrongDual ℂ S(Space d, ℂ)
```

lemma positionState_apply[\[source\]](#)

The defining property of position states.

```
@[simp]
lemma positionState_apply (x : Space d) (f : S(Space d, ℂ)) : positionState x f = f x
```

lemma eq_of_eq_positionState[\[source\]](#)

Two Schwartz maps are equal if they are equal on all position states.

```
lemma eq_of_eq_positionState {f g : S(Space d, ℂ)}
  (h : ∀ x, positionState x f = positionState x g) : f = g
```

2.37 Physlib.QuantumMechanics.HilbertSpaces.TensorProducts.CompleteTensorProduct

[Physlib/QuantumMechanics/HilbertSpaces/TensorProducts/CompleteTensorProduct.lean](#) on GitHub.

def CompleteTensorProduct[\[source\]](#)

The *__completion__* of the tensor product of two inner product spaces E and F over $\mathbb{k}$. By construction this produces a Hilbert space. The localized notations are $E \otimes_{\mathbb{h}} F$ and $E \otimes_{\mathbb{k}} F$, accessed by open scoped *CompleteTensorProduct*.

```
def CompleteTensorProduct (ℓk : Type*) [RCLike ℓk]
  (E : Type*) [NormedAddCommGroup E] [InnerProductSpace ℓk E]
  (F : Type*) [NormedAddCommGroup F] [InnerProductSpace ℓk F] : Type _
```

instance TensorProduct.instNontrivial[\[source\]](#)

```
instance _root_.TensorProduct.instNontrivial : Nontrivial (E ⊗[ℓk] F) where
```

instance CompleteTensorProduct.instNontrivial[\[source\]](#)

```
instance instNontrivial : Nontrivial (E ⊗h[ℓk] F) where
```

def coe'[\[source\]](#)

The canonical embedding of the tensor product into its completion.

```
@[coe]
def coe' : E ⊗[ℓk] F → E ⊗h[ℓk] F
```

instance Coe (E ⊗[ℓk] F) (E ⊗_h[ℓk] F)[\[source\]](#)

Coercion from $E \otimes_{\mathbb{k}} F$ to its completion.

```
instance : Coe (E ⊗[ℓk] F) (E ⊗h[ℓk] F)
```

lemma denseRange_coe[\[source\]](#)

```
lemma denseRange_coe : DenseRange (coe' : E ⊗[ℓk] F → E ⊗h[ℓk] F)
```

lemma coe_zero[\[source\]](#)

```
@[norm_cast]
lemma coe_zero : (0 : E ⊗[ℓk] F) = (0 : E ⊗h[ℓk] F)
```

lemma coe_eq_zero_iff[\[source\]](#)

```
@[simp]
lemma coe_eq_zero_iff : (x : E ⊗h[ℓk] F) = 0 ↔ x = 0
```

lemma coe_neg[\[source\]](#)

```
@[norm_cast]
lemma coe_neg : (-x : E ⊗[ℓk] F) = (-x : E ⊗h[ℓk] F)
```

lemma coe_sub[\[source\]](#)

```
@[norm_cast]
lemma coe_sub : (x - y : E  $\otimes$   $\mathbb{K}$  F) = (x - y : E  $\otimes_h$   $\mathbb{K}$  F)
```

lemma coe_add[\[source\]](#)

```
@[norm_cast]
lemma coe_add : (x + y : E  $\otimes$   $\mathbb{K}$  F) = (x + y : E  $\otimes_h$   $\mathbb{K}$  F)
```

lemma coe_smul[\[source\]](#)

```
@[simp, norm_cast]
lemma coe_smul : (c • x : E  $\otimes$   $\mathbb{K}$  F) = (c • x : E  $\otimes_h$   $\mathbb{K}$  F)
```

lemma inner_coe[\[source\]](#)

```
@[simp]
lemma inner_coe :  $\langle\langle x : E \otimes_h \mathbb{K} F, (y : E \otimes_h \mathbb{K} F) \rangle\rangle_{\mathbb{K}} = \langle\langle x, y \rangle\rangle_{\mathbb{K}}$ 
```

lemma norm_coe[\[source\]](#)

```
@[simp]
lemma norm_coe :  $\|(x : E \otimes_h \mathbb{K} F)\| = \|x\|$ 
```

def tIncl_{*l*}_{*i*}[\[source\]](#)

The canonical embedding of the tensor product into its completion as a linear isometry.

```
def tInclli : E  $\otimes$   $\mathbb{K}$  F  $\rightarrow_{l^i}$   $\mathbb{K}$  E  $\otimes_h$   $\mathbb{K}$  F
```

lemma coe_tIncl_{*l*}_{*i*}[\[source\]](#)

```
@[simp]
lemma coe_tInclli :  $\uparrow(tIncl_{l^i} : E \otimes F \rightarrow_{l^i} \mathbb{K} E \otimes_h F) = coe'$ 
```

def tIncl_{*L*}[\[source\]](#)

The canonical embedding of the tensor product into its completion as a continuous linear map.

```
def tInclL : E  $\otimes$   $\mathbb{K}$  F  $\rightarrow_L$   $\mathbb{K}$  E  $\otimes_h$   $\mathbb{K}$  F
```

lemma coe_tIncl_{*L*}[\[source\]](#)

```
@[simp]
lemma coe_tInclL :  $\uparrow(tIncl_L : E \otimes F \rightarrow_L \mathbb{K} E \otimes_h F) = coe'$ 
```

lemma norm_tInclL[\[source\]](#)

@[simp]

```
lemma norm_tInclL [Nontrivial E] [Nontrivial F] : ||(tInclL : E ⊗ F → L[ℓ] E ⊗h F)|| = 1
```

lemma induction_on[\[source\]](#)

An induction principle for CompleteTensorProduct combining those of Completion and TensorProduct.

@[elab_as_elim]

```
lemma induction_on {motive : E ⊗h[ℓ] F → Prop} (z : E ⊗h[ℓ] F)
  (zero : motive 0) (tmul : ∀ (x : E) (y : F), motive (x ⊗t[ℓ] y))
  (add : ∀ x y : E ⊗[ℓ] F, motive x → motive y → motive ↑(x + y))
  (closed : IsClosed {x | motive x}) : motive z
```

def comm[\[source\]](#)

The complete tensor product of inner product spaces is commutative, up to linear isometric equivalence.

```
def comm : E ⊗h[ℓ] F ≈l i[ℓ] F ⊗h[ℓ] E
```

lemma comm_symm[\[source\]](#)

@[simp]

```
lemma comm_symm : (comm ℓ E F).symm = comm ℓ F E
```

lemma comm_coe[\[source\]](#)

@[simp]

```
lemma comm_coe (x : E ⊗[ℓ] F) : comm ℓ E F x = TensorProduct.comm ℓ E F x
```

lemma denseRange_map[\[source\]](#)

The tensor product of a pair of linear maps with dense range also has dense range.

```
lemma _root_.TensorProduct.denseRange_map
  {R ℓ : Type*} [CommSemiring R] [RCLike ℓ] {σ : R →+* ℓ} [RingHomSurjective σ]
  {M : Type*} [AddCommMonoid M] [Module R M] {N : Type*} [AddCommMonoid N] [Module R N]
  {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℓ E]
  {F : Type*} [NormedAddCommGroup F] [InnerProductSpace ℓ F]
  {f : M →sl[σ] E} (hf : DenseRange f) {g : N →sl[σ] F} (hg : DenseRange g) :
  DenseRange (TensorProduct.map f g)
```

def assoc[\[source\]](#)

The complete tensor product of inner product spaces is associative, up to linear isometric equivalence.

```
def assoc : E ⊗h[ℓ] F ⊗h[ℓ] G ≈l i[ℓ] E ⊗h[ℓ] (F ⊗h[ℓ] G)
```

2.38 Physlib.QuantumMechanics.Hydrogen.LaplaceRungeLenzVector

[Physlib/QuantumMechanics/Hydrogen/LaplaceRungeLenzVector.lean](#) on GitHub.

lemma angularMomentum_commutation_Ldot_p

[\[source\]](#)

A supporting piece of angularMomentum_commutation_lrl: how $\mathbf{L}_{ij}$ commutes with the dot-product term $\mathbf{L}_k \cdot \mathbf{v} \mathbf{p}$ appearing in $H.lrlOperator$'s expanded form ($lrlOperator_eq'$).

```
lemma angularMomentum_commutation_Ldot_p (i j k : Fin H.d) :
  {L[H.d] i j, L k ·v p} =
    (I * ħ) • (δ[i,k] • (L j ·v p) - δ[j,k] • (L i ·v p))
```

2.39 Physlib.QuantumMechanics.OperatorAlgebra.Basic

[Physlib/QuantumMechanics/OperatorAlgebra/Basic.lean](#) on GitHub.

class OperatorAlgebra

[\[source\]](#)

A unital complex C^ -algebra with a compatible order making $\leq$ the usual positivity order. Mathlib doesn't pick one canonically, so definitions below that need $\leq$ take this instead of just $CStarAlgebra$.*

```
class OperatorAlgebra (A : Type*) extends CStarAlgebra A, PartialOrder A, StarOrderedRing A
```

abbrev Observable

[\[source\]](#)

*An observable is a self-adjoint element of A : position, momentum, energy, spin, Self-adjointness is exactly what makes an element a **measurable** quantity — it is what forces its spectrum, the possible measurement outcomes, to be real.*

```
noncomputable abbrev Observable (A : Type*) [OperatorAlgebra A]
```

abbrev PositiveElement

[\[source\]](#)

A positive element of A : an observable whose measurement outcomes are all ≥ 0 . Positivity is what gives observables a meaningful order ($a \leq b$ meaning $b - a$ is positive).

```
abbrev PositiveElement (A : Type*) [OperatorAlgebra A]
```

abbrev Effect

[\[source\]](#)

An effect is an observable between zero and the identity, representing a yes/no measurement outcome.

```
abbrev Effect (A : Type*) [OperatorAlgebra A]
```

structure POVM

[\[source\]](#)

A finite POVM on A : the most general notion of a measurement with outcomes in X , generalizing a single yes/no Effect to several possible outcomes.

```

structure POVM (A : Type*) [OperatorAlgebra A] (X : Type*) [Fintype X] where
  effect : X → Effect A
  sum_effect :  $\sum x, (\text{effect } x : A) = 1$ 

```

abbrev Unitary[\[source\]](#)

A unitary element of A: implements a reversible transformation of the system — a symmetry, or time evolution under a Hamiltonian — acting on observables by conjugation, $a \mapsto U a U^\star$.

```

noncomputable abbrev Unitary (A : Type*) [OperatorAlgebra A]

```

structure State[\[source\]](#)

A state on A: a positive complex-linear functional normalized by $\omega 1 = 1$. ωa is the expected outcome of measuring observable a in this state — a state records everything that can be learned about the system by measurement.

```

structure State (A : Type*) [OperatorAlgebra A] where
  toPositiveLinearMap : A →p [C] C
  map_one : toPositiveLinearMap 1 = 1

```

abbrev Channel[\[source\]](#)

A channel from A_1 to A_2 — physicists' name for a unital completely positive (UCP) map, the most general notion of dynamics this framework expresses.

```

abbrev Channel (A1 A2 : Type*) [OperatorAlgebra A1] [OperatorAlgebra A2]

```

2.40 Physlib.QuantumMechanics.OperatorAlgebra.Dynamics.Automorphism

[Physlib/QuantumMechanics/OperatorAlgebra/Dynamics/Automorphism.lean](#) on GitHub.

lemma conjStarAlgAut_surjective[\[source\]](#)

Every $\star$ -automorphism of $B(H)$ is implemented by unitary conjugation.

```

lemma conjStarAlgAut_surjective :
  Function.Surjective (Unitary.conjStarAlgAut C (B(H)))

```

lemma conjStarAlgAut_eq_iff[\[source\]](#)

Two unitaries implement the same $\star$ -automorphism of $B(H)$ exactly when they differ by a scalar phase.

```

lemma conjStarAlgAut_eq_iff (u v : Unitary (B(H))) :
  Unitary.conjStarAlgAut C (B(H)) u =
    Unitary.conjStarAlgAut C (B(H)) v ↔
     $\exists c : \text{unitary } \mathbb{C}, u = c \cdot v$ 

```

def ProjectiveUnitary[\[source\]](#)

The projective unitary group of H , obtained by quotienting out scalar phases.

```
def ProjectiveUnitary (H : Type*) [NormedAddCommGroup H] [InnerProductSpace ℂ H]
  [CompleteSpace H]
```

instance Group (ProjectiveUnitary H)[\[source\]](#)

```
noncomputable instance : Group (ProjectiveUnitary H)
```

def projectiveUnitaryEquivStarAlgAut[\[source\]](#)

Reversible transformations of $B(H)$ are precisely projective unitaries.

```
noncomputable def projectiveUnitaryEquivStarAlgAut :
  ProjectiveUnitary H ≈* ((B(H)) ≈*a [ℂ] (B(H)))
```

2.41 Physlib.QuantumMechanics.OperatorAlgebra.HilbertSpace

[Physlib/QuantumMechanics/OperatorAlgebra/HilbertSpace.lean](#) on GitHub.

instance instOperatorAlgebraBoundedOperators[\[source\]](#)

*Bounded operators form an *OperatorAlgebra* using *Mathlib*'s native *Hilbert-space* instances.*

```
noncomputable instance instOperatorAlgebraBoundedOperators : OperatorAlgebra B(H)
```

abbrev Representation[\[source\]](#)

A Hilbert-space representation of A as a unital $\star$ -homomorphism into $B(H)$.

```
abbrev Representation (A H : Type*) [OperatorAlgebra A] [NormedAddCommGroup H]
  [InnerProductSpace ℂ H] [CompleteSpace H]
```

2.42 Physlib.QuantumMechanics.OperatorAlgebra.Observables.Jordan

[Physlib/QuantumMechanics/OperatorAlgebra/Observables/Jordan.lean](#) on GitHub.

def jordan[\[source\]](#)

The symmetrized product of two observables.

```
noncomputable def jordan (a b : Observable A) : Observable A
```

lemma coe_jordan[\[source\]](#)

```
lemma coe_jordan (a b : Observable A) :
  (a ⊙ b : A) = (2-1 : ℝ) • ((a : A) * b + (b : A) * a)
```

lemma jordan_comm[\[source\]](#)

```
lemma jordan_comm (a b : Observable A) :
  a ∘ b = b ∘ a
```

lemma jordan_self[\[source\]](#)

```
lemma jordan_self (a : Observable A) :
  (a ∘ a : A) = (a : A) * a
```

lemma add_jordan[\[source\]](#)

```
lemma add_jordan (a b c : Observable A) :
  (a + b) ∘ c = a ∘ c + b ∘ c
```

lemma jordan_add[\[source\]](#)

```
lemma jordan_add (a b c : Observable A) :
  a ∘ (b + c) = a ∘ b + a ∘ c
```

lemma jordan_smul[\[source\]](#)

```
lemma jordan_smul (t : ℝ) (a b : Observable A) :
  a ∘ (t • b) = t • (a ∘ b)
```

lemma jordan_identity[\[source\]](#)

```
lemma jordan_identity (a b : Observable A) :
  (a ∘ b) ∘ (a ∘ a) = a ∘ (b ∘ (a ∘ a))
```

def JordanObservable[\[source\]](#)

Observables equipped with their Jordan multiplication.

This is a type synonym for `Observable A`, kept a `def` (rather than an `abbrev`) so that the `NonUnitalNonAssocCommRing` structure defined below on `JordanObservable A` is not silently inherited by `Observable A` itself, which has no canonical multiplication of its own.

```
noncomputable def JordanObservable (A : Type*) [OperatorAlgebra A]
```

instance instAddCommGroup[\[source\]](#)

```
noncomputable instance instAddCommGroup : AddCommGroup (JordanObservable A)
```

instance instModule[\[source\]](#)

```
noncomputable instance instModule : Module ℝ (JordanObservable A)
```

instance instNonUnitalNonAssocCommRing[\[source\]](#)

```
noncomputable instance instNonUnitalNonAssocCommRing :
  NonUnitalNonAssocCommRing (JordanObservable A) where
```

instance instIsCommJordan[\[source\]](#)

```
noncomputable instance instIsCommJordan :
  IsCommJordan (JordanObservable A) where
```

2.43 Physlib.QuantumMechanics.OperatorAlgebra.Observables.Lie

[Physlib/QuantumMechanics/OperatorAlgebra/Observables/Lie.lean](#) on GitHub.

instance instBracket[\[source\]](#)

The observable Lie bracket is the imaginary part of the algebra product.

```
noncomputable instance instBracket :
  Bracket (Observable A) (Observable A)
```

lemma coe_bracket[\[source\]](#)

```
@[simp]
lemma coe_bracket (a b : Observable A) :
  (([a, b] : Observable A) : A) =
    (-(Complex.I / 2)) • ((a : A) * b - (b : A) * a)
```

lemma add_bracket[\[source\]](#)

```
lemma add_bracket (a b c : Observable A) :
  [a + b, c] = [a, c] + [b, c]
```

lemma bracket_add[\[source\]](#)

```
lemma bracket_add (a b c : Observable A) :
  [a, b + c] = [a, b] + [a, c]
```

lemma bracket_self[\[source\]](#)

```
lemma bracket_self (a : Observable A) :
  [a, a] = 0
```

lemma lie_smul[\[source\]](#)

```
private lemma lie_smul (r : ℂ) (x y : A) :
  [x, r • y] = r • [x, y]
```

lemma smul_lie[\[source\]](#)

```
private lemma smul_lie (r : ℂ) (x y : A) :
  [r • x, y] = r • [x, y]
```

lemma leibniz_bracket[\[source\]](#)

```
lemma leibniz_bracket (a b c : Observable A) :
  {a, {b, c}} = {{a, b}, c} + {b, {a, c}}
```

instance instLieRing[\[source\]](#)

```
noncomputable instance instLieRing :
  LieRing (Observable A) where
```

lemma bracket_smul[\[source\]](#)

```
lemma bracket_smul (t : ℝ) (a b : Observable A) :
  {a, t • b} = t • {a, b}
```

instance instLieAlgebra[\[source\]](#)

```
noncomputable instance instLieAlgebra :
  LieAlgebra ℝ (Observable A) where
```

lemma bracket_one_right[\[source\]](#)

```
@[simp]
lemma bracket_one_right (a : Observable A) :
  {a, (1 : Observable A)} = 0
```

lemma bracket_one_left[\[source\]](#)

```
@[simp]
lemma bracket_one_left (a : Observable A) :
  {(1 : Observable A), a} = 0
```

2.44 Physlib.QuantumMechanics.PoschlTeller.Basic

[Physlib/QuantumMechanics/PoschlTeller/Basic.lean](#) on GitHub.

structure PoschlTeller[\[source\]](#)

A Pöschl-Teller system is specified by the particle mass m , the width parameter κ , and family number N (all positive). -

```
structure PoschlTeller where
```

```
  m : ℝ
  κ : ℝ
  N : ℕ
  m_pos : 0 < m
  κ_pos : 0 < κ
  N_pos : 0 < N
```

def potential[\[source\]](#)

*The Pöschl-Teller potential is $-(\hbar^2 * \kappa^2 * N * (N + 1)) / (2 * m * (\cosh(\kappa * x))^2)$. -*

```
def potential (x : Space 1) : ℝ
```

abbrev HS

[\[source\]](#)

The Hilbert space for the Pöschl-Teller system is SpaceDHilbertSpace 1.

```
@[nolint unusedArguments]
```

```
abbrev HS (_ : PoschlTeller) : Type _
```

def tanhCLM

[\[source\]](#)

Pointwise multiplication of Schwartz maps by $\tanh(\kappa x)$.

```
def tanhCLM : S(Space 1, ℂ) → L[ℂ] S(Space 1, ℂ)
```

def creationCLM

[\[source\]](#)

The creation operator, $1/\sqrt{2m}$ $(P + i\hbar\kappa \tanh(\kappa X))$

```
def creationCLM : S(Space 1, ℂ) → L[ℂ] S(Space 1, ℂ)
```

def annihilationCLM

[\[source\]](#)

The annihilation operator, $1/\sqrt{2m}$ $(P - i\hbar\kappa \tanh(\kappa X))$

```
def annihilationCLM : S(Space 1, ℂ) → L[ℂ] S(Space 1, ℂ)
```

def tanhOperator

[\[source\]](#)

The unbounded operator defined by pointwise multiplication by $\tanh(\kappa x)$.

```
def tanhOperator : Q.HS →l.[ℂ] Q.HS
```

def creationOperator

[\[source\]](#)

The creation unbounded operator, $1/\sqrt{2m}$ $(P + i\hbar\kappa \tanh(\kappa X))$

```
def creationOperator : Q.HS →l.[ℂ] Q.HS
```

def annihilationOperator

[\[source\]](#)

The annihilation unbounded operator, $1/\sqrt{2m}$ $(P - i\hbar\kappa \tanh(\kappa X))$

```
def annihilationOperator : Q.HS →l.[ℂ] Q.HS
```

2.45 Physlib.QuantumMechanics.QuantumSystem.Basic

[Physlib/QuantumMechanics/QuantumSystem/Basic.lean](#) on GitHub.

def mkESA[\[source\]](#)

Create a quantum system from a Hamiltonian operator which is merely essentially self-adjoint by taking its closure.

```
def mkESA {HS : Type*} [NormedAddCommGroup HS] [InnerProductSpace ℂ HS] [CompleteSpace HS]
  {H : HS →ℂ HS} (hH : IsEssentiallySelfAdjoint H) : QuantumSystem
```

2.46 Physlib.Relativity.LorentzGroup.Boosts.Axis

[Physlib/Relativity/LorentzGroup/Boosts/Axis.lean](#) on GitHub.

def SL2C.boostAxis[\[source\]](#)

*The $SL(2, \mathbb{C})$ lift of the boost along spatial axis i , with $\theta = x$, $1 = y$, and $2 = z$. The parameter t is multiplicative, and for $t > 0$ the rapidity is $2 * \log t$.*

```
noncomputable def boostAxis : Fin 3 → (t : ℝ) → t ≠ 0 → SL(2, ℂ)
| 0, t, ht =>
  (!!![(t : ℂ) + (t : ℂ)-1 / 2, ((t : ℂ) - (t : ℂ)-1) / 2;
    ((t : ℂ) - (t : ℂ)-1) / 2, ((t : ℂ) + (t : ℂ)-1) / 2], by
  have htc : (t : ℂ) ≠ 0 := Complex.ofReal_ne_zero.mpr ht
  rw [Matrix.det_fin_two_of]
  field_simp
  ring)
| 1, t, ht =>
  (!!![(t : ℂ) + (t : ℂ)-1 / 2, -Complex.I * ((t : ℂ) - (t : ℂ)-1) / 2;
    Complex.I * ((t : ℂ) - (t : ℂ)-1) / 2, ((t : ℂ) + (t : ℂ)-1) / 2], by
  have htc : (t : ℂ) ≠ 0 := Complex.ofReal_ne_zero.mpr ht
  have h2 : -Complex.I * ((t : ℂ) - (t : ℂ)-1) / 2 *
    (Complex.I * ((t : ℂ) - (t : ℂ)-1) / 2) =
    ((t : ℂ) - (t : ℂ)-1) / 2 * ((t : ℂ) - (t : ℂ)-1) / 2 := by
  have hI : -Complex.I * Complex.I = 1 := by
  rw [neg_mul, Complex.I_mul_I, neg_neg]
  calc -Complex.I * ((t : ℂ) - (t : ℂ)-1) / 2 *
    (Complex.I * ((t : ℂ) - (t : ℂ)-1) / 2)
  = (-Complex.I * Complex.I) *
    (((t : ℂ) - (t : ℂ)-1) / 2 * ((t : ℂ) - (t : ℂ)-1) / 2) := by
  ring
  _ = ((t : ℂ) - (t : ℂ)-1) / 2 * ((t : ℂ) - (t : ℂ)-1) / 2 := by
  rw [hI, one_mul]
  rw [Matrix.det_fin_two_of, h2]
  field_simp
  ring)
| 2, t, ht =>
  (!!![(t : ℂ), 0; 0, (t : ℂ)-1], by
  have htc : (t : ℂ) ≠ 0 := Complex.ofReal_ne_zero.mpr ht
  rw [Matrix.det_fin_two_of]
  simp [mul_inv_cancel, htc])
```

lemma boostAxis_inv[\[source\]](#)

Inverting an axis boost replaces its multiplicative parameter t by t^{-1} .

```
lemma boostAxis_inv (i : Fin 3) (t : ℝ) (ht : t ≠ 0) :
  (boostAxis i t ht)-1 = boostAxis i t-1 (inv_ne_zero ht)
```

lemma boostAxis_conjTranspose[\[source\]](#)

The matrix underlying an axis-boost lift is Hermitian.

```
lemma boostAxis_conjTranspose (i : Fin 3) (t : ℝ) (ht : t ≠ 0) :
  (boostAxis i t ht).1H = (boostAxis i t ht).1
```

lemma boostAxis_eq_conj[\[source\]](#)

Every axis boost is obtained by conjugating the z-axis boost by rotationZToAxis.

```
lemma boostAxis_eq_conj (i : Fin 3) (t : ℝ) (ht : t ≠ 0) :
  boostAxis i t ht =
    rotationZToAxis i * boostAxis 2 t ht * (rotationZToAxis i)-1
```

lemma exists_conj_boostAxis[\[source\]](#)

Every coordinate-axis boost is conjugate to the z-axis boost.

```
lemma exists_conj_boostAxis (i : Fin 3) :
  ∃ R : SL(2,ℂ), ∀ (t : ℝ) (ht : t ≠ 0),
    boostAxis i t ht = R * boostAxis 2 t ht * R-1
```

def LorentzGroup.boostAxis[\[source\]](#)

The Lorentz transformation induced by the multiplicatively parameterized $SL(2, \mathbb{C})$ boost along spatial axis i .

```
noncomputable def boostAxis (i : Fin 3) (t : ℝ) (ht : t ≠ 0) : LorentzGroup 3
```

lemma boostAxis_apply[\[source\]](#)

The entries of an axis boost in the Lorentz group.

```
lemma boostAxis_apply (i : Fin 3) (t : ℝ) (ht : t ≠ 0) (a b : Fin 1 ⊕ Fin 3) :
  (boostAxis i t ht).1 a b =
    if a = Sum.inl 0 ∧ b = Sum.inl 0 then (t ^ 2 + (t-1) ^ 2) / 2
    else if a = Sum.inl 0 ∧ b = Sum.inr i then -((t ^ 2 - (t-1) ^ 2) / 2)
    else if a = Sum.inr i ∧ b = Sum.inl 0 then -((t ^ 2 - (t-1) ^ 2) / 2)
    else if a = Sum.inr i ∧ b = Sum.inr i then (t ^ 2 + (t-1) ^ 2) / 2
    else if a = b then 1 else 0
```

2.47 Physlib.Relativity.MinkowskiMatrix

[Physlib/Relativity/MinkowskiMatrix.lean](#) on GitHub.

lemma vecMul_apply[\[source\]](#)

Right multiplication of a row vector by the Minkowski matrix multiplies each component by the corresponding diagonal sign.

```
lemma vecMul_apply (v : (Fin 1 ⊕ Fin d) → ℝ) (μ : Fin 1 ⊕ Fin d) :
  (v v * minkowskiMatrix) μ = v μ * minkowskiMatrix μ μ
```

lemma prod_diagonal[\[source\]](#)

The product of all diagonal entries of the Minkowski matrix is $(-1)^d$.

```
lemma prod_diagonal : ∏ μ : Fin 1 ⊕ Fin d, minkowskiMatrix μ μ = (-1) ^ d
```

lemma prod_diagonal_comp_of_injective[\[source\]](#)

Reindexing all diagonal entries injectively does not change their product.

```
lemma prod_diagonal_comp_of_injective {v : Fin (d + 1) → Fin 1 ⊕ Fin d}
(hv : Function.Injective v) :
  ∏ i, minkowskiMatrix (v i) (v i) = (-1) ^ d
```

2.48 Physlib.Relativity.PauliMatrices.Basic

[Physlib/Relativity/PauliMatrices/Basic.lean](#) on GitHub.

lemma pauliMatrix_anticommutator[\[source\]](#)

Pauli matrices satisfy $\{\sigma_i, \sigma_j\} = 2 \delta_{ij} I$.

```
lemma pauliMatrix_anticommutator (i j : Fin 3) :
  pauliMatrix (Sum.inr i) * pauliMatrix (Sum.inr j) +
  pauliMatrix (Sum.inr j) * pauliMatrix (Sum.inr i) =
    ((2 * kroneckerDelta i j : ℕ) : ℂ) •
    (1 : Matrix (Fin 2) (Fin 2) ℂ)
```

def vectorMatrix[\[source\]](#)

The matrix $a \cdot \sigma$ associated to a real three-vector a .

```
noncomputable def vectorMatrix (a : Fin 3 → ℝ) :
  Matrix (Fin 2) (Fin 2) ℂ
```

lemma vectorMatrix_anticommutator[\[source\]](#)

The anticommutator of two Pauli vectors is twice their Euclidean dot product times the identity.

```
lemma vectorMatrix_anticommutator (a b : Fin 3 → ℝ) :
  vectorMatrix a * vectorMatrix b + vectorMatrix b * vectorMatrix a =
    ((2 * (a ·v b) : ℝ) : ℂ) •
    (1 : Matrix (Fin 2) (Fin 2) ℂ)
```

lemma vectorMatrix_commutator[\[source\]](#)

The commutator of two Pauli vectors is twice i times the Pauli vector associated to their cross product.

```
lemma vectorMatrix_commutator (a b : Fin 3 → ℝ) :
  vectorMatrix a * vectorMatrix b - vectorMatrix b * vectorMatrix a =
    (2 * Complex.I) • vectorMatrix (a ×3 b)
```

lemma vectorMatrix_mul_vectorMatrix[\[source\]](#)

Product formula for Pauli vectors: $(a \cdot \sigma)(b \cdot \sigma) = (a \cdot b) I + i (a \times b) \cdot \sigma$.

```
lemma vectorMatrix_mul_vectorMatrix (a b : Fin 3 → ℝ) :
  vectorMatrix a * vectorMatrix b =
    ((a ·v b : ℝ) : ℂ) •
      (1 : Matrix (Fin 2) (Fin 2) ℂ) +
      Complex.I • vectorMatrix (a ×3 b)
```

lemma vectorMatrix_sq[\[source\]](#)

The square of $a \cdot \sigma$ is $|a|^2 I$.

```
lemma vectorMatrix_sq (a : Fin 3 → ℝ) :
  vectorMatrix a * vectorMatrix a =
    (∑ i : Fin 3, a i ^ 2 : ℝ) • 1
```

2.49 Physlib.Relativity.PauliMatrices.Relations

[Physlib/Relativity/PauliMatrices/Relations.lean](#) on GitHub.

lemma pauliContr_mul_dualWeyl_eq_ofRat[\[source\]](#)

Rational-complex components of a contraction of $\sigma^{^^}$ with a copy whose Weyl indices are dualized.

```
lemma pauliContr_mul_dualWeyl_eq_ofRat :
  {σ^^ | μ α β ⊗ σ^^ | ν τ(α') τ(β)}T = ofRat (fun b =>
    ∑ x : Fin 2, pauliContrComponent (b 0) (b 1) x *
      pauliContrDownComponent (b 2) x (b 3))
```

lemma dualWeyl_mul_pauliContr_eq_ofRat[\[source\]](#)

Rational-complex components of the reverse contraction of a Weyl-dualized $\sigma^{^^}$ with $\sigma^{^^}$.

```
lemma dualWeyl_mul_pauliContr_eq_ofRat :
  {σ^^ | μ τ(α) τ(β) ⊗ σ^^ | ν α β'}T = ofRat (fun b =>
    ∑ x : Fin 2, pauliContrDownComponent (b 0) (b 1) x *
      pauliContrComponent (b 2) x (b 3))
```

lemma leviCivita_mul_pauliDual[\[source\]](#)

Contracting $\varepsilon_{4\mathbb{C}}$ with a Lorentz-dualized $\sigma^{^^}$ agrees with contracting it with pauliCo .

```
lemma leviCivita_mul_pauliDual :
  ({ε4ℂ | μ ν ρ κ ⊗ σ^^ | τ(κ) α β =
    ε4ℂ | μ ν ρ κ ⊗ σ^^ | κ α β'}T : Prop)
```

lemma pauliContr_mul_pauliContrDown_mul_pauliContr[\[source\]](#)

Equation (2.26), the three-Pauli identity $\hat{\sigma}^\mu \bar{\sigma}^\nu \hat{\sigma}^\rho = g^{\{\mu\nu\}} \hat{\sigma}^\rho - g^{\{\mu\rho\}} \hat{\sigma}^\nu + g^{\{\nu\rho\}} \hat{\sigma}^\mu + i \varepsilon^{\{\mu\nu\rho\}} \sigma_{\underline{\kappa}}$, with barred and lowered forms expressed through index dualization τ .

```

lemma pauliContr_mul_pauliContrDown_mul_pauliContr : ({
  σ^^ | μ α β ⊗ σ^^ | ν τ(α') τ(β) ⊗ σ^^ | ρ α' β' =
  (((((η | μ ν ⊗ σ^^ | ρ α β') + (-(η | μ ρ ⊗ σ^^ | ν α β'))))
    + (η | ν ρ ⊗ σ^^ | μ α β'))
    + (Complex.I •τ (ε4ℂ | μ ν ρ κ ⊗ σ^^ | τ(κ) α β'))))
}^T : Prop)

```

lemma pauliContrDown_mul_pauliContr_mul_pauliContrDown

[source]

Equation (2.27), the conjugate three-Pauli identity $\bar{\sigma}^\mu \sigma^\nu \bar{\sigma}^\rho = g^{\{\mu\nu\}} \bar{\sigma}^\rho - g^{\{\mu\rho\}} \bar{\sigma}^\nu + g^{\{\nu\rho\}} \bar{\sigma}^\mu - i \varepsilon^{\{\mu\nu\rho\}} \bar{\sigma}_\kappa$, with barred and lowered forms expressed through index dualization τ .

```

lemma pauliContrDown_mul_pauliContr_mul_pauliContrDown : ({
  σ^^ | μ τ(α) τ(β) ⊗ σ^^ | ν α β' ⊗ σ^^ | ρ τ(α') τ(β') =
  (((((η | μ ν ⊗ σ^^ | ρ τ(α') τ(β))
    + (-(η | μ ρ ⊗ σ^^ | ν τ(α') τ(β))))
    + (η | ν ρ ⊗ σ^^ | μ τ(α') τ(β)))
    + ((-Complex.I) •τ (ε4ℂ | μ ν ρ κ ⊗ σ^^ | τ(κ) τ(α') τ(β))))
}^T : Prop)

```

2.50 Physlib.Relativity.PauliMatrices.SelfAdjoint

Physlib/Relativity/PauliMatrices/SelfAdjoint.lean on GitHub.

lemma trace_pauliMatrix_mul_pauliMatrix

[source]

Pauli matrices are orthogonal with respect to the trace pairing: $\text{tr}(\sigma_\mu \sigma_\nu) = \delta_{\mu\nu}$.

```

@[simp]
lemma trace_pauliMatrix_mul_pauliMatrix (μ ν : Fin 1 ⊕ Fin 3) :
  Matrix.trace (pauliMatrix μ * pauliMatrix ν) = ((2 * kroneckerDelta μ ν : ℕ) : ℂ)

```

def pauliCoeff

[source]

The four real Pauli coefficients of a self-adjoint 2×2 matrix.

```

noncomputable def pauliCoeff
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  Fin 1 ⊕ Fin 3 → ℝ
| Sum.inl 0 => 1 / 2 * (Matrix.trace (σ0 * A.1)).re
| Sum.inr 0 => 1 / 2 * (Matrix.trace (σ1 * A.1)).re
| Sum.inr 1 => 1 / 2 * (Matrix.trace (σ2 * A.1)).re
| Sum.inr 2 => 1 / 2 * (Matrix.trace (σ3 * A.1)).re

```

lemma sum_pauliCoeff

[source]

Every self-adjoint 2×2 matrix is its Pauli decomposition.

```

lemma sum_pauliCoeff
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  ∑ i, pauliCoeff A i • pauliSelfAdjoint i = A

```

lemma eq_sum_pauli[\[source\]](#)

Every self-adjoint 2×2 matrix is a real linear combination of the Pauli matrices.

```
lemma eq_sum_pauli
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  A = ∑ i, pauliCoeff A i • pauliSelfAdjoint i
```

def scalarCoeff[\[source\]](#)

The coefficient of the identity in the Pauli decomposition.

```
noncomputable def scalarCoeff
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) : ℝ
```

def vectorCoeff[\[source\]](#)

The three spatial Pauli coefficients.

```
noncomputable def vectorCoeff
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) : Fin 3 → ℝ
```

def vectorPart[\[source\]](#)

The traceless Pauli-vector part $\mathbf{a} \cdot \boldsymbol{\sigma}$.

```
noncomputable def vectorPart
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  Matrix (Fin 2) (Fin 2) ℂ
```

def pauliRadius[\[source\]](#)

The Euclidean length of the spatial Pauli coefficients.

```
noncomputable def pauliRadius
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) : ℝ
```

lemma pauliRadius_sq[\[source\]](#)

The Pauli radius is the square root of the squared Euclidean length of the spatial Pauli coefficients.

```
lemma pauliRadius_sq
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  pauliRadius A ^ 2 =
  ∑ i : Fin 3, vectorCoeff A i ^ 2
```

lemma pauliRadius_nonneg[\[source\]](#)

```
@[simp]
lemma pauliRadius_nonneg
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  0 ≤ pauliRadius A
```

lemma matrix_eq_scalar_add_vector[\[source\]](#)

A self-adjoint matrix is its scalar part plus its Pauli-vector part.

```
lemma matrix_eq_scalar_add_vector
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  A.val = (scalarCoeff A : ℂ) • 1 + vectorPart A
```

lemma trace_eq_two_mul_scalarCoeff[\[source\]](#)

The trace of a self-adjoint matrix is twice its scalar Pauli coefficient.

```
lemma trace_eq_two_mul_scalarCoeff
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  Matrix.trace A.1 = 2 * scalarCoeff A
```

lemma trace_vectorPart[\[source\]](#)

The Pauli-vector part has zero trace.

```
@[simp]
lemma trace_vectorPart
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  Matrix.trace (vectorPart A) = 0
```

lemma vectorPart_sq[\[source\]](#)

The square of the vector part of a self-adjoint matrix is its squared Pauli radius times the identity.

```
lemma vectorPart_sq
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  vectorPart A * vectorPart A =
    (pauliRadius A ^ 2 : ℝ) • 1
```

lemma trace_pauliSelfAdjoint'_mul[\[source\]](#)

Trace orthogonality of the covariant Pauli basis: $\text{tr} (\sigma'_a \sigma'_b) = 2 \delta_{\{a\ b\}}$.

```
lemma trace_pauliSelfAdjoint'_mul (a b : Fin 1 ⊕ Fin 3) :
  Matrix.trace ((pauliSelfAdjoint' a).1 * (pauliSelfAdjoint' b).1) =
    if a = b then 2 else 0
```

lemma trace_pauliSelfAdjoint'_mul_apply[\[source\]](#)

The trace pairing of a covariant Pauli matrix with an arbitrary matrix, expressed through the matrix entries.

```
lemma trace_pauliSelfAdjoint'_mul_apply (l : Fin 1 ⊕ Fin 3)
  (N : Matrix (Fin 2) (Fin 2) ℂ) :
  Matrix.trace ((pauliSelfAdjoint' l).1 * N) =
    match l with
    | Sum.inl 0 => N 0 0 + N 1 1
    | Sum.inr 0 => -(N 0 1 + N 1 0)
    | Sum.inr 1 => -(Complex.I * (N 0 1 - N 1 0))
    | Sum.inr 2 => -(N 0 0 - N 1 1)
```

2.51 Physlib.Relativity.PauliMatrices.ToTensor

[Physlib/Relativity/PauliMatrices/ToTensor.lean](#) on GitHub.

def pauliContrComponent

[\[source\]](#)

Rational-complex components of the contravariant Pauli four-vector.

```
def pauliContrComponent (mu : Fin 4) (a b : Fin 2) : Physlib.RatComplexNum
```

def pauliContrDownComponent

[\[source\]](#)

Rational-complex components of the contravariant conjugate Pauli four-vector.

```
def pauliContrDownComponent (mu : Fin 4) (a b : Fin 2) : Physlib.RatComplexNum
```

lemma toTensor_dualLeft_eq_ofRat

[\[source\]](#)

Rational-complex components of $\sigma^{\mu\nu}$ after dualizing its left-handed Weyl index.

```
lemma toTensor_dualLeft_eq_ofRat :
  {σμν | μ τ(α) β}T =
    ofRat (fun b =>
      ∑ x : Fin 2, pauliContrComponent (b 0) x (b 2) *
        (if x.val = 0 ∧ (b 1).val = 1 then 1 else
          if (b 1).val = 0 ∧ x.val = 1 then -1 else 0))
```

lemma toTensor_dualWeyl_eq_ofRat

[\[source\]](#)

Rational-complex components of $\sigma^{\mu\nu}$ after dualizing both Weyl indices.

```
lemma toTensor_dualWeyl_eq_ofRat :
  {σμν | μ τ(α) τ(β)}T =
    ofRat (fun b =>
      pauliContrDownComponent (b 0) (b 2) (b 1))
```

lemma toTensor_dualLorentz_eq_ofRat

[\[source\]](#)

Rational-complex components of $\sigma^{\mu\nu}$ after dualizing its Lorentz index.

```
lemma toTensor_dualLorentz_eq_ofRat :
  {σμν | τ(μ) α β}T =
    ofRat (fun b => pauliContrDownComponent (b 0) (b 1) (b 2))
```

lemma toTensor_dualLorentzLeft_eq_ofRat

[\[source\]](#)

Rational-complex components of $\sigma^{\mu\nu}$ after dualizing its Lorentz and left-handed Weyl indices.

```
lemma toTensor_dualLorentzLeft_eq_ofRat :
  {σμν | τ(μ) τ(α) β}T = ofRat (fun b =>
    ∑ x : Fin 2, pauliContrDownComponent (b 0) x (b 2) *
      (if x.val = 0 ∧ (b 1).val = 1 then 1 else
        if (b 1).val = 0 ∧ x.val = 1 then -1 else 0))
```

lemma toTensor_dualAll_eq_ofRat[\[source\]](#)

Rational-complex components of $\sigma^{^^}$ after dualizing all three indices.

```
lemma toTensor_dualAll_eq_ofRat :
  { $\sigma^{^^}$  |  $\tau(\mu) \tau(\alpha) \tau(\beta)\}^T =$ 
    ofRat (fun b => pauliContrComponent (b 0) (b 2) (b 1))
```

lemma toTensor_dualWeyl_eq_pauliContrDown[\[source\]](#)

Dualizing both Weyl indices of $\sigma^{^^}$ gives $\sigma^_{}$.

```
lemma toTensor_dualWeyl_eq_pauliContrDown :
  ({ $\sigma^{^^}$  |  $\mu \tau(\alpha) \tau(\beta) = \sigma^_{}$  |  $\mu \beta \alpha\}^T : \text{Prop})$ 
```

lemma toTensor_dualAll_eq_pauliCoDown[\[source\]](#)

Dualizing all three indices of $\sigma^{^^}$ gives $\sigma_{}$.

```
lemma toTensor_dualAll_eq_pauliCoDown :
  ({ $\sigma^{^^}$  |  $\tau(\mu) \tau(\alpha) \tau(\beta) = \sigma_{}$  |  $\mu \beta \alpha\}^T : \text{Prop})$ 
```

lemma pauliDual_eq_pauliCo[\[source\]](#)

Lowering the Lorentz index of $\sigma^{^^}$ with τ gives $\sigma^_{}$.

```
lemma pauliDual_eq_pauliCo :
  ({ $\sigma^{^^}$  |  $\tau(\mu) \alpha \beta = \sigma^_{}$  |  $\mu \alpha \beta\}^T : \text{Prop})$ 
```

lemma pauliContrDownDual_eq_pauliCoDown[\[source\]](#)

Lowering the Lorentz index of $\sigma^_{}$ with τ gives $\sigma_{}$.

```
lemma pauliContrDownDual_eq_pauliCoDown :
  ({ $\sigma^_{}$  |  $\tau(\mu) \beta \alpha = \sigma_{}$  |  $\mu \beta \alpha\}^T : \text{Prop})$ 
```

2.52 Physlib.Relativity.SL2C.AxisRotations

[Physlib/Relativity/SL2C/AxisRotations.lean](#) on GitHub.

def rotationZToAxis[\[source\]](#)

The $SL(2, \mathbb{C})$ rotation carrying the z-axis to axis i.

```
noncomputable def rotationZToAxis : Fin 3 → SL(2, ℂ)
| 0 =>
  (((Real.sqrt 2 : ℝ) : ℂ))⁻¹ • !![1, -1; 1, 1], by
    rw [Matrix.det_smul, Matrix.det_fin_two_of, Fintype.card_fin, inv_pow]
    norm_num [← Complex.ofReal_pow, Real.sq_sqrt]
| 1 =>
  (((Real.sqrt 2 : ℝ) : ℂ))⁻¹ • !![1, Complex.I; Complex.I, 1], by
    rw [Matrix.det_smul, Matrix.det_fin_two_of, Fintype.card_fin, inv_pow,
      Complex.I_mul_I]
    norm_num [← Complex.ofReal_pow, Real.sq_sqrt]
| 2 => 1
```

lemma rotationZToAxis_zero_mul_diagonal_mul_inv[\[source\]](#)

Conjugating $\text{diag}(a, b)$ by the rotation to the x-axis expresses it in the x-axis basis.

```
lemma rotationZToAxis_zero_mul_diagonal_mul_inv (a b : ℂ) :
  (rotationZToAxis 0).1 * !![a, 0; 0, b] * ((rotationZToAxis 0)⁻¹).1 =
    !![(a + b) / 2, (a - b) / 2; (a - b) / 2, (a + b) / 2]
```

lemma rotationZToAxis_one_mul_diagonal_mul_inv[\[source\]](#)

Conjugating $\text{diag}(a, b)$ by the rotation to the y-axis expresses it in the y-axis basis.

```
lemma rotationZToAxis_one_mul_diagonal_mul_inv (a b : ℂ) :
  (rotationZToAxis 1).1 * !![a, 0; 0, b] * ((rotationZToAxis 1)⁻¹).1 =
    !![(a + b) / 2, -Complex.I * (a - b) / 2;
      Complex.I * (a - b) / 2, (a + b) / 2]
```

lemma rotationZToAxis_two_mul_diagonal_mul_inv[\[source\]](#)

Conjugating $\text{diag}(a, b)$ by the identity rotation leaves it unchanged.

```
lemma rotationZToAxis_two_mul_diagonal_mul_inv (a b : ℂ) :
  (rotationZToAxis 2).1 * !![a, 0; 0, b] * ((rotationZToAxis 2)⁻¹).1 =
    !![a, 0; 0, b]
```

2.53 Physlib.Relativity.SL2C.Basic

[Physlib/Relativity/SL2C/Basic.lean](#) on GitHub.

lemma toLorentzGroup_eq_trace[\[source\]](#)

The matrix elements of the covering map through the trace pairing: $L(M)_{\{i\ j\}} = \frac{1}{2} \text{tr}(\sigma'_i \cdot M \sigma'_j M^\dagger)$.

```
lemma toLorentzGroup_eq_trace (M : SL(2, ℂ)) (i j : Fin 1 ⊕ Fin 3) :
  (((toLorentzGroup M).1 i j : ℝ) : ℂ) =
    Matrix.trace ((PauliMatrix.pauliSelfAdjoint' i).1 *
      (M.1 * (PauliMatrix.pauliSelfAdjoint' j).1 * M.1H)) / 2
```

2.54 Physlib.Relativity.Tensors.ComponentIdx.Contraction

[Physlib/Relativity/Tensors/ComponentIdx/Contraction.lean](#) on GitHub.

lemma ofFinEquiv_dropPair[\[source\]](#)

Restoring the two entries dropped from a component index recovers that component index.

```
@[simp]
lemma ofFinEquiv_dropPair {n : ℕ} {c : Fin (n + 1 + 1) → C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (r : ComponentIdx (S := S) c) :
  (ofFinEquiv (S := S) hij (r.dropPair i j) (r i, r j)).1 = r
```

2.55 Physlib.Relativity.Tensors.Contraction.CrossToSlot

[Physlib/Relativity/Tensors/Contraction/CrossToSlot.lean](#) on GitHub.

lemma crossToSlot_basis_repr_apply[\[source\]](#)

A component of `crossToSlot` is the corresponding component of `crossToEnd`, with the surviving indices rotated back into the contracted slot.

```
lemma crossToSlot_basis_repr_apply {nA : ℕ} {c : Fin (nA + 1) → C} {cM : Fin 2 → C}
  (i : Fin (nA + 1)) (j : Fin 2) (hc : S.τ (c i) = cM j) (M : Tensor S cM)
  (t : Tensor S c)
  (φ : ComponentIdx (S := S) (Function.update c i (cM (j.succAbove 0)))) :
  (basis _).repr (crossToSlot i j hc M t) φ =
    (basis _).repr (crossToEnd i j hc t M) (fun m =>
      basisIdxCongr ((IsReindexing.crossToSlot_cycle i j).inv_perserve_color m)
        (φ ((IsReindexing.crossToSlot_cycle i j).inv
          (Fin.cycleIcc i (Fin.last nA)).symm m)))
```

2.56 Physlib.Relativity.Tensors.Contraction.SuccSuccAbove

[Physlib/Relativity/Tensors/Contraction/SuccSuccAbove.lean](#) on GitHub.

lemma succSuccAbove_succAbove_comm_apply[\[source\]](#)

Pointwise form of commuting deletion of one slot with deletion of a pair.

```
lemma succSuccAbove_succAbove_comm_apply {n : ℕ} (i j : Fin (n + 1 + 1 + 1))
  (k : Fin (n + 1)) (m : Fin n) :
  (i.succSuccAbove j k).succAbove
    (((Fin.predAbove 0 (i.succSuccAbove j k)).predAbove i).succSuccAbove
      ((Fin.predAbove 0 (i.succSuccAbove j k)).predAbove j) m) =
    i.succSuccAbove j (k.succAbove m)
```

2.57 Physlib.Relativity.Tensors.Evaluation

[Physlib/Relativity/Tensors/Evaluation.lean](#) on GitHub.

lemma evalT_evalT_basis[\[source\]](#)

Commutation of two evaluations on a tensor basis vector.

```
lemma evalT_evalT_basis {n : ℕ} {c : Fin (n + 1 + 1) → C}
  (k1 : Fin (n + 1 + 1)) (k2 : Fin (n + 1)) (φ1 : basisIdx (c k1))
  (φ2 : basisIdx ((c ∘ k1.succAbove) k2)) (ψ : ComponentIdx (S := S) c) :
  evalT k2 φ2 (evalT k1 φ1 (basis (S := S) c ψ)) =
    permT id (IsReindexing.succAbove_succAbove_comm k1 k2)
      (evalT (k2.predAbove k1)
        (basisIdxCongr
          (congrArg c (Fin.succAbove_succAbove_predAbove k1 k2).symm) φ1)
        (evalT (k1.succAbove k2) φ2 (basis (S := S) c ψ)))
```

lemma evalT_evalT[\[source\]](#)

Evaluating two tensor indices commutes, up to the canonical reindexing identifying the two possible orders in which the indices are removed.

```
lemma evalT_evalT {n : ℕ} {c : Fin (n + 1 + 1) → C}
  (k1 : Fin (n + 1 + 1)) (k2 : Fin (n + 1)) (φ1 : basisIdx (c k1))
  (φ2 : basisIdx ((c ∘ k1.succAbove) k2)) (t : Tensor S c) :
  evalT k2 φ2 (evalT k1 φ1 t) =
```

```
permT id (IsReindexing.succAbove_succAbove_comm k1 k2)
  (evalT (k2.predAbove k1)
    (basisIdxCongr
      (congrArg c (Fin.succAbove_succAbove_predAbove k1 k2).symm) φ1)
    (evalT (k1.succAbove k2) φ2 t))
```

2.58 Physlib.Relativity.Tensors.LeviCivita.Basic

[Physlib/Relativity/Tensors/LeviCivita/Basic.lean](#) on GitHub.

lemma euclidLeviCivita_eq_leviCivitaSymbol

[\[source\]](#)

The Euclidean Levi-Civita symbol in dimension 4 is the general-dimension Levi-Civita symbol `leviCivitaSymbol` at $\iota = \text{Fin } 4$, carried to the reals.

```
lemma _root_.euclidLeviCivita_eq_leviCivitaSymbol (g : Fin 4 → Fin 4) :
  euclidLeviCivita g = (leviCivitaSymbol g : ℝ)
```

lemma leviCivita_basis_repr_eq_leviCivitaSymbol

[\[source\]](#)

The components of the Levi-Civita tensor in the standard basis are the general-dimension Levi-Civita symbol `leviCivitaSymbol` of the multi-index at $\iota = \text{Fin } 4$.

```
lemma leviCivita_basis_repr_eq_leviCivitaSymbol
  (b : ComponentIdx (S := realLorentzTensor 3) ![Color.up, Color.up, Color.up, Color.up]) :
  (Tensor.basis _).repr ε4 b
    = (leviCivitaSymbol (fun i => finSumFinEquiv (b i)) : ℝ)
```

2.59 Physlib.Relativity.Tensors.LeviCivita.Complex

[Physlib/Relativity/Tensors/LeviCivita/Complex.lean](#) on GitHub.

lemma leviCivita_isReindexing

[\[source\]](#)

The index colors of the complexified real Levi-Civita tensor agree with four complex contravariant Lorentz indices.

```
lemma leviCivita_isReindexing : IsReindexing
  (realLorentzTensor.colorToComplex ◦
    ![realLorentzTensor.Color.up, realLorentzTensor.Color.up,
      realLorentzTensor.Color.up, realLorentzTensor.Color.up])
  ![Color.up, Color.up, Color.up, Color.up] id
```

lemma basisIdxCongr_val

[\[source\]](#)

A color cast in the complex Lorentz basis preserves the underlying finite index value.

```
private lemma basisIdxCongr_val {c c1 : Color} (h : c = c1)
  (i : Fin (complexLorentzTensor.repDim c)) :
  (TensorSpecies.basisIdxCongr
    (basisIdx := fun c => Fin (complexLorentzTensor.repDim c)) h i).val = i.val
```

def leviCivita[\[source\]](#)

The Levi-Civita tensor $\varepsilon^{up\delta}$ as a complex Lorentz tensor, with $\varepsilon^{0123} = 1$.

```
noncomputable def leviCivita : CT[.up, .up, .up, .up]
```

lemma leviCivita_eq_ofRat[\[source\]](#)

The complex Levi-Civita tensor has the Levi-Civita symbol as its real component and zero imaginary component in the standard basis.

```
lemma leviCivita_eq_ofRat : ε4ℂ = ofRat (fun
  b : ComponentIdx (S := complexLorentzTensor) ![Color.up, Color.up, Color.up, Color.up] =>
  (generalizedKroneckerDelta
    (fun i => Fin.cast (by fin_cases i <=> rfl) (b i)) (id : Fin 4 → Fin 4), 0))
```

2.60 Physlib.Relativity.Tensors.LeviCivita.Contractions

[Physlib/Relativity/Tensors/LeviCivita/Contractions.lean](#) on GitHub.

lemma euclidLeviCivita_symbol_contract_one_last[\[source\]](#)

***Triple Euclidean Levi-Civita contraction with the free index last.** This is the same contraction as euclidLeviCivita_symbol_contract_one, in the slot order produced by the tensor notation for $\varepsilon^{\{\mu\nu\rho\}} \varepsilon_{\{\mu\nu\rho\}}$.*

```
lemma euclidLeviCivita_symbol_contract_one_last (a b : Fin 4) :
  ∑ h : Fin 3 → Fin 4, euclidLeviCivita (Fin.snoc h a) * euclidLeviCivita (Fin.snoc h b)
  = 6 * ((kroneckerDelta a b : ℕ) : ℝ)
```

lemma leviCivita_lowered_basis_repr_apply[\[source\]](#)

Lowering all four indices of the Levi-Civita tensor changes the sign of every standard-basis component. This is the tensor-component form of the Lorentzian orientation factor $\det \eta = -1$.

```
lemma leviCivita_lowered_basis_repr_apply
  (b : ComponentIdx (S := realLorentzTensor 3)
    ![Color.down, Color.down, Color.down, Color.down]) :
  (Tensor.basis _).repr ({ε4 | τ(μ) τ(ν) τ(ρ) τ(σ)}T) b =
  - (Tensor.basis _).repr ε4 b
```

lemma leviCivita_basis_contract_self[\[source\]](#)

The sum of the squared standard-basis components of the contravariant Levi-Civita tensor is $4! = 24$.

```
lemma leviCivita_basis_contract_self :
  ∑ b : ComponentIdx (S := realLorentzTensor 3)
    ![Color.up, Color.up, Color.up, Color.up],
  (Tensor.basis _).repr ε4 b * (Tensor.basis _).repr ε4 b = 24
```

lemma leviCivita_basis_contract_three[\[source\]](#)

Contracting the first three standard-basis components of two contravariant Levi-Civita tensors gives $3! = 6$ times the Kronecker delta on the remaining components.

```
lemma leviCivita_basis_contract_three (a b : Fin 1 ⊗ Fin 3) :
  ∑ h : Fin 3 → Fin 1 ⊗ Fin 3,
    (Tensor.basis _).repr ε4 (Fin.snoc h a) *
    (Tensor.basis _).repr ε4 (Fin.snoc h b) =
    6 * (if a = b then 1 else 0)
```

lemma leviCivita_contract_three_basis_repr_apply

[\[source\]](#)

The standard-basis component formula for the tensor contraction $\varepsilon^{\{\mu\nu\rho\sigma\}} \varepsilon_{\{\mu\nu\rho\tau\}}$.

```
lemma leviCivita_contract_three_basis_repr_apply
  (b : ComponentIdx (S := realLorentzTensor 3) ![Color.up, Color.down]) :
  (Tensor.basis _).repr
    {ε4 | μ ν ρ σ ⊗ ε4 | τ(μ) τ(ν) τ(ρ) τ(σ)}T b =
  ∑ h : Fin 3 → Fin 1 ⊗ Fin 3,
    (Tensor.basis _).repr ε4 (Fin.snoc h (b 0)) *
    (Tensor.basis _).repr ({ε4 | τ(μ) τ(ν) τ(ρ) τ(σ)}T) (Fin.snoc h (b 1))
```

lemma leviCivita_contract_self_eq_sum

[\[source\]](#)

Fully contracting the tensor product of ε_4 and its fully lowered form is the sum of the products of their matching standard-basis components.

```
@[nolint checkType]
lemma leviCivita_contract_self_eq_sum :
  {ε4 | μ ν ρ σ ⊗ ε4 | τ(μ) τ(ν) τ(ρ) τ(σ)}T.toField =
  ∑ b : ComponentIdx (S := realLorentzTensor 3)
    ![Color.up, Color.up, Color.up, Color.up],
    (Tensor.basis _).repr ε4 b *
    (Tensor.basis _).repr ({ε4 | τ(μ) τ(ν) τ(ρ) τ(σ)}T) b
```

2.61 Physlib.Relativity.Tensors.MetricTensor

[Physlib/Relativity/Tensors/MetricTensor.lean](#) on GitHub.

lemma metricTensor_basis_repr

[\[source\]](#)

A component of the metric tensor is the corresponding component of the metric intertwiner in the tensor-product basis.

```
lemma metricTensor_basis_repr (c : C) (φ : ComponentIdx (S := S) ![c, c]) :
  (Tensor.basis _).repr (metricTensor (S := S) c) φ =
  (Module.Basis.tensorProduct (b c) (b c)).repr ((S.metric c) (1 : k)) (φ 0, φ 1)
```

2.62 Physlib.Relativity.Tensors.RealTensor.Contraction.CrossToEnd

[Physlib/Relativity/Tensors/RealTensor/Contraction/CrossToEnd.lean](#) on GitHub.

lemma crossToEnd_basis_repr_apply_eq_fin

[\[source\]](#)

For real Lorentz tensors, the component formula for `crossToEnd` collapses to one sum because the standard contravariant and covariant bases are dual under contraction.

```

lemma crossToEnd_basis_repr_apply_eq_fin {d nA nB : ℕ} {cA : Fin (nA + 1) → Color}
{cB : Fin (nB + 1) → Color} (i : Fin (nA + 1)) (j : Fin (nB + 1))
(hc : (realLorentzTensor d).τ (cA i) = cB j) (t : ℝT(d, cA)) (M : ℝT(d, cB))
(φ : ComponentIdx (S := realLorentzTensor d)
  (Fin.append (cA ∘ i.succAbove) (cB ∘ j.succAbove))) :
(Tensor.basis _).repr (crossToEnd i j hc t M) φ =
  ∑ x : Fin 1 ⊗ Fin d,
    (Tensor.basis cA).repr t (i.insertNth x (fun m => φ (Fin.castAdd nB m))) *
    (Tensor.basis cB).repr M (j.insertNth x (fun m => φ (Fin.natAdd nA m)))

```

2.63 Physlib.Relativity.Tensors.RealTensor.Metrics.Basic

[Physlib/Relativity/Tensors/RealTensor/Metrics/Basic.lean](#) on GitHub.

lemma metricTensor_repr_apply_eq_minkowskiMatrix

[\[source\]](#)

The component matrix of either real Lorentz metric tensor is the Minkowski matrix.

```

lemma metricTensor_repr_apply_eq_minkowskiMatrix {d : ℕ} (c : Color)
(φ : ComponentIdx (S := realLorentzTensor d) ![c, c]) :
(Tensor.basis _).repr (metricTensor (S := realLorentzTensor d) c) φ =
  minkowskiMatrix (φ 0) (φ 1)

```

lemma toDualMapAtIndex_basis_repr_apply

[\[source\]](#)

Raising or lowering one index of a real Lorentz tensor contracts that slot's components with the Minkowski matrix.

```

lemma toDualMapAtIndex_basis_repr_apply {d n : ℕ} {c : Fin (n + 1) → Color}
(i : Fin (n + 1)) (t : ℝT(d, c))
(φ : ComponentIdx (S := realLorentzTensor d)
  (Function.update c i ((realLorentzTensor d).τ (c i)))) :
(Tensor.basis _).repr (Tensor.toDualMapAtIndex (S := realLorentzTensor d) i t) φ =
  ∑ x : Fin 1 ⊗ Fin d,
    (Tensor.basis c).repr t (i.insertNth x (fun m => φ (i.succAbove m))) *
    minkowskiMatrix x (φ i)

```

lemma toDualMapAtIndex_basis_repr_apply_eq_mul

[\[source\]](#)

In the standard Lorentz basis, raising or lowering an index multiplies the component with that index fixed by the corresponding diagonal entry of the Minkowski metric.

```

lemma toDualMapAtIndex_basis_repr_apply_eq_mul {d n : ℕ} {c : Fin (n + 1) → Color}
(i : Fin (n + 1)) (t : ℝT(d, c))
(φ : ComponentIdx (S := realLorentzTensor d)
  (Function.update c i ((realLorentzTensor d).τ (c i)))) :
(Tensor.basis _).repr (Tensor.toDualMapAtIndex (S := realLorentzTensor d) i t) φ =
  (Tensor.basis c).repr t φ * minkowskiMatrix (φ i) (φ i)

```

2.64 Physlib.Relativity.Tensors.RealTensor.Units.Basic

[Physlib/Relativity/Tensors/RealTensor/Units/Basic.lean](#) on GitHub.

lemma unitTensor_repr_apply[\[source\]](#)

In the standard contravariant and covariant bases, either real Lorentz unit tensor has Kronecker-delta components.

```
lemma unitTensor_repr_apply {d : ℕ} (c : Color)
  (φ : ComponentIdx (S := realLorentzTensor d) ![(realLorentzTensor d).τ c, c]) :
  (Tensor.basis _).repr (unitTensor (S := realLorentzTensor d) c) φ =
    if φ 0 = φ 1 then 1 else 0
```

2.65 Physlib.Relativity.Tensors.Reindexing

[Physlib/Relativity/Tensors/Reindexing.lean](#) on GitHub.

lemma inv_eq_self_of_pointwise_eq[\[source\]](#)

```
lemma inv_eq_self_of_pointwise_eq {n : ℕ} {c c1 : Fin n → C} {σ : Fin n → Fin n}
  (h : IsReindexing c c1 σ) (hσ : ∀ x, σ x = x) (x : Fin n) :
  h.inv σ x = x
```

lemma inv_id_eq[\[source\]](#)

```
lemma inv_id_eq {n : ℕ} {c c1 : Fin n → C}
  (h : IsReindexing c c1 (id : Fin n → Fin n)) (x : Fin n) :
  h.inv (id : Fin n → Fin n) x = x
```

lemma inv_cast_eq[\[source\]](#)

```
lemma inv_cast_eq {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C} (e : m = n)
  (h : IsReindexing c c1 (Fin.cast e)) (x : Fin n) :
  h.inv (Fin.cast e) x = Fin.cast e.symm x
```

lemma inv_equiv_symm_eq[\[source\]](#)

```
lemma inv_equiv_symm_eq {n : ℕ} {c c1 : Fin n → C} (e : Equiv.Perm (Fin n))
  (h : IsReindexing c c1 ↑e.symm) (x : Fin n) :
  h.inv ↑e.symm x = e x
```

2.66 Physlib.Relativity.Tensors.UnitTensor

[Physlib/Relativity/Tensors/UnitTensor.lean](#) on GitHub.

lemma unitTensor_basis_repr[\[source\]](#)

A component of the unit tensor is the corresponding component of the unit intertwiner in the tensor-product basis.

```
lemma unitTensor_basis_repr (c : C) (φ : ComponentIdx (S := S) ![(S.τ c, c)]) :
  (Tensor.basis _).repr (unitTensor (S := S) c) φ =
    (Module.Basis.tensorProduct (b (S.τ c)) (b c)).repr ((S.unit c) (1 : k))
    (φ 0, φ 1)
```

2.67 Physlib.SpaceAndTime.ReferenceFrame

[Physlib/SpaceAndTime/ReferenceFrame.lean](#) on GitHub.

structure ReferenceFrame

[\[source\]](#)

A time-indexed choice of affine origin and displacement basis in d -dimensional space.

```
structure ReferenceFrame (d : ℕ) where
  origin : Time → Space d
  basis : Time → Module.Basis (Fin d) ℝ (EuclideanSpace ℝ (Fin d))
```

def fromReferencePoints

[\[source\]](#)

Build a reference frame from the trajectories of a collection of reference points.

At each time, the reference points must form an affine basis: none is redundant, and together they span the whole space. One reference point is chosen as the origin, and the displacements from it to the remaining points form the coordinate basis. The resulting frame need not be inertial or orthonormal.

```
def ReferenceFrame.fromReferencePoints
  (referencePoints : Finset (Time → Space d))
  (independence : ∀ t, AffineIndependent ℝ fun point : referencePoints => point.val t)
  (spans_space : ∀ t, affineSpan ℝ {point.val t | point : referencePoints} = T) :
  ReferenceFrame d
```

def IsMetricConserved

[\[source\]](#)

Whether the frame basis induces the same inner product on coordinates at every time.

```
def IsMetricConserved (frame : ReferenceFrame d) : Prop
```

def Orthonormal

[\[source\]](#)

Whether the frame's coordinate basis is orthonormal at every time.

```
def Orthonormal (frame : ReferenceFrame d) : Prop
```

lemma Orthonormal.isMetricConserved

[\[source\]](#)

An orthonormal frame conserves its coordinate metric.

```
lemma Orthonormal.isMetricConserved (h : frame.Orthonormal) : frame.IsMetricConserved
```

instance Fact frame.IsMetricConserved

[\[source\]](#)

```
instance [h : Fact frame.Orthonormal] : Fact frame.IsMetricConserved
```

structure IsInertial

[\[source\]](#)

Whether a reference frame is related to its initial grid by uniform translation alone.

```
structure IsInertial (frame : ReferenceFrame d) : Prop where
  origin_moves_uniformly :
    ∃ velocity, ∀ t1 t2, frame.origin t2 -v frame.origin t1 = (t2 - t1).val • velocity
  basis_conserved : ∀ t1 t2, frame.basis t1 = frame.basis t2
```

def velocity

[\[source\]](#)

The time-independent velocity of an inertial frame's coordinate origin.

```
def IsInertial.velocity (h : frame.IsInertial) : EuclideanSpace ℝ (Fin d)
```

lemma IsInertial.isMetricConserved

[\[source\]](#)

An inertial frame conserves its coordinate metric.

```
lemma IsInertial.isMetricConserved (h : frame.IsInertial) : frame.IsMetricConserved
```

instance Fact frame.IsMetricConserved

[\[source\]](#)

Inertiality provides the conserved metric needed for metric operations on frame vectors.

```
instance [h : Fact frame.IsInertial] : Fact frame.IsMetricConserved
```

structure Vector

[\[source\]](#)

The d real components used to express a vector quantity relative to frame.

```
structure Vector (frame : ReferenceFrame d) where
  components : Fin d → ℝ
```

def componentEquiv

[\[source\]](#)

Equivalence between frame vectors and coordinate components

```
def componentEquiv : frame.Vector ≈ (Fin d → ℝ)
```

instance AddCommGroup frame.Vector

[\[source\]](#)

```
instance : AddCommGroup frame.Vector
```

instance Module ℝ frame.Vector

[\[source\]](#)

```
instance : Module ℝ frame.Vector
```

def componentLinearEquiv

[\[source\]](#)

Linear equivalence between frame vectors and coordinate components.

```
def componentLinearEquiv : frame.Vector ≈ℝ (Fin d → ℝ)
```

def dispEquiv[\[source\]](#)

Equivalence between frame vectors and geometric displacements in space, defined by the frame's basis at t .

```
def dispEquiv (t : Time) : frame.Vector  $\approx_L[\mathbb{R}]$  EuclideanSpace  $\mathbb{R}$  (Fin d)
```

instance TopologicalSpace frame.Vector[\[source\]](#)

Use the same topology for frame vectors as components' product topology.

```
instance : TopologicalSpace frame.Vector
```

def componentContLinearEquiv[\[source\]](#)

Continuous linear equivalence between frame vectors and coordinate components.

```
def componentContLinearEquiv (frame : ReferenceFrame d) : frame.Vector  $\approx_L[\mathbb{R}]$  (Fin d  $\rightarrow \mathbb{R}$ )
```

instance FiniteDimensional $\mathbb{R}$ frame.Vector[\[source\]](#)

```
instance : FiniteDimensional  $\mathbb{R}$  frame.Vector
```

def contDispEquiv[\[source\]](#)

Continuous equivalence between frame vectors and geometric displacements in space, defined by the frame's basis at t .

```
def contDispEquiv (t : Time) : frame.Vector  $\approx_L[\mathbb{R}]$  EuclideanSpace  $\mathbb{R}$  (Fin d)
```

instance NormedAddCommGroup frame.Vector[\[source\]](#)

The physical norm on frame vectors, pulled back from geometric displacement space.

```
instance [_h : Fact frame.IsMetricConserved] : NormedAddCommGroup frame.Vector
```

instance InnerProductSpace $\mathbb{R}$ frame.Vector[\[source\]](#)

The physical inner product on frame vectors, pulled back from geometric displacement space.

```
instance [Fact frame.IsMetricConserved] : InnerProductSpace  $\mathbb{R}$  frame.Vector where
```

lemma norm_euclidean_if_orthonormal[\[source\]](#)

In an orthonormal frame, the norm is the square root of squared components.

```
lemma norm_euclidean_if_orthonormal [h : Fact frame.Orthonormal] :  
   $\forall v : \text{frame.Vector}, \|v\|^2 = \sum i, (v.\text{components } i)^2$ 
```

lemma inner_euclidean_if_orthonormal[\[source\]](#)

In an orthonormal frame, the inner product is the sum of component products.

```
lemma inner_euclidean_if_orthonormal [h : Fact frame.Orthonormal] :
  ∀ v w : frame.Vector, inner ℝ v w = ∑ i, v.components i * w.components i
```

2.68 Physlib.SpaceAndTime.Space.Derivatives.Curl

[Physlib/SpaceAndTime/Space/Derivatives/Curl.lean](#) on GitHub.

lemma curl_eq_sum_leviCivitaSymbol[\[source\]](#)

*The components of the curl as a contraction with the Levi-Civita symbol, $(\nabla \times f) \times$
 $i = \sum_j \epsilon_{ijk} \partial_j f_k$.*

```
lemma curl_eq_sum_leviCivitaSymbol (f : Space → EuclideanSpace ℝ (Fin 3))
  (x : Space) (i : Fin 3) :
  (∇ × f) x i = ∑ j, ∑ k, (leviCivitaSymbol ![i, j, k] : ℝ) * ∂[j] (fun y => f y k) x
```

2.69 Physlib.SpaceAndTime.Space.LengthUnit

[Physlib/SpaceAndTime/Space/LengthUnit.lean](#) on GitHub.

lemma div_mul_div[\[source\]](#)

The unit-ratio cocycle at $\mathbb{R}_{\geq 0}$ (the un-coerced form of `div_mul_div_coe`).

```
lemma div_mul_div (x y z : LengthUnit) : (x / y) * (y / z) = x / z
```

2.70 Physlib.SpaceAndTime.Space.Module

[Physlib/SpaceAndTime/Space/Module.lean](#) on GitHub.

lemma eval_hasTemperateGrowth[\[source\]](#)

```
@[fun_prop]
lemma eval_hasTemperateGrowth {d} (i : Fin d) :
  Function.HasTemperateGrowth (fun p : Space d => p i)
```

2.71 Physlib.SpaceAndTime.Time.Derivatives

[Physlib/SpaceAndTime/Time/Derivatives.lean](#) on GitHub.

lemma toRealCLE_symm_one[\[source\]](#)

The canonical equivalence `toRealCLE.symm` : $\mathbb{R} \simeq L[\mathbb{R}]$ Time sends $1 : \mathbb{R}$ to $1 : \text{Time}$.

```
lemma toRealCLE_symm_one : toRealCLE.symm (1 : ℝ) = (1 : Time)
```

lemma hasDerivAt_comp_toRealCLE_symm[\[source\]](#)

Bridge from the time derivative to `HasDerivAt`: if $w : \text{Time} \rightarrow M$ is differentiable at `toRealCLE.symm` τ , then the curve $\tau \mapsto w$ (`toRealCLE.symm` τ) on $\mathbb{R}$ has derivative $\partial_\tau w$ (`toRealCLE.symm` τ) at τ .

```

lemma hasDerivAt_comp_toRealCLE_symm [NormedAddCommGroup M] [NormedSpace ℝ M]
  (w : Time → M) (τ : ℝ) (hw : DifferentiableAt ℝ w (toRealCLE.symm τ)) :
  HasDerivAt (fun τ : ℝ => w (toRealCLE.symm τ)) (∂t w (toRealCLE.symm τ)) τ

```

lemma deriv_comp_toRealCLE_of_hasDerivAt

[\[source\]](#)

The converse of the bridge hasDerivAt_comp_toRealCLE_symm: if the curve $\gamma : \mathbb{R} \rightarrow M$ has derivative v at toRealCLE t , then the curve $t \mapsto \gamma (\text{toRealCLE } t)$ on Time has time derivative v at t .

```

lemma deriv_comp_toRealCLE_of_hasDerivAt [NormedAddCommGroup M] [NormedSpace ℝ M]
  (γ : ℝ → M) (t : Time) (v : M) (h : HasDerivAt γ v (toRealCLE t)) :
  ∂t (fun s => γ (toRealCLE s)) t = v

```

lemma deriv_comp_neg

[\[source\]](#)

The chain rule for the time derivative under the reversal of time: the derivative of $t \mapsto f (-t)$ at t is minus the derivative of f at $-t$.

```

lemma deriv_comp_neg {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
  (f : Time → M) (t : Time) (hf : DifferentiableAt ℝ f (-t)) :
  ∂t (fun s => f (-s)) t = -∂t f (-t)

```

lemma deriv_deriv_comp_neg

[\[source\]](#)

The second derivative is unchanged by the reversal of time: for a smooth curve f , the second derivative of $t \mapsto f (-t)$ at t is the second derivative of f at $-t$, the two changes of sign cancelling.

```

lemma deriv_deriv_comp_neg {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
  (f : Time → M) (hf : ContDiff ℝ ∞ f) (t : Time) :
  ∂t (∂t (fun s => f (-s))) t = ∂t (∂t f) (-t)

```

2.72 Physlib.SpaceAndTime.Time.TimeUnit

[Physlib/SpaceAndTime/Time/TimeUnit.lean](#) on GitHub.

lemma div_mul_div

[\[source\]](#)

The unit-ratio cocycle at $\mathbb{R}_{\geq 0}$ (the un-coerced form of `div_mul_div_coe`).

```

lemma div_mul_div (x y z : TimeUnit) : (x / y) * (y / z) = x / z

```

2.73 Physlib.StatisticalMechanics.CanonicalEnsemble.Lemmas

[Physlib/StatisticalMechanics/CanonicalEnsemble/Lemmas.lean](#) on GitHub.

def meanEnergyT

[\[source\]](#)

The mean energy as a function of the real-valued temperature t .

```

noncomputable def meanEnergyT (C : CanonicalEnsemble ι) (t : ℝ) : ℝ

```

2.74 Physlib.StatisticalMechanics.CanonicalEnsemble.TwoState

[Physlib/StatisticalMechanics/CanonicalEnsemble/TwoState.lean](#) on GitHub.

lemma twoState_helmholtzFreeEnergy_eq_T_neq_zero

[source]

An instance of twoState_helmholtzFreeEnergy_eq assuming $T \neq 0$

```
lemma twoState_helmholtzFreeEnergy_eq_T_neq_zero (E₀ E₁ : ℝ) (T : Temperature) (Th : T ≠ 0) :
  (twoState E₀ E₁).helmholtzFreeEnergy T =
    (E₀ + E₁) / 2 - Real.log (2 * Real.cosh (β T * (E₁ - E₀) / 2)) / β T
```

2.75 Physlib.Thermodynamics.Temperature.TemperatureUnits

[Physlib/Thermodynamics/Temperature/TemperatureUnits.lean](#) on GitHub.

lemma div_mul_div

[source]

The unit-ratio cocycle at $\mathbb{R}_{\geq 0}$ (the un-coerced form of div_mul_div_coe).

```
lemma div_mul_div (x y z : TemperatureUnit) : (x / y) * (y / z) = x / z
```

2.76 Physlib.Units.Basic

[Physlib/Units/Basic.lean](#) on GitHub.

structure LTMCTUnitChoices

[source]

The choice of units.

```
@[ext]
structure LTMCTUnitChoices where
  length : LengthUnit
  time : TimeUnit
  mass : MassUnit
  charge : ChargeUnit
  temperature : TemperatureUnit
```

def dimScale

[source]

Given two choices of units $u1$ and $u2$ and a dimension d , the element of $\mathbb{R}_{\geq 0}$ corresponding to the scaling (by definition) of a quantity of dimension d when changing from units $u1$ to $u2$.

```
noncomputable def dimScale (u1 u2 : LTMCTUnitChoices) : Dimension LTMCTDimensionBase →* ℝ₊₀
  where
```

lemma dimScale_apply

[source]

```
lemma dimScale_apply (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
  dimScale u1 u2 d =
    (u1.length / u2.length) ^ (d.length : ℝ) *
    (u1.time / u2.time) ^ (d.time : ℝ) *
    (u1.mass / u2.mass) ^ (d.mass : ℝ) *
    (u1.charge / u2.charge) ^ (d.charge : ℝ) *
    (u1.temperature / u2.temperature) ^ (d.temperature : ℝ)
```

lemma dimScale_self[\[source\]](#)

```
@[simp]
lemma dimScale_self (u : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
  dimScale u u d = 1
```

lemma dimScale_one[\[source\]](#)

```
@[simp]
lemma dimScale_one (u1 u2 : LTMCTUnitChoices) :
  dimScale u1 u2 1 = 1
```

lemma dimScale_transitive[\[source\]](#)

```
lemma dimScale_transitive (u1 u2 u3 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
  dimScale u1 u2 d * dimScale u2 u3 d = dimScale u1 u3 d
```

lemma dimScale_mul_symm[\[source\]](#)

```
@[simp]
lemma dimScale_mul_symm (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
  dimScale u1 u2 d * dimScale u2 u1 d = 1
```

lemma dimScale_coe_mul_symm[\[source\]](#)

```
@[simp]
lemma dimScale_coe_mul_symm (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
  (toReal (dimScale u1 u2 d)) * (toReal (dimScale u2 u1 d)) = 1
```

lemma dimScale_ne_zero[\[source\]](#)

```
@[simp]
lemma dimScale_ne_zero (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
  dimScale u1 u2 d ≠ 0
```

lemma dimScale_symm[\[source\]](#)

```
lemma dimScale_symm (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
  dimScale u1 u2 d = (dimScale u2 u1 d)-1
```

lemma dimScale_of_inv_eq_swap[\[source\]](#)

```
lemma dimScale_of_inv_eq_swap (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
  dimScale u1 u2 d-1 = dimScale u2 u1 d
```

lemma smul_dimScale_injective[\[source\]](#)

```
@[simp]
lemma smul_dimScale_injective {M : Type} [MulAction ℝ≥0 M] (u1 u2 : LTMCTUnitChoices)
  (d : Dimension LTMCTDimensionBase) (m1 m2 : M) :
  (u1.dimScale u2 d) • m1 = (u1.dimScale u2 d) • m2 ↔ m1 = m2
```

lemma dimScale_pos[\[source\]](#)

```
@[simp]
lemma dimScale_pos (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
  0 < (dimScale u1 u2 d)
```

def SI[\[source\]](#)

The choice of units corresponding to SI units, that is - meters, - seconds, - kilograms, - coulombs, - kelvin.

```
noncomputable def SI : LTMCTUnitChoices where
```

lemma SI_length[\[source\]](#)

```
@[simp]
lemma SI_length : SI.length = LengthUnit.meters
```

lemma SI_time[\[source\]](#)

```
@[simp]
lemma SI_time : SI.time = TimeUnit.seconds
```

lemma SI_mass[\[source\]](#)

```
@[simp]
lemma SI_mass : SI.mass = MassUnit.kilograms
```

lemma SI_charge[\[source\]](#)

```
@[simp]
lemma SI_charge : SI.charge = ChargeUnit.coulombs
```

lemma SI_temperature[\[source\]](#)

```
@[simp]
lemma SI_temperature : SI.temperature = TemperatureUnit.kelvin
```

def SIPrimed[\[source\]](#)

A LTMCTUnitChoices which is related to SI by a prime scaling of each of the underlying units. This is useful in proving that a result is not dimensionally correct.

```
noncomputable def SIPrimed : LTMCTUnitChoices where
```

lemma dimScale_SI_SIPrimed[\[source\]](#)

```
@[simp]
lemma dimScale_SI_SIPrimed (d : Dimension LTMCTDimensionBase) :
  dimScale SI SIPrimed d =
    (2-1 : ℝ≥0) ^ (d.length : ℝ) *
    (3-1 : ℝ≥0) ^ (d.time : ℝ) *
    (5-1 : ℝ≥0) ^ (d.mass : ℝ) *
```

```
(7-1 : ℝ≥0) ^ (d.charge : ℝ) *
(11-1 : ℝ≥0) ^ (d.temperature : ℝ)
```

lemma dimScale_SIPrimed_SI[\[source\]](#)

```
@[simp]
lemma dimScale_SIPrimed_SI (d : Dimension LTMCTDimensionBase) :
  dimScale SIPrimed SI d =
    (2 : ℝ≥0) ^ (d.length : ℝ) *
    (3 : ℝ≥0) ^ (d.time : ℝ) *
    (5 : ℝ≥0) ^ (d.mass : ℝ) *
    (7 : ℝ≥0) ^ (d.charge : ℝ) *
    (11 : ℝ≥0) ^ (d.temperature : ℝ)
```

instance CoeFun (Dimensionful M) (fun _ => LTMCTUnitChoices → M)[\[source\]](#)

```
instance {M : Type} [CarriesDimension M] :
  CoeFun (Dimensionful M) (fun _ => LTMCTUnitChoices → M) where
```

2.77 Physlib.Units.Dimension

[Physlib/Units/Dimension.lean](#) on GitHub.

class DimensionBasis[\[source\]](#)

A choice of exponent-tuple representation for a basis B. Native addition on Exponents is used for dimension multiplication, while exponentEquiv provides the basis-generic API.

```
class DimensionBasis (B : Type) where
  Exponents : Type
  [addCommGroup : AddCommGroup Exponents]
  exponentEquiv : Exponents ≈+ (B → Dimension.Exponent)
```

def exponent[\[source\]](#)

The exponent of a dimension at a base dimension.

```
def exponent (d : Dimension B) : B → Exponent
```

def ofFunction[\[source\]](#)

Construct a dimension from an exponent function.

```
def ofFunction (f : B → Exponent) : Dimension B
```

lemma ofFunction_exponent[\[source\]](#)

```
@[simp]
lemma ofFunction_exponent (f : B → Exponent) (b : B) : (ofFunction f).exponent b = f b
```

instance Pow (Dimension B) Exponent[\[source\]](#)

*Raising a dimension to an **Exponent** power. Unlike the $\mathbb{Q}$ -valued power, this preserves reducible arithmetic for concrete fractional exponents.*

```
@[default_instance 10000]
instance : Pow (Dimension B) Exponent where
```

lemma epow_exponent[\[source\]](#)

```
@[simp]
lemma epow_exponent (d : Dimension B) (c : Exponent) (b : B) :
  (d ^ c).exponent b = d.exponent b * c
```

2.78 Physlib.Units.Exponent

[Physlib/Units/Exponent.lean](#) on GitHub.

structure Exponent[\[source\]](#)

A rational dimension exponent with reducible arithmetic.

```
structure Exponent where
  toRat :  $\mathbb{Q}$ 
```

instance Repr Exponent[\[source\]](#)

```
instance : Repr Exponent where
```

def equivRat[\[source\]](#)

*The equivalence between **Exponent** and the rational numbers.*

```
def equivRat : Exponent  $\approx$   $\mathbb{Q}$ 
```

def ofRat[\[source\]](#)

Regard a rational number as a dimension exponent.

```
def ofRat (q :  $\mathbb{Q}$ ) : Exponent
```

lemma ofRat_toRat[\[source\]](#)

```
@[simp]
lemma ofRat_toRat (q :  $\mathbb{Q}$ ) : (ofRat q).toRat = q
```

def gcdAux[\[source\]](#)

Fuel-bounded Euclidean algorithm used to make exponent normalization reducible.

```
def gcdAux : Nat → Nat → Nat → Nat
| 0, _, n => n
| fuel + 1, m, n => if m = 0 then n else gcdAux fuel (n % m) m
```

lemma gcdAux_eq_nat_gcd[\[source\]](#)

```
private lemma gcdAux_eq_nat_gcd (fuel m n : Nat) (m_lt_fuel : m < fuel) :
  gcdAux fuel m n = Nat.gcd m n
```

def gcd[\[source\]](#)

Reducible greatest common divisor used when normalizing an exponent.

```
def gcd (m n : Nat) : Nat
```

lemma gcd_eq_nat_gcd[\[source\]](#)

The reducible exponent GCD agrees with Nat.gcd.

```
lemma gcd_eq_nat_gcd (m n : Nat) : gcd m n = Nat.gcd m n
```

def normalize[\[source\]](#)

Construct an exponent by normalizing a numerator and a nonzero denominator.

```
def normalize (num : Int) (den : Nat) (den_ne_zero : den ≠ 0) : Exponent
```

lemma normalize_toRat[\[source\]](#)

```
lemma normalize_toRat (num : Int) (den : Nat) (den_ne_zero : den ≠ 0) :
  (normalize num den den_ne_zero).toRat = Rat.normalize num den den_ne_zero
```

def add[\[source\]](#)

Reducible addition of dimension exponents.

```
def add (a b : Exponent) : Exponent
```

instance Add Exponent[\[source\]](#)

```
instance : Add Exponent
```

lemma add_equiv[\[source\]](#)

```
lemma add_equiv (a b : Exponent) : equivRat (add a b) = equivRat a + equivRat b
```

def sub[\[source\]](#)

Reducible subtraction of dimension exponents.

```
def sub (a b : Exponent) : Exponent
```

instance Sub Exponent[\[source\]](#)

```
instance : Sub Exponent
```

lemma sub_equiv[\[source\]](#)

```
lemma sub_equiv (a b : Exponent) : equivRat (sub a b) = equivRat a - equivRat b
```

def mul[\[source\]](#)

Reducible multiplication of dimension exponents.

```
def mul (a b : Exponent) : Exponent
```

instance Mul Exponent[\[source\]](#)

```
instance : Mul Exponent
```

lemma mul_equiv[\[source\]](#)

```
lemma mul_equiv (a b : Exponent) : equivRat (mul a b) = equivRat a * equivRat b
```

def inv[\[source\]](#)

Reducible inversion of a dimension exponent, with $\theta^{-1} = \theta$.

```
def inv (a : Exponent) : Exponent
```

instance Inv Exponent[\[source\]](#)

```
instance : Inv Exponent
```

lemma inv_equiv[\[source\]](#)

```
lemma inv_equiv (a : Exponent) : equivRat (inv a) = (equivRat a)-1
```

def div[\[source\]](#)

Reducible division of dimension exponents.

```
def div (a b : Exponent) : Exponent
```

instance Div Exponent[\[source\]](#)

```
instance : Div Exponent
```

lemma div_equiv[\[source\]](#)

```
lemma div_equiv (a b : Exponent) : equivRat (div a b) = equivRat a / equivRat b
```

instance instField[\[source\]](#)

```
instance instField : Field Exponent
```

def ringEquivRat[\[source\]](#)*The ring equivalence between dimension exponents and rational numbers.*

```
def ringEquivRat : Exponent  $\approx$ +*  $\mathbb{Q}$  where
```

instance Coe Exponent $\mathbb{Q}$ [\[source\]](#)*Regard a dimension exponent as a rational number.*

```
instance : Coe Exponent  $\mathbb{Q}$ 
```

lemma coe_inj[\[source\]](#)

```
@[simp, norm_cast]
```

```
lemma coe_inj {a b : Exponent} : (a :  $\mathbb{Q}$ ) = b  $\leftrightarrow$  a = b
```

lemma coe_zero[\[source\]](#)

```
@[simp, norm_cast]
```

```
lemma coe_zero : ((0 : Exponent) :  $\mathbb{Q}$ ) = 0
```

lemma coe_one[\[source\]](#)

```
@[simp, norm_cast]
```

```
lemma coe_one : ((1 : Exponent) :  $\mathbb{Q}$ ) = 1
```

lemma coe_ofNat[\[source\]](#)

```
@[simp, norm_cast]
```

```
lemma coe_ofNat (n :  $\mathbb{N}$ ) [n.AtLeastTwo] : ((ofNat(n) : Exponent) :  $\mathbb{Q}$ ) = ofNat(n)
```

lemma coe_add[\[source\]](#)

```
@[simp, norm_cast]
```

```
lemma coe_add (a b : Exponent) : ((a + b : Exponent) :  $\mathbb{Q}$ ) = a + b
```

lemma coe_sub[\[source\]](#)

```
@[simp, norm_cast]
```

```
lemma coe_sub (a b : Exponent) : ((a - b : Exponent) :  $\mathbb{Q}$ ) = a - b
```

lemma coe_neg[\[source\]](#)

```
@[simp, norm_cast]
```

```
lemma coe_neg (a : Exponent) : ((-a : Exponent) :  $\mathbb{Q}$ ) = -a
```

lemma coe_mul[\[source\]](#)

```
@[simp, norm_cast]
```

```
lemma coe_mul (a b : Exponent) : ((a * b : Exponent) :  $\mathbb{Q}$ ) = a * b
```

lemma coe_inv[\[source\]](#)

@[simp, norm_cast]

lemma coe_inv (a : Exponent) : ((a⁻¹ : Exponent) : ℚ) = (a : ℚ)⁻¹**lemma coe_div**[\[source\]](#)

@[simp, norm_cast]

lemma coe_div (a b : Exponent) : ((a / b : Exponent) : ℚ) = (a : ℚ) / b**instance LinearOrder Exponent**[\[source\]](#)**instance** : LinearOrder Exponent**lemma coe_le_coe**[\[source\]](#)

@[simp, norm_cast]

lemma coe_le_coe {a b : Exponent} : (a : ℚ) ≤ b ↔ a ≤ b**lemma coe_lt_coe**[\[source\]](#)

@[simp, norm_cast]

lemma coe_lt_coe {a b : Exponent} : (a : ℚ) < b ↔ a < b**instance IsStrictOrderedRing Exponent**[\[source\]](#)**instance** : IsStrictOrderedRing Exponent**instance CharZero Exponent**[\[source\]](#)**instance** : CharZero Exponent **where****lemma add_eq_instField_add**[\[source\]](#)**lemma** add_eq_instField_add : add = instField.add**lemma sub_eq_instField_sub**[\[source\]](#)**lemma** sub_eq_instField_sub : sub = instField.sub**lemma inv_eq_instField_inv**[\[source\]](#)**lemma** inv_eq_instField_inv : inv = instField.inv**lemma mul_eq_instField_mul**[\[source\]](#)**lemma** mul_eq_instField_mul : mul = instField.mul

lemma div_eq_instField_div[\[source\]](#)

```
lemma div_eq_instField_div : div = instField.div
```

lemma tuple_arithmetic_defeq[\[source\]](#)

```
lemma tuple_arithmetic_defeq :
  let Length : Exponent × Exponent
```

lemma rational_arithmetic_defeq[\[source\]](#)

```
lemma rational_arithmetic_defeq :
  ((2 / 3 + 5 / 7) * (11 / 13 - 1 / 2) : Exponent) = 87 / 182
```

lemma inverse_arithmetic_defeq[\[source\]](#)

```
lemma inverse_arithmetic_defeq : ((-3 / 4 : Exponent)-1 + 5 / 6) = -1 / 2
```

2.79 Physlib.Units.ISQDimensionBase

[Physlib/Units/ISQDimensionBase.lean](#) on GitHub.

instance Fintype ISQDimensionBase[\[source\]](#)

```
instance : Fintype ISQDimensionBase where
```

instance DimensionBasis ISQDimensionBase[\[source\]](#)

```
instance : DimensionBasis ISQDimensionBase
```

2.80 Physlib.Units.LTMCTDimensionBase

[Physlib/Units/LTMCTDimensionBase.lean](#) on GitHub.

instance Fintype LTMCTDimensionBase[\[source\]](#)

```
instance : Fintype LTMCTDimensionBase where
```

abbrev Exponents[\[source\]](#)

The fixed five-component exponent tuple for PhysLib's default dimension basis.

```
abbrev Exponents
```

instance DimensionBasis LTMCTDimensionBase[\[source\]](#)

```
instance : DimensionBasis LTMCTDimensionBase where
```

lemma exponent_length[\[source\]](#)

```
@[simp]
lemma exponent_length (d : Dimension LTMCTDimensionBase) : d.exponent .length = d.length
```

lemma exponent_time[\[source\]](#)

```
@[simp]
lemma exponent_time (d : Dimension LTMCTDimensionBase) : d.exponent .time = d.time
```

lemma exponent_mass[\[source\]](#)

```
@[simp]
lemma exponent_mass (d : Dimension LTMCTDimensionBase) : d.exponent .mass = d.mass
```

lemma exponent_charge[\[source\]](#)

```
@[simp]
lemma exponent_charge (d : Dimension LTMCTDimensionBase) : d.exponent .charge = d.charge
```

lemma exponent_temperature[\[source\]](#)

```
@[simp]
lemma exponent_temperature (d : Dimension LTMCTDimensionBase) :
  d.exponent .temperature = d.temperature
```

lemma component_npow[\[source\]](#)

```
private lemma component_npow (component : Dimension LTMCTDimensionBase → Exponent)
  (b : LTMCTDimensionBase) (h : ∀ d, d.exponent b = component d)
  (d : Dimension LTMCTDimensionBase) (n : ℕ) :
  component (d ^ n) = n • component d
```

2.81 Physlib.Units.ParametricDimensionExamples

[Physlib/Units/ParametricDimensionExamples.lean](#) on GitHub.

instance DimensionBasis Info[\[source\]](#)

```
instance : DimensionBasis Info
```

2.82 Physlib.Units.ParametricUnits

[Physlib/Units/ParametricUnits.lean](#) on GitHub.

def toScale[\[source\]](#)

Read a five-field LTMCTUnitChoices as a UnitScale over LTMCTDimensionBase.

```
noncomputable def toScale (u : LTMCTUnitChoices) : UnitScale LTMCTDimensionBase where
```

2.83 Physlib.Units.SIUnitChoices

[Physlib/Units/SIUnitChoices.lean](#) on GitHub.

structure CurrentUnit

[\[source\]](#)

A unit of electric current — a choice of positive-real magnitude. The SI coherent choice is the ampere.

```
structure CurrentUnit where
```

```
  val : ℝ
  property : 0 < val
```

lemma CurrentUnit.val_ne_zero

[\[source\]](#)

```
@[simp]
```

```
lemma val_ne_zero (x : CurrentUnit) : x.val ≠ 0
```

lemma CurrentUnit.val_pos

[\[source\]](#)

```
lemma val_pos (x : CurrentUnit) : 0 < x.val
```

instance Inhabited CurrentUnit

[\[source\]](#)

```
instance : Inhabited CurrentUnit where
```

instance HDiv CurrentUnit CurrentUnit ℝ≥0

[\[source\]](#)

```
noncomputable instance : HDiv CurrentUnit CurrentUnit ℝ≥0 where
```

lemma CurrentUnit.div_eq_val

[\[source\]](#)

```
lemma div_eq_val (x y : CurrentUnit) :
  x / y = (⟨x.val / y.val, div_nonneg x.val_pos.le y.val_pos.le⟩ : ℝ≥0)
```

def CurrentUnit.scale

[\[source\]](#)

Scale a unit of electric current by a positive real factor.

```
def scale (r : ℝ) (x : CurrentUnit) (hr : 0 < r := by norm_num) : CurrentUnit
```

lemma CurrentUnit.scale_div_self

[\[source\]](#)

```
@[simp]
```

```
lemma scale_div_self (x : CurrentUnit) (r : ℝ) (hr : 0 < r) :
  scale r x hr / x = (⟨r, le_of_lt hr⟩ : ℝ≥0)
```

lemma CurrentUnit.self_div_scale

[\[source\]](#)

```
@[simp]
```

```
lemma self_div_scale (x : CurrentUnit) (r : ℝ) (hr : 0 < r) :
  x / scale r x hr =
    (⟨1 / r, _root_.div_nonneg (by simp) (le_of_lt hr)⟩ : ℝ≥0)
```

lemma CurrentUnit.scale_one[\[source\]](#)

```
@[simp]
lemma scale_one (x : CurrentUnit) : scale 1 x = x
```

lemma CurrentUnit.scale_div_scale[\[source\]](#)

```
@[simp]
lemma scale_div_scale
  (x1 x2 : CurrentUnit) {r1 r2 : ℝ} (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 x1 hr1 / scale r2 x2 hr2 =
    ((r1, le_of_lt hr1) / (r2, le_of_lt hr2)) * (x1 / x2)
```

lemma CurrentUnit.scale_scale[\[source\]](#)

```
@[simp]
lemma scale_scale (x : CurrentUnit) (r1 r2 : ℝ) (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 (scale r2 x hr2) hr1 =
    scale (r1 * r2) x (mul_pos hr1 hr2)
```

def amperes[\[source\]](#)

The SI coherent unit of electric current, the ampere.

```
def amperes : CurrentUnit
```

def milliamperes[\[source\]](#)

One milliampere, equal to 10^{-3} amperes.

```
noncomputable def milliamperes : CurrentUnit
```

structure AmountUnit[\[source\]](#)

A unit of amount of substance — a choice of positive-real magnitude. The SI coherent choice is the mole.

```
structure AmountUnit where
  val : ℝ
  property : 0 < val
```

lemma AmountUnit.val_ne_zero[\[source\]](#)

```
@[simp]
lemma val_ne_zero (x : AmountUnit) : x.val ≠ 0
```

lemma AmountUnit.val_pos[\[source\]](#)

```
lemma val_pos (x : AmountUnit) : 0 < x.val
```

instance Inhabited AmountUnit[\[source\]](#)

```
instance : Inhabited AmountUnit where
```

instance HDiv AmountUnit AmountUnit $\mathbb{R}_{\geq 0}$ [\[source\]](#)

```
noncomputable instance : HDiv AmountUnit AmountUnit  $\mathbb{R}_{\geq 0}$  where
```

lemma AmountUnit.div_eq_val[\[source\]](#)

```
lemma div_eq_val (x y : AmountUnit) :
  x / y = ((x.val / y.val, div_nonneg x.val_pos.le y.val_pos.le) :  $\mathbb{R}_{\geq 0}$ )
```

def AmountUnit.scale[\[source\]](#)

Scale a unit of amount of substance by a positive real factor.

```
def scale (r :  $\mathbb{R}$ ) (x : AmountUnit) (hr : 0 < r := by norm_num) : AmountUnit
```

lemma AmountUnit.scale_div_self[\[source\]](#)

```
@[simp]
lemma scale_div_self (x : AmountUnit) (r :  $\mathbb{R}$ ) (hr : 0 < r) :
  scale r x hr / x = ((r, le_of_lt hr) :  $\mathbb{R}_{\geq 0}$ )
```

lemma AmountUnit.self_div_scale[\[source\]](#)

```
@[simp]
lemma self_div_scale (x : AmountUnit) (r :  $\mathbb{R}$ ) (hr : 0 < r) :
  x / scale r x hr =
    ((1 / r, _root_.div_nonneg (by simp) (le_of_lt hr)) :  $\mathbb{R}_{\geq 0}$ )
```

lemma AmountUnit.scale_one[\[source\]](#)

```
@[simp]
lemma scale_one (x : AmountUnit) : scale 1 x = x
```

lemma AmountUnit.scale_div_scale[\[source\]](#)

```
@[simp]
lemma scale_div_scale
  (x1 x2 : AmountUnit) {r1 r2 :  $\mathbb{R}$ } (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 x1 hr1 / scale r2 x2 hr2 =
    ((r1, le_of_lt hr1) / (r2, le_of_lt hr2)) * (x1 / x2)
```

lemma AmountUnit.scale_scale[\[source\]](#)

```
@[simp]
lemma scale_scale (x : AmountUnit) (r1 r2 :  $\mathbb{R}$ ) (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 (scale r2 x hr2) hr1 =
    scale (r1 * r2) x (mul_pos hr1 hr2)
```

def moles [\[source\]](#)

The SI coherent unit of amount of substance, the mole.

```
def moles : AmountUnit
```

structure LuminousIntensityUnit [\[source\]](#)

A unit of luminous intensity — a choice of positive-real magnitude. The SI coherent choice is the candela.

```
structure LuminousIntensityUnit where
  val : ℝ
  property : 0 < val
```

lemma LuminousIntensityUnit.val_ne_zero [\[source\]](#)

```
@[simp]
lemma val_ne_zero (x : LuminousIntensityUnit) : x.val ≠ 0
```

lemma LuminousIntensityUnit.val_pos [\[source\]](#)

```
lemma val_pos (x : LuminousIntensityUnit) : 0 < x.val
```

instance Inhabited LuminousIntensityUnit [\[source\]](#)

```
instance : Inhabited LuminousIntensityUnit where
```

instance HDiv LuminousIntensityUnit LuminousIntensityUnit $\mathbb{R}_{\geq 0}$ [\[source\]](#)

```
noncomputable instance : HDiv LuminousIntensityUnit LuminousIntensityUnit  $\mathbb{R}_{\geq 0}$  where
```

lemma LuminousIntensityUnit.div_eq_val [\[source\]](#)

```
lemma div_eq_val (x y : LuminousIntensityUnit) :
  x / y = ({x.val / y.val, div_nonneg x.val_pos.le y.val_pos.le} :  $\mathbb{R}_{\geq 0}$ )
```

def LuminousIntensityUnit.scale [\[source\]](#)

Scale a unit of luminous intensity by a positive real factor.

```
def scale
  (r : ℝ) (x : LuminousIntensityUnit) (hr : 0 < r := by norm_num) :
  LuminousIntensityUnit
```

lemma LuminousIntensityUnit.scale_div_self [\[source\]](#)

```
@[simp]
lemma scale_div_self (x : LuminousIntensityUnit) (r : ℝ) (hr : 0 < r) :
  scale r x hr / x = ({r, le_of_lt hr} :  $\mathbb{R}_{\geq 0}$ )
```

lemma LuminousIntensityUnit.self_div_scale[\[source\]](#)

```
@[simp]
lemma self_div_scale (x : LuminousIntensityUnit) (r : ℝ) (hr : 0 < r) :
  x / scale r x hr =
    ((1 / r, _root_.div_nonneg (by simp) (le_of_lt hr)) : ℝ≥0)
```

lemma LuminousIntensityUnit.scale_one[\[source\]](#)

```
@[simp]
lemma scale_one (x : LuminousIntensityUnit) : scale 1 x = x
```

lemma LuminousIntensityUnit.scale_div_scale[\[source\]](#)

```
@[simp]
lemma scale_div_scale
  (x1 x2 : LuminousIntensityUnit)
  {r1 r2 : ℝ}
  (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 x1 hr1 / scale r2 x2 hr2 =
    ((r1, le_of_lt hr1) / (r2, le_of_lt hr2)) * (x1 / x2)
```

lemma LuminousIntensityUnit.scale_scale[\[source\]](#)

```
@[simp]
lemma scale_scale
  (x : LuminousIntensityUnit)
  (r1 r2 : ℝ)
  (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 (scale r2 x hr2) hr1 =
    scale (r1 * r2) x (mul_pos hr1 hr2)
```

def candelas[\[source\]](#)

The SI coherent unit of luminous intensity, the candela.

```
def candelas : LuminousIntensityUnit
```

instance UnitMagnitudeCatalog ISQDimensionBase[\[source\]](#)

```
noncomputable instance : UnitMagnitudeCatalog ISQDimensionBase where
```

lemma prod_univ_ISQDimensionBase[\[source\]](#)

The product over the seven ISQ base quantities, in canonical enumeration order — the ISQDimensionBase companion to prod_univ_LTMCTDimensionBase, a reusable fact about the Fintype enumeration for folding Finset.prod over the ISQ basis.

```
lemma prod_univ_ISQDimensionBase {M : Type} [CommMonoid M] (f : ISQDimensionBase → M) :
  [] b, f b = f .length * f .mass * f .time * f .current * f .temperature
  * f .amount * f .luminousIntensity
```

abbrev SIUnitChoices[\[source\]](#)

*A ****typed SI unit choice****: a typed unit at every ISQ base quantity. This is the seven-slot, current-based sibling of the five-slot, charge-based LTMCTUnitChoices, produced by the same basis-generic UnitSystem / UnitMagnitudeCatalog machinery.*

```
abbrev SIUnitChoices
```

def SI[\[source\]](#)

The coherent SI unit choice: metre, kilogram, second, ampere, kelvin, mole, candela.

```
noncomputable def SI : SIUnitChoices
```

def dimScale[\[source\]](#)

*The dimension-scaling homomorphism over the ISQ basis, obtained ***for free*** from the generic UnitScale.dimScale fold — no per-basis hand-rolling.*

```
noncomputable def dimScale (u1 u2 : SIUnitChoices) : Dimension ISQDimensionBase →* ℝ≥0
```

lemma dimScale_self[\[source\]](#)

A quantity of ISQ dimension does not rescale between a unit choice and itself.

```
lemma dimScale_self (u : SIUnitChoices) (d : Dimension ISQDimensionBase) :  
  dimScale u u d = 1
```

2.84 Physlib.Units.UnitDependent

[Physlib/Units/UnitDependent.lean](#) on GitHub.

instance UnitDependent LTMCTUnitChoices[\[source\]](#)

```
noncomputable instance : UnitDependent LTMCTUnitChoices where
```

lemma scaleUnit_apply_fst[\[source\]](#)

```
@[simp]  
lemma LTMCTUnitChoices.scaleUnit_apply_fst (u1 u2 : LTMCTUnitChoices) :  
  (scaleUnit u1 u2 u1) = u2
```

lemma dimScale_scaleUnit[\[source\]](#)

```
@[simp]  
lemma LTMCTUnitChoices.dimScale_scaleUnit {u1 u2 u : LTMCTUnitChoices}  
  (d : Dimension LTMCTDimensionBase) :  
  u.dimScale (scaleUnit u1 u2 u) d = u1.dimScale u2 d
```

2.85 Physlib.Units.UnitSystem

[Physlib/Units/UnitSystem.lean](#) on GitHub.

class UnitMagnitudeCatalog[\[source\]](#)

A *menu of typed units* for a basis B : for each base dimension $b : B$, a *type* $\text{Unit } b$ of admissible units (a dimension may offer several — metre, kilometre, ...) and a *function* mag giving each its single positive-real magnitude. It lists the options and picks none — a UnitSystem makes the pick. One catalog underlies many unit systems (SI , CGS , ...); the magnitude is the data toScale reads to drive rescaling.

```
class UnitMagnitudeCatalog (B : Type) where
  Unit : B → Type
  mag : {b : B} → Unit b → ℝ≥0
  mag_pos : ∀ {b : B} (u : Unit b), 0 < mag u
```

def UnitSystem[\[source\]](#)

A *unit system* over a basis B with a $\text{UnitMagnitudeCatalog}$: one typed unit chosen per base dimension — the pick off the catalog's menu, and the formal counterpart of SI , CGS , Over the default basis, $u.\text{length} : \text{LengthUnit}$, and a MassUnit cannot be placed in the length slot.

```
def UnitSystem (B : Type) [UnitMagnitudeCatalog B]
```

theorem ext[\[source\]](#)

Two unit systems agree once they pick the same unit at every base dimension: UnitSystem is extensional, as befits a choice-per-dimension.

```
@[ext]
```

```
theorem ext {u v : UnitSystem B} (h : ∀ b, u b = v b) : u = v
```

def toScale[\[source\]](#)

Forget a unit system to its magnitude layer $\text{UnitScale } B$: keep each chosen unit's magnitude, drop its type — the layer where $\text{UnitScale.dimScale}$ is already proved.

```
noncomputable def toScale (u : UnitSystem B) : UnitScale B where
```

lemma toScale_scale[\[source\]](#)

```
@[simp]
```

```
lemma toScale_scale (u : UnitSystem B) (b : B) :
  (toScale u).scale b = UnitMagnitudeCatalog.mag (u b)
```

instance UnitMagnitudeCatalog LTMCTDimensionBase[\[source\]](#)

```
noncomputable instance : UnitMagnitudeCatalog LTMCTDimensionBase where
```

def equivUnitSystem[\[source\]](#)

The bespoke five-field LTMCTUnitChoices and the generic $\text{UnitSystem } \text{LTMCTDimensionBase}$ are the same data.

```
def equivUnitSystem : LTMCTUnitChoices ≈ UnitSystem LTMCTDimensionBase where
```

lemma toScale_equivUnitSystem[\[source\]](#)

The typed generic projection agrees with the bespoke `LTMCTUnitChoices.toScale`: reading `LTMCTUnitChoices` as a `UnitSystem` and forgetting to the magnitude layer is the same as the bespoke `toScale`.

```
lemma toScale_equivUnitSystem (u : LTMCTUnitChoices) :
  (equivUnitSystem u).toScale = u.toScale
```

lemma prod_univ_LTMCTDimensionBase[\[source\]](#)

The product over the five default base quantities, in canonical enumeration order — a reusable fact about the `Fintype` enumeration of `LTMCTDimensionBase`, for folding `Finset.prod` over the default basis (e.g. in a hand-rolled scaling law).

```
lemma prod_univ_LTMCTDimensionBase {M : Type} [CommMonoid M]
  (f : LTMCTDimensionBase → M) :
  [] b, f b = f .length * f .time * f .mass * f .charge * f .temperature
```

lemma length_ratio[\[source\]](#)

```
private lemma length_ratio (u1 u2 : LTMCTUnitChoices) :
  u1.length / u2.length = u1.toScale.scale .length / u2.toScale.scale .length
```

lemma time_ratio[\[source\]](#)

```
private lemma time_ratio (u1 u2 : LTMCTUnitChoices) :
  u1.time / u2.time = u1.toScale.scale .time / u2.toScale.scale .time
```

lemma mass_ratio[\[source\]](#)

```
private lemma mass_ratio (u1 u2 : LTMCTUnitChoices) :
  u1.mass / u2.mass = u1.toScale.scale .mass / u2.toScale.scale .mass
```

lemma charge_ratio[\[source\]](#)

```
private lemma charge_ratio (u1 u2 : LTMCTUnitChoices) :
  u1.charge / u2.charge = u1.toScale.scale .charge / u2.toScale.scale .charge
```

lemma temperature_ratio[\[source\]](#)

```
private lemma temperature_ratio (u1 u2 : LTMCTUnitChoices) :
  u1.temperature / u2.temperature =
  u1.toScale.scale .temperature / u2.toScale.scale .temperature
```

lemma dimScale_eq_toScale_dimScale[\[source\]](#)

The hand-rolled five-factor `LTMCTUnitChoices.dimScale` equals the basis-generic `UnitScale.dimScale` fold at `LTMCTDimensionBase`, applied to `toScale`. The scaling law therefore has a single source of truth on the magnitude layer `UnitScale B`.

```
lemma dimScale_eq_toScale_dimScale (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
```

```
u1.dimScale u2 d = UnitScale.dimScale u1.toScale u2.toScale d
```

2.86 Physlib.Units.WithDim.Basic

[Physlib/Units/WithDim/Basic.lean](#) on GitHub.

instance Inhabited (WithDim d M)

[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [Inhabited M] :  
  Inhabited (WithDim d M) where
```

instance Zero (WithDim d M)

[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [Zero M] :  
  Zero (WithDim d M) where
```

instance Add (WithDim d M)

[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [Add M] :  
  Add (WithDim d M) where
```

instance Neg (WithDim d M)

[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [Neg M] :  
  Neg (WithDim d M) where
```

instance Sub (WithDim d M)

[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [Sub M] :  
  Sub (WithDim d M) where
```

instance AddSemigroup (WithDim d M)

[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [AddSemigroup M] :  
  AddSemigroup (WithDim d M) where
```

instance AddCommSemigroup (WithDim d M)

[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [AddCommSemigroup M] :  
  AddCommSemigroup (WithDim d M) where
```

instance AddMonoid (WithDim d M)

[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [AddMonoid M] :  
  AddMonoid (WithDim d M) where
```

instance AddCommMonoid (WithDim d M)

[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [AddCommMonoid M] :  
  AddCommMonoid (WithDim d M) where
```

instance AddGroup (WithDim d M)[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [AddGroup M] :
  AddGroup (WithDim d M) where
```

instance AddCommGroup (WithDim d M)[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [AddCommGroup M] :
  AddCommGroup (WithDim d M) where
```

instance LE (WithDim d M)[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [LE M] :
  LE (WithDim d M) where
```

instance LT (WithDim d M)[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [LT M] :
  LT (WithDim d M) where
```

instance Preorder (WithDim d M)[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [Preorder M] :
  Preorder (WithDim d M) where
```

instance PartialOrder (WithDim d M)[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [PartialOrder M] :
  PartialOrder (WithDim d M) where
```

instance MulAction $\mathbb{R}_{\geq 0}$ (WithDim d M)[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] (d : Dimension B) (M : Type) [MulAction  $\mathbb{R}_{\geq 0}$  M] :
  MulAction  $\mathbb{R}_{\geq 0}$  (WithDim d M) where
```

instance HMul (WithDim d1 $\mathbb{R}$) (WithDim d2 $\mathbb{R}$) (WithDim (d1 * d2) $\mathbb{R}$)[\[source\]](#)

```
instance {B : Type} [DimensionBasis B] {d1 d2 : Dimension B} :
  HMul (WithDim d1  $\mathbb{R}$ ) (WithDim d2  $\mathbb{R}$ ) (WithDim (d1 * d2)  $\mathbb{R}$ ) where
```

instance HDiv (WithDim d1 $\mathbb{R}$) (WithDim d2 $\mathbb{R}$) (WithDim (d1 * d2⁻¹) $\mathbb{R}$)[\[source\]](#)

```
noncomputable instance {B : Type} [DimensionBasis B] (d1 d2 : Dimension B) :
  HDiv (WithDim d1  $\mathbb{R}$ ) (WithDim d2  $\mathbb{R}$ ) (WithDim (d1 * d2-1)  $\mathbb{R}$ ) where
```

2.87 PhyslibAlpha.Mathematics.LadderSystem.Basic

[PhyslibAlpha/Mathematics/LadderSystem/Basic.lean](#) on GitHub.

def bracketRight[\[source\]](#)

Bracketing on the right by a fixed element, as a K -linear self-map.

```
def bracketRight (y : L) : L →L[K] L where
```

def bracketLeft[\[source\]](#)

Bracketing on the left by a fixed element, as a K -linear self-map.

```
def bracketLeft (x : L) : L →L[K] L where
```

theorem lie_apply_eq_of_lie_basis_eq[\[source\]](#)

A K -linear map agreeing with the bracket on basis pairs agrees with the bracket everywhere.

```
theorem LinearMap.lie_apply_eq_of_lie_basis_eq (b : Basis ι K L) (f : L →L[K] L')
(h : ∀ i j, f {b i, b j} = {f (b i), f (b j)}) (x y : L) :
f {x, y} = {f x, f y}
```

theorem lie_apply_eq_of_lie_span_eq[\[source\]](#)

*Same, from a spanning set rather than a basis (`LinearMap.ext_on` in place of `Basis.ext`)
— for generators that span but aren't worth packaging as an indexed basis.*

```
theorem LinearMap.lie_apply_eq_of_lie_span_eq {s : Set L} (hs : Submodule.span K s = L)
(f : L →L[K] L') (h : ∀ x ∈ s, ∀ y ∈ s, f {x, y} = {f x, f y}) (x y : L) :
f {x, y} = {f x, f y}
```

structure LadderSystem[\[source\]](#)

A system of d independent ladder pairs on a K -vector space V : annihilation operators a_i and creation operators ac_i ($i : \text{Fin } d$), satisfying the canonical commutation relations. A physical system exhibits its own concrete operators as one instance of this (e.g. `QuantumMechanics/HarmonicOscillator/LadderOperators.lean`'s `toLadderSystem`, via `ContinuousLinearMap.toLinearMap`) and inherits every result in `LadderSystem` for free.

```
structure LadderSystem (K V : Type*) [Field K] [AddCommGroup V] [Module K V] (d : ℕ) where
  a : Fin d → Module.End K V
  ac : Fin d → Module.End K V
  comm_a_ac : ∀ i j, ({a i, ac j} : Module.End K V) = if i = j then 1 else 0
  comm_a_a : ∀ i j, ({a i, a j} : Module.End K V) = 0
  comm_ac_ac : ∀ i j, ({ac i, ac j} : Module.End K V) = 0
```

lemma assoc_leibniz[\[source\]](#)

```
lemma assoc_leibniz (x y z : Module.End K V) :
({x, y * z} : Module.End K V) = {x, y} * z + y * {x, z}
```

lemma assoc_leibniz'[\[source\]](#)

```
lemma assoc_leibniz' (x y z : Module.End K V) :
  ([x * y, z] : Module.End K V) = x * [y, z] + [x, z] * y
```

def E

[\[source\]](#)

The $gl(d)$ bilinears $E\ i\ j := ac\ i \cdot a\ j$.

```
def E (i j : Fin d) : Module.End K V
```

theorem lie_E

[\[source\]](#)

E satisfies the defining relations of $gl(d)$: $[E\ i\ j, E\ k\ l] = \delta_{jk} E\ i\ l - \delta_{li} E\ k\ j$.

```
theorem lie_E (i j k l : Fin d) :
  ([L.E i j, L.E k l] : Module.End K V)
  = ( $\delta[j,k]$  : K) • L.E i l - ( $\delta[l,i]$  : K) • L.E k j
```

lemma lie_single

[\[source\]](#)

The matrix units satisfy the same relations, at the level of $gl(d)$ itself.

```
lemma lie_single (i j k l : Fin d) :
  ([Matrix.single i j (1 : K), Matrix.single k l (1 : K)] : Matrix (Fin d) (Fin d) K)
  = ( $\delta[j,k]$  : K) • Matrix.single i l 1 - ( $\delta[l,i]$  : K) • Matrix.single k j 1
```

def toGLHomLinear

[\[source\]](#)

The K -linear map sending matrix unit $Matrix.single\ i\ j\ 1$ to $E\ i\ j$.

```
noncomputable def toGLHomLinear (L : LadderSystem K V d) :
  Matrix (Fin d) (Fin d) K →l[K] Module.End K V
```

lemma toGLHomLinear_single

[\[source\]](#)

```
lemma toGLHomLinear_single (L : LadderSystem K V d) (i j : Fin d) :
  L.toGLHomLinear (Matrix.single i j (1 : K)) = L.E i j
```

lemma toGLHomLinear_lie

[\[source\]](#)

toGLHomLinear respects brackets.

```
lemma toGLHomLinear_lie (L : LadderSystem K V d) (x y : Matrix (Fin d) (Fin d) K) :
  L.toGLHomLinear [x, y] = [L.toGLHomLinear x, L.toGLHomLinear y]
```

def toGLHom

[\[source\]](#)

The $gl(d)$ -representation on V generated by a ladder system, bundled as a $LieHom$.

```
noncomputable def toGLHom (L : LadderSystem K V d) :
  Matrix (Fin d) (Fin d) K →l[K] Module.End K V
```

lemma toLieModule[\[source\]](#)

The companion LieModule instance.

```
lemma toLieModule (L : LadderSystem K V d) :
  @LieModule K (Matrix (Fin d) (Fin d) K) V _ _ _ _ L.toLieRingModule
```

def N[\[source\]](#)

The number operator for mode i , $N_i = E_{ii} = a_{ii}a_i$.

```
def N (i : Fin d) : Module.End K V
```

lemma lie_N_a[\[source\]](#)

$[N_i, a_j] = -\delta_{ij}a_i$.

```
lemma lie_N_a (i j : Fin d) :
  ({L.N i, L.a j} : Module.End K V) = (-δ[i,j] : K) • L.a i
```

lemma lie_N_ac[\[source\]](#)

$[N_i, ac_j] = \delta_{ij}ac_i$.

```
lemma lie_N_ac (i j : Fin d) :
  ({L.N i, L.ac j} : Module.End K V) = (δ[i,j] : K) • L.ac i
```

lemma lie_N_N[\[source\]](#)

$[N_i, N_j] = 0$: the number operators commute among themselves.

```
lemma lie_N_N (i j : Fin d) : ({L.N i, L.N j} : Module.End K V) = 0
```

def totalN[\[source\]](#)

The total number operator, $\sum_i N_i$ – the total excitation count across every mode.

```
def totalN (L : LadderSystem K V d) : Module.End K V
```

theorem lie_totalN_E[\[source\]](#)

*The total number operator is central in $\mathfrak{gl}(d)$: it commutes with every generator E_{ij} , not just the diagonal ones (contrast `lie_N_N`, which only says the diagonal N_i commute *among themselves*). Consequently any operator built from `L.totalN` alone – in particular an *isotropic* Hamiltonian – preserves `vacuumSpan` sector-by-sector for the same reason `vacuumSpanLieSubmodule` does: it is `L.toGLHomLinear` applied to a central element of $\mathfrak{gl}(d)$ (the identity matrix), so it acts diagonally on the whole $\mathfrak{gl}(d)$ -module structure.*

```
theorem lie_totalN_E (j k : Fin d) : ({L.totalN, L.E j k} : Module.End K V) = 0
```

2.88 PhyslibAlpha.Mathematics.LadderSystem.Irreducibility

[PhyslibAlpha/Mathematics/LadderSystem/Irreducibility.lean](#) on GitHub.

def moveOneTo[\[source\]](#)

α with one quantum moved from color j to color i . Only meaningful for $i \neq j$; the degenerate $i = j$ case is handled separately by `N_word`.

```
def moveOneTo (α : Fin d → ℕ) (i j : Fin d) : Fin d → ℕ
```

lemma countWord_moveOneTo_perm[\[source\]](#)

```
lemma countWord_moveOneTo_perm {i j : Fin d} (hij : i ≠ j) {α : Fin d → ℕ} :
  List.Perm (i :: (countWord d α).erase j) (countWord d (moveOneTo α i j))
```

lemma E_word_countWord[\[source\]](#)

The transfer formula: $E_{i j}$ moves one quantum from color j to color i , scaled by j 's occupation number.

```
lemma E_word_countWord {Ω : V} (P : L.HasVacuum Ω) {i j : Fin d} (hij : i ≠ j)
  {α : Fin d → ℕ} :
  L.E i j (L.word (countWord d α) Ω) =
    (α j : K) • L.word (countWord d (moveOneTo α i j)) Ω
```

lemma mapsTo_aeval[\[source\]](#)

A submodule invariant under an endomorphism M is invariant under any polynomial in M .

```
lemma mapsTo_aeval {W : Submodule K V} {M : Module.End K V} (hW : ∀ w ∈ W, M w ∈ W)
  (p : Polynomial K) : ∀ w ∈ W, (Polynomial.aeval M p) w ∈ W
```

lemma exists_word_countWord_mem_of_ne_bot[\[source\]](#)

A nonzero $E_{i j}$ -invariant submodule of $\text{vacuumSpan } L \Omega n$ contains an occupation-number basis vector. A Lagrange-interpolation polynomial in the diagonal separator $\sum_i (n+1)^i \cdot N_i$ projects a nonzero element onto one of its basis components.

```
lemma exists_word_countWord_mem_of_ne_bot {Ω : V} (P : L.HasVacuum Ω) (n : ℕ)
  {W : Submodule K V} (hWle : W ≤ vacuumSpan L Ω n)
  (hWE : ∀ i j, ∀ w ∈ W, L.E i j w ∈ W) (hWbot : W ≠ ⊥) :
  ∃ α : CountFun d n, L.word (countWord d α.1) Ω ∈ W
```

lemma sum_eq_length_countWord[\[source\]](#)

The total occupation number matches the length of the canonical word representing it.

```
lemma sum_eq_length_countWord (α : Fin d → ℕ) : (∑ c, α c) = (countWord d α).length
```

lemma sum_moveOneTo[\[source\]](#)

Moving one quantum between two modes preserves the total occupation number.

```
lemma sum_moveOneTo {i j : Fin d} (hij : i ≠ j) {α : Fin d → ℕ} (hj : α j ≠ 0) :
  (∑ c, moveOneTo α i j c) = ∑ c, α c
```

lemma moveOneTo_self[\[source\]](#)

*moveOneTo at the color it moves quanta *into* just increments that color's count.*

```
lemma moveOneTo_self (α : Fin d → ℕ) (i j : Fin d) : moveOneTo α i j i = α i + 1
```

lemma moveOneTo_moveOneTo[\[source\]](#)

Moving a quantum from j to i and then immediately back from i to j is the identity.

```
lemma moveOneTo_moveOneTo {i j : Fin d} (hij : i ≠ j) {α : Fin d → ℕ} (hj : α j ≠ 0) :
  moveOneTo (moveOneTo α i j) j i = α
```

lemma word_countWord_moveOneTo_mem[\[source\]](#)

The atomic move: if W is E i j-invariant and contains the word for β, and mode k is occupied, W also contains the word obtained by moving one quantum from k to any other mode l.

```
lemma word_countWord_moveOneTo_mem {Ω : V} (P : L.HasVacuum Ω) {W : Submodule K V}
  (hWE : ∀ i j, ∀ w ∈ W, L.E i j w ∈ W) {β : Fin d → ℕ}
  (hmem : L.word (countWord d β) Ω ∈ W) {l k : Fin d} (hkl : l ≠ k) (hk : β k ≠ 0) :
  L.word (countWord d (moveOneTo β l k)) Ω ∈ W
```

lemma word_countWord_hub_mem_of_mem[\[source\]](#)

Every occupation-number word reaches the mode-0 hub word, given W is E i j-invariant and contains it. Proved by strong induction on the mass sitting outside mode 0.

```
lemma word_countWord_hub_mem_of_mem {hd : 0 < d} {Ω : V} (P : L.HasVacuum Ω)
  {W : Submodule K V} (hWE : ∀ i j, ∀ w ∈ W, L.E i j w ∈ W) :
  ∀ m : ℕ, ∀ α : Fin d → ℕ, (Σ c, α c) - α ((0, hd) : Fin d) = m →
  L.word (countWord d α) Ω ∈ W →
  L.word (countWord d (fun c => if c = ((0, hd) : Fin d) then (Σ c, α c) else 0)) Ω ∈ W
```

lemma word_countWord_mem_of_hub_mem[\[source\]](#)

The hub word reaches every occupation-number word, given W is E i j-invariant and contains it. Proved by strong induction on the mass sitting outside mode 0 in the target.

```
lemma word_countWord_mem_of_hub_mem {hd : 0 < d} {Ω : V} (P : L.HasVacuum Ω)
  {W : Submodule K V} (hWE : ∀ i j, ∀ w ∈ W, L.E i j w ∈ W) :
  ∀ m : ℕ, ∀ α : Fin d → ℕ, (Σ c, α c) - α ((0, hd) : Fin d) = m →
  L.word (countWord d (fun c => if c = ((0, hd) : Fin d) then (Σ c, α c) else 0)) Ω ∈ W →
  L.word (countWord d α) Ω ∈ W
```

lemma word_countWord_mem_of_mem[\[source\]](#)

Connectivity: any two occupation-number words of the same total excitation number reach each other, given W is E i j-invariant and contains one of them.

```
lemma word_countWord_mem_of_mem {hd : 0 < d} {Ω : V} (P : L.HasVacuum Ω)
  {W : Submodule K V} (hWE : ∀ i j, ∀ w ∈ W, L.E i j w ∈ W) {α α' : Fin d → ℕ}
```

```
(hsum : (∑ c, α c) = ∑ c, α' c) (hmem : L.word (countWord d α) Ω ∈ W) :
L.word (countWord d α') Ω ∈ W
```

theorem vacuumSpan_eq_of_ne_bot[\[source\]](#)

vacuumSpan L Ω n is $gl(d)$ -irreducible. The only submodules of vacuumSpan L Ω n invariant under every E_{ij} are $\perp$ and vacuumSpan L Ω n itself.

```
theorem vacuumSpan_eq_of_ne_bot {hd : 0 < d} {Ω : V} (P : L.HasVacuum Ω) (n : ℕ)
{W : Submodule K V} (hWle : W ≤ vacuumSpan L Ω n)
(hWE : ∀ i j, ∀ w ∈ W, L.E i j w ∈ W) (hWbot : W ≠ ⊥) :
W = vacuumSpan L Ω n
```

2.89 PhyslibAlpha.Mathematics.LadderSystem.OccupationBasis

[PhyslibAlpha/Mathematics/LadderSystem/OccupationBasis.lean](#) on GitHub.

abbrev CountFun[\[source\]](#)

A degree-n count function on d colors.

```
abbrev CountFun (d n : ℕ)
```

def countFunEquivSym[\[source\]](#)

Degree-n count functions on d colors biject with Sym (Fin d) n.

```
noncomputable def countFunEquivSym (d n : ℕ) : CountFun d n ≈ Sym (Fin d) n
```

instance instFintypeCountFun[\[source\]](#)

```
noncomputable instance instFintypeCountFun {d n : ℕ} : Fintype (CountFun d n)
```

lemma card_countFun[\[source\]](#)

There are exactly $(d + n - 1).choose\ n$ degree-n count functions on d colors.

```
lemma card_countFun (d n : ℕ) : Fintype.card (CountFun d n) = (d + n - 1).choose n
```

lemma pow_a_comm_ac[\[source\]](#)

Annihilation and creation operators of different colors commute at any power.

```
lemma pow_a_comm_ac {i c : Fin d} (hic : i ≠ c) (k : ℕ) (y : V) :
((L.a i) ^ k) (L.ac c y) = L.ac c (((L.a i) ^ k) y)
```

lemma word_peel_eq_count[\[source\]](#)

Exact annihilation: applying a i exactly count i times removes every i, scaled by the factorial.

```
lemma word_peel_eq_count (L : LadderSystem K V d) (i : Fin d) {x : V} (hx : L.a i x = 0) :
∀ v : List (Fin d),
```

```

      ((L.a i) ^ (v.count i)) (L.word v x) = (v.count i).factorial • L.word (v.filter (· != i))
    x
| [] => by simp [word]
| c :: v' => by
  by_cases hic : i = c
  · subst hic
  have hpeel

```

def annMono[\[source\]](#)

The joint annihilation monomial for a list of colors cs and exponents α : $\prod_{c \in cs} (a \ c)^{\alpha \ c}$.

```
def annMono (cs : List (Fin d)) (α : Fin d → ℕ) : Module.End K V
```

lemma annMono_cons[\[source\]](#)

```
lemma annMono_cons (c : Fin d) (cs : List (Fin d)) (α : Fin d → ℕ) :
  L.annMono (c :: cs) α = (L.a c) ^ (α c) * L.annMono cs α
```

lemma annMono_eq_count[\[source\]](#)

Exact annihilation of every color in the list produces the product of factorials.

```
lemma annMono_eq_count (L : LadderSystem K V d) {x : V} (hx : ∀ i, L.a i x = 0) :
  ∀ (cs : List (Fin d)), cs.Nodup → ∀ (α : Fin d → ℕ) (v : List (Fin d)),
    (∀ c ∈ cs, v.count c = α c) →
      (L.annMono cs α) (L.word v x)
        = ((cs.map (fun c => (α c).factorial)).prod) •
          L.word (v.filter (fun y => !cs.contains y)) x
| [], _, _, _ => by simp
| c :: cs', hnodup, α, v, hmatch => by
  obtain (hcnotmem, hnodup')
```

lemma countWord_filter_finRange[\[source\]](#)

$\text{countWord } \alpha$ contains only colors from $\text{List.finRange } d$, i.e. everything.

```
lemma countWord_filter_finRange {d : ℕ} (α : Fin d → ℕ) :
  (LadderSystem.countWord d α).filter (fun y => !(List.finRange d).contains y) = []
```

lemma encode_inj0n_aux[\[source\]](#)

A base- b positional encoding is injective on tuples with digits $< b$.

```
lemma encode_inj0n_aux : ∀ (d b : ℕ), 0 < b → ∀ γ γ' : Fin d → ℕ,
  (∀ c, γ c < b) → (∀ c, γ' c < b) →
  (∑ i : Fin d, γ i * b ^ (i : ℕ)) = (∑ i : Fin d, γ' i * b ^ (i : ℕ)) → γ = γ'
```

def countEncode[\[source\]](#)

The base- $(n+1)$ positional encoding of a degree- n count function; injective on such functions.

```
def countEncode (n : ℕ) {d : ℕ} (γ : Fin d → ℕ) : ℕ
```

lemma countEncode_inj0n

[\[source\]](#)

```
lemma countEncode_inj0n {d n : ℕ} {γ γ' : Fin d → ℕ}
  (hγ : ∑ c, γ c = n) (hγ' : ∑ c, γ' c = n) (heq : countEncode n γ = countEncode n γ') :
  γ = γ'
```

lemma word_countWord_ne_zero

[\[source\]](#)

Every occupation-number state is nonzero: peeling every color's annihilation operator down to the vacuum leaves a nonzero factorial multiple of Ω .

```
lemma word_countWord_ne_zero {Ω : V} (hΩ : L.HasVacuum Ω) (α : Fin d → ℕ) :
  L.word (countWord d α) Ω ≠ 0
```

lemma hasEigenvector_word_countWord

[\[source\]](#)

The occupation-number states are the eigenvectors, at pairwise-distinct eigenvalues, of the single diagonal operator $M := \sum_i (n+1)^i \cdot N_i$.

```
lemma hasEigenvector_word_countWord {Ω : V} (hΩ : L.HasVacuum Ω) (n : ℕ) (α : CountFun d n) :
  Module.End.HasEigenvector (∑ i : Fin d, ((n + 1 : K)) ^ (i : ℕ) • L.N i)
    (countEncode n α.1 : K) (L.word (countWord d α.1) Ω)
```

theorem linearIndependent_word_countFun

[\[source\]](#)

The occupation-number family is linearly independent because its elements are eigenvectors at pairwise-distinct eigenvalues (`Module.End.eigenvectors_linearIndependent'`).

```
theorem linearIndependent_word_countFun {Ω : V} (P : L.HasVacuum Ω) (n : ℕ) :
  LinearIndependent K (fun α : CountFun d n => L.word (countWord d α.1) Ω)
```

lemma span_word_countFun_eq_vacuumSpan

[\[source\]](#)

The occupation-number family spans `vacuumSpan L Ω n`.

```
lemma span_word_countFun_eq_vacuumSpan {Ω : V} (n : ℕ) :
  Submodule.span K (Set.range fun α : CountFun d n => L.word (countWord d α.1) Ω)
    = L.vacuumSpan Ω n
```

def vacuumBasis

[\[source\]](#)

A basis of `vacuumSpan L Ω n` indexed by degree- n occupation-count functions.

```
noncomputable def vacuumBasis {Ω : V} (P : L.HasVacuum Ω) (n : ℕ) :
  Basis (CountFun d n) K (L.vacuumSpan Ω n)
```

lemma vacuumBasis_apply

[\[source\]](#)

```
lemma vacuumBasis_apply {Ω : V} (P : L.HasVacuum Ω) (n : ℕ) (α : CountFun d n) :
  (vacuumBasis L P n α : V) = L.word (countWord d α.1) Ω
```

theorem finrank_vacuumSpan_eq_choose[\[source\]](#)

The dimension of $\text{vacuumSpan } L \ \Omega \ n$ is $(d+n-1).choose \ n$, the degeneracy of the n -th level of a d -mode bosonic system.

```
theorem finrank_vacuumSpan_eq_choose {Ω : V} (P : L.HasVacuum Ω) (n : ℕ) :
  Module.finrank K (L.vacuumSpan Ω n) = (d + n - 1).choose n
```

lemma finiteDimensional_vacuumSpan[\[source\]](#)

```
lemma finiteDimensional_vacuumSpan {Ω : V} (P : L.HasVacuum Ω) (n : ℕ) :
  FiniteDimensional K (L.vacuumSpan Ω n)
```

lemma vacuumSpan_ne_bot[\[source\]](#)

```
lemma vacuumSpan_ne_bot {Ω : V} (P : L.HasVacuum Ω) (hd : 0 < d) (n : ℕ) :
  L.vacuumSpan Ω n ≠ ⊥
```

2.90 PhyslibAlpha.Mathematics.LadderSystem.SymmetricPower

[PhyslibAlpha/Mathematics/LadderSystem/SymmetricPower.lean](#) on GitHub.

def vacuumSpanSymEquiv[\[source\]](#)

$\text{vacuumSpan } L \ \Omega \ n$ is linearly isomorphic to $\text{Sym}^n(K^d)$ (realized as the free K -module on $\text{Sym}(\text{Fin } d) \ n$), matching the natural monomial-type basis on one side to the occupation-number basis on the other.

```
noncomputable def vacuumSpanSymEquiv {Ω : V} (P : L.HasVacuum Ω) (n : ℕ) :
  (Sym (Fin d) n →• K) ≈[K] L.vacuumSpan Ω n
```

lemma vacuumSpanSymEquiv_single[\[source\]](#)

The isomorphism sends the basis vector for multiset s to the occupation-number state with that multiset's count function.

```
lemma vacuumSpanSymEquiv_single {Ω : V} (P : L.HasVacuum Ω) (n : ℕ) (s : Sym (Fin d) n) :
  (L.vacuumSpanSymEquiv P n (Finsupp.single s 1) : V)
  = L.word (countWord d ((countFunEquivSym d n).symm s).1) Ω
```

2.91 PhyslibAlpha.Mathematics.LadderSystem.Vacuum

[PhyslibAlpha/Mathematics/LadderSystem/Vacuum.lean](#) on GitHub.

structure HasVacuum[\[source\]](#)

A vacuum: a nonzero vector killed by every annihilation operator.

```
structure HasVacuum (Ω : V) : Prop where
  ne_zero : Ω ≠ 0
  ann : ∀ i, L.a i Ω = 0
```

def word[\[source\]](#)

A word of creation operators applied to a fixed vector.

```
def word (L : LadderSystem K V d) (v : List (Fin d)) (x : V) : V
```

lemma word_cons[\[source\]](#)

```
lemma word_cons (L : LadderSystem K V d) (i : Fin d) (v : List (Fin d)) (x : V) :  
  L.word (i :: v) x = L.ac i (L.word v x)
```

lemma word_perm[\[source\]](#)

A word depends only on the multiset of colors it represents, not their order.

```
lemma word_perm (L : LadderSystem K V d) {v v' : List (Fin d)} (h : v.Perm v') (x : V) :  
  L.word v x = L.word v' x
```

lemma word_peel[\[source\]](#)

Peeling: a i applied to a word removes one occurrence of color i , scaled by its count.

```
lemma word_peel (L : LadderSystem K V d) (i : Fin d) {x : V} (hx : L.a i x = 0) :  
  ∀ v : List (Fin d), L.a i (L.word v x) = (v.count i) • L.word (v.erase i) x  
| [] => by simp [word, hx]  
| c :: v' => by  
  have hcomm : L.a i * L.ac c  
    = L.ac c * L.a i + (if i = c then (1 : Module.End K V) else 0)
```

lemma E_word[\[source\]](#)

How E_{ij} acts on a word: removes one occurrence of j , adds one of i , scaled by j 's count.

```
lemma E_word (L : LadderSystem K V d) {x : V} (hx : ∀ i, L.a i x = 0) (i j : Fin d)  
  (v : List (Fin d)) :  
  (L.E i j) (L.word v x) = (v.count j) • L.word (i :: v.erase j) x
```

lemma N_word[\[source\]](#)

Words are eigenvectors of the number operator, with eigenvalue their own color count.

```
lemma N_word (L : LadderSystem K V d) {x : V} (hx : ∀ i, L.a i x = 0) (i : Fin d)  
  (v : List (Fin d)) : (L.N i) (L.word v x) = (v.count i) • L.word v x
```

def countWord[\[source\]](#)

The canonical word representing a count function α : α c copies of color c , in increasing order.

```
def countWord (d : ℕ) (α : Fin d → ℕ) : List (Fin d)
```

lemma count_countWord[\[source\]](#)

countWord α has exactly α i copies of color i .

```
lemma count_countWord {d : ℕ} (α : Fin d → ℕ) (i : Fin d) :
  (countWord d α).count i = α i
```

lemma sum_count_eq_length[\[source\]](#)

A word's length is the sum of its per-color counts.

```
lemma sum_count_eq_length {d : ℕ} : ∀ v : List (Fin d), (∑ c : Fin d, v.count c) = v.length
| [] => by simp
| a :: v' => by
  have hcount : ∀ c : Fin d, (a :: v').count c = v'.count c + (if a = c then 1 else 0)
```

def vacuumSpan[\[source\]](#)

The n -th excitation-number sector generated by a vacuum: the span of all length- n creation words.

```
def vacuumSpan (L : LadderSystem K V d) (Ω : V) (n : ℕ) : Submodule K V
```

lemma exists_ofFn_eq_of_length_eq[\[source\]](#)

```
lemma exists_ofFn_eq_of_length_eq {l : List (Fin d)} {n : ℕ} (h : l.length = n) :
  ∃ w : Fin n → Fin d, List.ofFn w = l
```

lemma word_mem_vacuumSpan_of_length_eq[\[source\]](#)

Any word of the right length lies in vacuumSpan.

```
lemma word_mem_vacuumSpan_of_length_eq (L : LadderSystem K V d) (Ω : V) {n : ℕ}
{l : List (Fin d)} (h : l.length = n) : L.word l Ω ∈ vacuumSpan L Ω n
```

lemma forall_toGLHomLinear_mem_of_forall_E_mem[\[source\]](#)

E -invariance of a submodule extends to full $gl(d)$ -invariance, by linearity off the matrix-unit basis.

```
lemma forall_toGLHomLinear_mem_of_forall_E_mem (L : LadderSystem K V d) {W : Submodule K V}
(hw : ∀ i j, ∀ w ∈ W, (L.E i j) w ∈ W) (x : Matrix (Fin d) (Fin d) K) {w : V}
(hw : w ∈ W) : L.toGLHomLinear x w ∈ W
```

lemma E_mem_vacuumSpan[\[source\]](#)

vacuumSpan is closed under every $gl(d)$ generator E i j . Unconditional: no FiniteDimensional/spanning hypothesis on the ambient V is used or needed.

```
lemma E_mem_vacuumSpan (L : LadderSystem K V d) {Ω : V} (P : L.HasVacuum Ω) (n : ℕ)
(i j : Fin d) : ∀ v ∈ vacuumSpan L Ω n, (L.E i j) v ∈ vacuumSpan L Ω n
```

def vacuumSpanLieSubmodule[\[source\]](#)

vacuumSpan, bundled as a genuine $gl(d)$ Lie submodule.

```
def vacuumSpanLieSubmodule (L : LadderSystem K V d) {Ω : V} (P : L.HasVacuum Ω) (n : ℕ) :
  haveI
```

2.92 PhyslibAlpha.QuantumMechanics.HarmonicOscillator.Basic

[PhyslibAlpha/QuantumMechanics/HarmonicOscillator/Basic.lean](#) on GitHub.

lemma potentialFunction_apply[\[source\]](#)

The potential function, expanded: $\frac{1}{2}m \cdot \sum_i \omega_i^2 x_i^2$.

```
lemma potentialFunction_apply (x : Space d) :
  Q.potentialFunction x = (2⁻¹ * Q.m) * ∑ i, (Q.ω i) ^ 2 * (x i) ^ 2
```

lemma potentialFunction_continuous[\[source\]](#)

The potential function for the harmonic oscillator is continuous, hence a.e. strongly measurable.

```
lemma potentialFunction_continuous : Continuous Q.potentialFunction
```

2.93 PhyslibAlpha.QuantumMechanics.HarmonicOscillator.LadderOperators

[PhyslibAlpha/QuantumMechanics/HarmonicOscillator/LadderOperators.lean](#) on GitHub.

def annihilationCLM[\[source\]](#)

*The annihilation operator for mode i , $\mathbf{a}_i = (\sqrt{2})^{-1}(\xi_i^{-1}\mathbf{x}_i + i(\xi_i/\hbar)\mathbf{p}_i)$, where ξ_i is the characteristic length of that mode (*HarmonicOscillator.ξ*).*

```
def annihilationCLM (Q : HarmonicOscillator d) (i : Fin d) :
  S(Space d, ℂ) → L[ℂ] S(Space d, ℂ)
```

lemma annihilationCLM_apply_fun[\[source\]](#)

```
lemma annihilationCLM_apply_fun (Q : HarmonicOscillator d) (i : Fin d) (ψ : S(Space d, ℂ)) :
  Q.annihilationCLM i ψ =
    (√2 : ℂ)⁻¹ • ((Q.ξ i : ℂ)⁻¹ • x i ψ + (I * Q.ξ i / ħ) • p i ψ)
```

def creationCLM[\[source\]](#)

The creation operator for mode i , $\mathbf{a}_i^+ = (\sqrt{2})^{-1}(\xi_i^{-1}\mathbf{x}_i - i(\xi_i/\hbar)\mathbf{p}_i)$.

```
def creationCLM (Q : HarmonicOscillator d) (i : Fin d) :
  S(Space d, ℂ) → L[ℂ] S(Space d, ℂ)
```

lemma creationCLM_apply_fun[\[source\]](#)

```
lemma creationCLM_apply_fun (Q : HarmonicOscillator d) (i : Fin d) (ψ : S(Space d, ℂ)) :
  Q.creationCLM i ψ =
    (√2 : ℂ)⁻¹ • ((Q.ξ i : ℂ)⁻¹ • x i ψ - (I * Q.ξ i / ħ) • p i ψ)
```

theorem annihilationCLM_comm_annihilationCLM[\[source\]](#)

$$[a_i, a_j] = 0.$$

@[simp]

theorem annihilationCLM_comm_annihilationCLM (Q : HarmonicOscillator d) (i j : Fin d) :
 ({Q.annihilationCLM i, Q.annihilationCLM j} : $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$) = 0

theorem creationCLM_comm_creationCLM[\[source\]](#)

$$[a_i^+, a_j^+] = 0.$$

@[simp]

theorem creationCLM_comm_creationCLM (Q : HarmonicOscillator d) (i j : Fin d) :
 ({Q.creationCLM i, Q.creationCLM j} : $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$) = 0

theorem annihilationCLM_comm_creationCLM[\[source\]](#)

The canonical commutation relations for the ladder operators, $[a_i, a_j^+] = \delta_{ij}$.

theorem annihilationCLM_comm_creationCLM (Q : HarmonicOscillator d) (i j : Fin d) :
 ({Q.annihilationCLM i, Q.creationCLM j} : $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$) =
 ($\delta[i, j] : \mathbb{C}$) • (ContinuousLinearMap.id $\mathbb{C} \mathcal{S}(\text{Space } d, \mathbb{C})$)

def toLadderSystem[\[source\]](#)

*The annihilation and creation operators on Schwartz space, bundled as a LadderSystem.
 The number operator and its commutation relations below are obtained from this structure.*

def toLadderSystem (Q : HarmonicOscillator d) : LadderSystem $\mathbb{C} (\mathcal{S}(\text{Space } d, \mathbb{C}))$ d **where**

def numberCLM[\[source\]](#)

The number operator for mode i, $N_i = a_i^+ \circ a_i$, via toLadderSystem.N.

def numberCLM (Q : HarmonicOscillator d) (i : Fin d) :
 $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$

theorem toLadderSystem_N_toLinearMap[\[source\]](#)

theorem toLadderSystem_N_toLinearMap (Q : HarmonicOscillator d) (i : Fin d) :
 Q.toLadderSystem.N i = (Q.numberCLM i).toLinearMap

theorem numberCLM_comm_annihilationCLM[\[source\]](#)

$[N_i, a_j] = -\delta_{ij} a_i$, from the general LadderSystem.lie_N_a applied to toLadderSystem.

theorem numberCLM_comm_annihilationCLM (Q : HarmonicOscillator d) (i j : Fin d) :
 ({Q.numberCLM i, Q.annihilationCLM j} : $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$) =
 ($-\delta[i, j] : \mathbb{C}$) • Q.annihilationCLM i

theorem numberCLM_comm_creationCLM[\[source\]](#)

$[N_i, a_j^+] = \delta_{ij} a_i^+$, from the general *LadderSystem.lie_N_ac* applied to *toLadderSystem*.

```
theorem numberCLM_comm_creationCLM (Q : HarmonicOscillator d) (i j : Fin d) :
  ([Q.numberCLM i, Q.creationCLM j] :  $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$ ) =
  ( $\delta[i,j] : \mathbb{C}$ ) • Q.creationCLM i
```

theorem numberCLM_comm_numberCLM[\[source\]](#)

$[N_i, N_j] = 0$, from the general *LadderSystem.lie_N_N* applied to *toLadderSystem*.

```
@[simp]
theorem numberCLM_comm_numberCLM (Q : HarmonicOscillator d) (i j : Fin d) :
  ([Q.numberCLM i, Q.numberCLM j] :  $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$ ) = 0
```

2.94 PhyslibAlpha.QuantumMechanics.HarmonicOscillator.Vacuum

[PhyslibAlpha/QuantumMechanics/HarmonicOscillator/Vacuum.lean](#) on GitHub.

lemma deriv_realGaussian[\[source\]](#)

The real-valued Gaussian $\exp(-\|x\|^2/2)$'s coordinate derivative, $\partial_i g = -x_i \cdot g$.

```
lemma deriv_realGaussian (x : Space d) (i : Fin d) :
  Space.deriv i (fun x : Space d => Real.exp (-2-1 * ||x|| ^ 2)) x =
  -x i * Real.exp (-2-1 * ||x|| ^ 2)
```

lemma stdGaussian_apply'[\[source\]](#)

stdGaussian (Space d) $\mathbb{C}$'s pointwise value is the real-valued Gaussian, cast to $\mathbb{C}$.

```
lemma stdGaussian_apply' (x : Space d) :
  (stdGaussian (Space d)  $\mathbb{C}$  : Space d  $\rightarrow \mathbb{C}$ ) x = Real.exp (-2-1 * ||x|| ^ 2)
```

lemma deriv_stdGaussian[\[source\]](#)

The coordinate derivative of the standard Gaussian, $\partial_i \Omega = -x_i \cdot \Omega$.

```
lemma deriv_stdGaussian (x : Space d) (i : Fin d) :
  Space.deriv i (fun x : Space d => (stdGaussian (Space d)  $\mathbb{C}$  : Space d  $\rightarrow \mathbb{C}$ ) x) x =
  (-x i :  $\mathbb{C}$ ) * (stdGaussian (Space d)  $\mathbb{C}$  : Space d  $\rightarrow \mathbb{C}$ ) x
```

lemma momentumCLM_stdGaussian[\[source\]](#)

The momentum operator applied to the standard Gaussian, $\mathbf{p}_i \Omega = -i\hbar \cdot (-x_i) \Omega = i\hbar x_i \cdot \Omega$.

```
lemma momentumCLM_stdGaussian (i : Fin d) (x : Space d) :
  (p[d] i (stdGaussian (Space d)  $\mathbb{C}$ ) :  $\mathcal{S}(\text{Space } d, \mathbb{C})$ ) x =
  (I *  $\hbar$  * x i :  $\mathbb{C}$ ) * (stdGaussian (Space d)  $\mathbb{C}$  : Space d  $\rightarrow \mathbb{C}$ ) x
```

def diagEquiv[\[source\]](#)

The diagonal rescaling of Space d by the characteristic lengths, $(Q.\text{diagEquiv } x)_i = \xi_i x_i$. Composing the standard Gaussian with its inverse turns the standard Gaussian's isotropic width into the anisotropic width ξ_i required by a vacuum of Q , whether or not Q itself is isotropic (contrast `stdGaussian/hasVacuum_stdGaussian_of_forall_xi_eq_one` below, which only covers $\forall i, \xi_i = 1$).

```
noncomputable def diagEquiv (Q : HarmonicOscillator d) : Space d  $\approx$  L[ $\mathbb{R}$ ] Space d
```

def vacuumGaussian[\[source\]](#)

The vacuum Gaussian of Q : the anisotropic Gaussian $\exp(-\sum_i (x_i/\xi_i)^2/2)$, $Q.\text{diagEquiv}$ applied to the standard one. Equals `stdGaussian (Space d)` $\mathbb{C}$ exactly when every $\xi_i = 1$.

```
noncomputable def vacuumGaussian (Q : HarmonicOscillator d) :  $\mathcal{S}$ (Space d,  $\mathbb{C}$ )
```

lemma vacuumGaussian_apply[\[source\]](#)

```
lemma vacuumGaussian_apply (x : Space d) :  
(Q.vacuumGaussian : Space d  $\rightarrow$   $\mathbb{C}$ ) x = Real.exp (-2-1 * ||Q.diagEquiv.symm x|| ^ 2)
```

lemma diagEquiv_symm_basis[\[source\]](#)

$Q.\text{diagEquiv.symm}$ applied to a basis vector rescales it by $(\xi_i)^{-1}$.

```
lemma diagEquiv_symm_basis (i : Fin d) :  
Q.diagEquiv.symm (Space.basis i) = (Q. $\xi$  i)-1 • Space.basis i
```

lemma fderiv_norm_sq_diagEquiv_symm[\[source\]](#)

The Fréchet derivative of the anisotropic Gaussian's real exponent $\|Q.\text{diagEquiv.symm} \cdot\|^2$, at the CLM level (mirrors `fderiv_norm_sq_apply`, composed with $Q.\text{diagEquiv.symm}$).

```
lemma fderiv_norm_sq_diagEquiv_symm (x : Space d) :  
fderiv  $\mathbb{R}$  (fun x : Space d => ||Q.diagEquiv.symm x|| ^ 2) x =  
(2 • innerSL  $\mathbb{R}$  (Q.diagEquiv.symm x))  $\circ$  L (Q.diagEquiv.symm : Space d  $\rightarrow$  L[ $\mathbb{R}$ ] Space d)
```

lemma deriv_realGaussian_diag[\[source\]](#)

The real-valued anisotropic Gaussian's coordinate derivative, $\partial_i g = -(x_i/\xi_i^2) \cdot g$.

```
lemma deriv_realGaussian_diag (x : Space d) (i : Fin d) :  
Space.deriv i (fun x : Space d => Real.exp (-2-1 * ||Q.diagEquiv.symm x|| ^ 2)) x =  
-(x i / (Q. $\xi$  i) ^ 2) * Real.exp (-2-1 * ||Q.diagEquiv.symm x|| ^ 2)
```

lemma deriv_vacuumGaussian[\[source\]](#)

The coordinate derivative of the vacuum Gaussian, $\partial_i \Omega = -(x_i/\xi_i^2) \cdot \Omega$.

```
lemma deriv_vacuumGaussian (x : Space d) (i : Fin d) :  
Space.deriv i (fun x : Space d => (Q.vacuumGaussian : Space d  $\rightarrow$   $\mathbb{C}$ ) x) x =  
(-(x i / (Q. $\xi$  i) ^ 2 :  $\mathbb{R}$ ) :  $\mathbb{C}$ ) * (Q.vacuumGaussian : Space d  $\rightarrow$   $\mathbb{C}$ ) x
```

lemma momentumCLM_vacuumGaussian[\[source\]](#)

The momentum operator applied to the vacuum Gaussian, $\mathbf{p}_i \Omega = i\hbar(x_i/\xi_i^2) \cdot \Omega$.

```
lemma momentumCLM_vacuumGaussian (i : Fin d) (x : Space d) :
  (p[d] i (Q.vacuumGaussian) : S(Space d, ℂ)) x =
    (I * ħ * (x i / (Q.ξ i) ^ 2 : ℝ) : ℂ) * (Q.vacuumGaussian : Space d → ℂ) x
```

theorem annihilationCLM_vacuumGaussian[\[source\]](#)

Every annihilationCLM i kills the vacuum Gaussian, without an isotropy or unit-length assumption.

```
theorem annihilationCLM_vacuumGaussian (i : Fin d) :
  Q.annihilationCLM i (Q.vacuumGaussian) = 0
```

theorem hasVacuum_vacuumGaussian[\[source\]](#)

Q.vacuumGaussian is a vacuum for Q.toLadderSystem, without an isotropy assumption.

```
theorem hasVacuum_vacuumGaussian : Q.toLadderSystem.HasVacuum (Q.vacuumGaussian) where
```

lemma annihilationCLM_stdGaussian_of_xi_eq_one[\[source\]](#)

Every annihilationCLM i kills the standard Gaussian, provided that mode has unit characteristic length. Independent of hasVacuum_vacuumGaussian/annihilationCLM_vacuumGaussian above: the hypothesis here is genuinely weaker (unit length in mode i alone, not every mode), so this isn't a corollary of the general anisotropic result.

```
lemma annihilationCLM_stdGaussian_of_xi_eq_one (i : Fin d) (hξ : Q.ξ i = 1) :
  Q.annihilationCLM i (stdGaussian (Space d) ℂ) = 0
```

lemma hasVacuum_stdGaussian_of_forall_xi_eq_one[\[source\]](#)

stdGaussian (Space d) ℂ is a vacuum for Q.toLadderSystem when every mode has unit characteristic length.

```
lemma hasVacuum_stdGaussian_of_forall_xi_eq_one (hξ : ∀ i, Q.ξ i = 1) :
  Q.toLadderSystem.HasVacuum (stdGaussian (Space d) ℂ)
```

2.95 PhyslibAlpha.QuantumMechanics.StinespringDilation

[PhyslibAlpha/QuantumMechanics/StinespringDilation.lean](#) on GitHub.

def convexComb[\[source\]](#)

Density matrices are closed under real convex combinations.

```
def densityMatrix.convexComb {R : Type*} [RCLike R]
  {d : ℕ} (p₀ p₁ : densityMatrix (Fin d) (R := R)) {t : R}
  (hp₀ : 0 ≤ t) (hp₁ : 0 ≤ 1 - t) : densityMatrix (Fin d) (R := R)
```

def predAboveOfNe[\[source\]](#)

See discussion at <https://leanprover.zulipchat.com/#narrow/channel/217875-Is-there-code-for-X.3F/topic/succAbove.20and.20predAbove.20lemmas/with/584270574>

```
def Fin.predAboveOfNe {n : ℕ} {k i : Fin n}
  (h : i ≠ k) : Fin (n - 1)
```

lemma predAboveOfNe_injective[\[source\]](#)

A "missing lemma" for Fin types.

```
lemma Fin.predAboveOfNe_injective (n : ℕ) (k x y : Fin n)
  (hx : x ≠ k) (hy : y ≠ k)
  (heq : Fin.predAboveOfNe hx = Fin.predAboveOfNe hy) : x = y
```

def generalDilation[\[source\]](#)

This generalization of Stinespring dilation has the right "shape" but otherwise nothing specific to it.

```
def generalDilation {R : Type*}
  {m r : Type*} [DecidableEq r]
  (z : r)
  (S : Matrix (m × r) m R)
  (M : Matrix (m × r) (m × r) R) :
  Matrix (m × r) (m × r) R
```

def stinespringUnitaryFormE[\[source\]](#)

The Stinespring unitary form, general version.

```
def stinespringUnitaryFormE {R : Type*} [RCLike R] {m r : ℕ}
  {K : Fin r → Matrix (Fin m) (Fin m) R}
  (hK : ∑ i, (K i)H * K i = 1) (z : Fin r) (e : Matrix (Fin r) (Fin r) R)
  (p : Matrix (Fin m) (Fin m) R) :
  (Matrix (Fin m) (Fin m) R)
```

def stinespringGeneralFormE[\[source\]](#)

Even more general form of the Stinespring dilation.

```
def stinespringGeneralFormE {R : Type*} [RCLike R]
  {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
  (K : r → Matrix m m R) (z : r) (e : Matrix r r R)
  (M : Matrix (m × r) (m × r) R)
```

2.96 QuantumInfo.Channels.CPTP

[QuantumInfo/Channels/CPTP.lean](#) on GitHub.

theorem IsUnitary_iff_uConj[\[source\]](#)

A channel is unitary iff it can be written as conjugation by a unitary.

```
theorem IsUnitary_iff_uConj (Λ : CPTPMap dIn dIn) : IsUnitary Λ ↔ ∃ U, ∀ ρ, Λ ρ = ρ.uConj U
```

2.97 QuantumInfo.ForMathlib.IsMaximalSelfAdjoint

[QuantumInfo/ForMathlib/IsMaximalSelfAdjoint.lean](#) on GitHub.

instance instTrivialStarIsMaximalSelfAdjoint [\[source\]](#)

Every TrivialStar CommSemiring is its own maximal self adjoints.

```
instance instTrivialStarIsMaximalSelfAdjoint {R} [Star R] [TrivialStar R] [CommSemiring R] :
  IsMaximalSelfAdjoint R R where
```

instance instRCLikeIsMaximalSelfAdjoint [\[source\]](#)

$\mathbb{R}$ is the maximal self adjoint elements over RCLike

```
instance instRCLikeIsMaximalSelfAdjoint {α} [RCLike α] : IsMaximalSelfAdjoint ℝ α where
```

2.98 QuantumInfo.ForMathlib.LinearEquiv

[QuantumInfo/ForMathlib/LinearEquiv.lean](#) on GitHub.

def ofRelabel [\[source\]](#)

The R -linear equivalence $(d_2 \rightarrow R) \approx_l[R] (d \rightarrow R)$ that relabels the coordinates of a function along an index equivalence $e : d \approx d_2$. This is the linear-equivalence packaging of `Equiv.piCongrLeft`.

```
@[simps]
def ofRelabel (e : d ≈ d₂) : (d₂ → R) ≈l[R] (d → R)
```

def euclideanOfRelabel [\[source\]](#)

The $\mathbb{k}$ -linear equivalence `EuclideanSpace  $\mathbb{k}$  d₂ ≈l[ $\mathbb{k}$ ] EuclideanSpace  $\mathbb{k}$  d` that relabels the coordinates of a vector along an index equivalence $e : d \approx d_2$. This is the `EuclideanSpace` analogue of `LinearEquiv.ofRelabel`, obtained by transporting it across the `WithLp` identifications.

```
@[simps!]
def euclideanOfRelabel (e : d ≈ d₂) : EuclideanSpace ℝ d₂ ≈l[ℝ] EuclideanSpace ℝ d
```

theorem ofRelabel_refl [\[source\]](#)

```
@[simp]
theorem ofRelabel_refl : ofRelabel R (.refl d) = LinearEquiv.refl R (d → R)
```

theorem euclideanOfRelabel_refl [\[source\]](#)

```
@[simp]
theorem euclideanOfRelabel_refl : euclideanOfRelabel ℝ (.refl d) =
  LinearEquiv.refl ℝ (EuclideanSpace ℝ d)
```

2.99 QuantumInfo.Operators.Unitary

[QuantumInfo/Operators/Unitary.lean](#) on GitHub.

def uConj [source]

Conjugate a state by a unitary matrix (applying the unitary as an evolution).

```
def uConj (p : MState d) (U : U[d]) : MState d where
```

theorem uConj_spectrum_eq [source]

You might think this should only be true up to permutation, so that it would read like $\exists \sigma : \text{Equiv.Perm } d, (p.\text{uConj } U).\text{spectrum} = p.\text{spectrum.relabel } \sigma$. But since eigenvalues of a matrix are always canonically sorted, this is actually an equality.

```
@[simp]
theorem uConj_spectrum_eq (p : MState d) (U : U[d]) :
  (p.uConj U).spectrum = p.spectrum
```

theorem inner_uConj [source]

```
@[simp]
theorem inner_uConj (p σ : MState d) (U : U[d]) : ⟨⟨U ◁ p, U ◁ σ⟩⟩_Prob = ⟨⟨p, σ⟩⟩_Prob
```

2.100 scripts.APIMapIndex

[scripts/APIMapIndex.lean](#) on GitHub.

structure APIMapRequirement [source]

One entry of a map's Requirements list.

```
structure APIMapRequirement where
  description : String := ""
  done : Bool := false
  location : String := ""
```

structure APIMapData [source]

The contents of one API-map.yaml file.

```
structure APIMapData where
  path : System.FilePath
  version : String := ""
  title : String := ""
  overview : String := ""
  parentAPIs : List String := []
  references : List String := []
  requirements : List APIMapRequirement := []
```

def doneCount [source]

```
def APIMapData.doneCount (map : APIMapData) : Nat
```

def totalCount [source]

```
def APIMapData.totalCount (map : APIMapData) : Nat
```

def moduleContext[\[source\]](#)

The dotted module context of a map, e.g. Physlib.SpaceAndTime.Time.

```
def APIMapData.moduleContext (map : APIMapData) : String
```

def githubLink[\[source\]](#)

```
def APIMapData.githubLink (map : APIMapData) : String
```

def stripQuotes[\[source\]](#)

```
def stripQuotes (value : String) : String
```

def afterKey[\[source\]](#)

The scalar value of a line after its key: prefix.

```
def afterKey (line : String) (key : String) : String
```

def leadingSpaces[\[source\]](#)

```
def leadingSpaces (line : String) : Nat
```

def blockMarker[\[source\]](#)

some fold? when the scalar value opens a block: | literal (fold? is false) or > folded (fold? is true), with optional chomping indicator.

```
def blockMarker (value : String) : Option Bool
```

def finishBlock[\[source\]](#)

Assembles collected block-scalar lines: dedent to the minimum indent, drop trailing blank lines, and for a folded block join into one line.

```
def finishBlock (rawLines : List String) (fold : Bool) : String
```

inductive BlockTarget[\[source\]](#)

The block scalar currently being collected, if any.

```
inductive BlockTarget
| overview
| reqDescription
| reqLocation
```

inductive ListTarget[\[source\]](#)

The top-level list section the parser is inside, if any.

```
inductive ListTarget
| parentAPIs
| references
```

```
| requirements
| none
```

def parseAPIMap[\[source\]](#)

Reads one API-map.yaml. The parser is line-based and graceful: fields it cannot read are left at their defaults instead of raising errors.

```
def parseAPIMap (path : System.FilePath) : IO APIMapData
```

def findAPIMaps[\[source\]](#)

```
partial def findAPIMaps (directory : System.FilePath) : IO (List System.FilePath)
```

def yamlQuote[\[source\]](#)

A double-quoted YAML scalar.

```
def yamlQuote (value : String) : String
```

def yamlStringList[\[source\]](#)

A key with a list of quoted strings, or [] when empty.

```
def yamlStringList (key : String) (indent : String) (items : List String) : String
```

def yamlScalarOrBlock[\[source\]](#)

A key with a quoted scalar, or a | block when the value is multi-line. Block content is indented two spaces past indent.

```
def yamlScalarOrBlock (key : String) (indent : String) (value : String) : String
```

def APIMapRequirement.toYAML[\[source\]](#)

```
def APIMapRequirement.toYAML (requirement : APIMapRequirement) : String
```

def APIMapData.toYAML[\[source\]](#)

```
def APIMapData.toYAML (map : APIMapData) : String
```

def indexYAML[\[source\]](#)

```
def indexYAML (maps : List APIMapData) : String
```

2.101 scripts.get_cache

[scripts/get_cache.lean](#) on GitHub.

def helpText[\[source\]](#)

```
def helpText : String
```

def say[\[source\]](#)

println, then flush stdout immediately. Without this, messages printed before spawning a subprocess can sit in a buffer and appear out of order (or not at all until the child exits) whenever stdout is piped rather than a terminal – e.g. lake exe get_cache | tee log.txt.

```
def say (s : String) : IO Unit
```

def runStreamed[\[source\]](#)

Run a subprocess, inheriting stdout/stderr, with optional extra environment variables. Returns whether it exited successfully.

```
def runStreamed (cmd : String) (args : Array String)
  (env : Array (String × Option String) := #[]) : IO Bool
```

def knownFlags[\[source\]](#)

The options this program understands. Anything else is rejected up front, rather than silently ignored and treated as "no flags given".

```
def knownFlags : List String
```

def toolchainTag[\[source\]](#)

The current toolchain as a cache-scope path component: / and : become -, whitespace is dropped (matching the workflow's tr -d '[:space:]').

```
def toolchainTag : IO String
```

def scopeFor[\[source\]](#)

The cache scope for one half of the project. Each half gets its own so the two CI jobs do not overwrite each other's mappings; the toolchain is a path component because Lake ignores --toolchain for verbatim scopes. .github/workflows/publish-cache.yml builds the same strings.

```
def scopeFor (tc : String) (half : String) : String
```

2.102 scripts.PhyslibAlpha.alphaFileImports

[scripts/PhyslibAlpha/alphaFileImports.lean](#) on GitHub.

def extractModuleNameFromFilePath[\[source\]](#)

```
def extractModuleNameFromFilePath (path : FilePath) : String
```

def extractModuleNameFromImport[\[source\]](#)

```
def extractModuleNameFromImport (importString : String) : String
```

def checkAllFilesImported[\[source\]](#)

```
def checkAllFilesImported (directory : String) (mainFilePath : String) : (IO Bool)
```

2.103 scripts.PhyslibAlpha.noAlphaImports

[scripts/PhyslibAlpha/noAlphaImports.lean](#) on GitHub.

def areNoAlphaImports[\[source\]](#)

Returns True if not files in Physlib or QuantumInfo import import any PhyslibAlpha files, and False otherwise, printing the offending files and imports to the standard output.

```
def areNoAlphaImports (modules : List String) : IO Bool
```